

CSC-3TC33

Algorithmique et fondements de l'informatique

Langages - partie b

Bertrand Meyer

Automne 2024

Définition

Un **langage** est un ensemble de mots sur un alphabet, c'est-à-dire une partie de Σ^* .



Exemples de langage

- $\mathcal{L}_1 = \{\text{foo}, \text{bar}, \text{baz}\}$ sur l'alphabet $\Sigma = \{a, b, f, o, r, z\}$.
 $\mathcal{L}_1$ a pour cardinal 3.
- $\mathcal{L}_2$ formé des mots de longueur 42 sur l'alphabet $\Sigma = \{p, q, r\}$.
 $\mathcal{L}_2$ a pour cardinal 3^{42} .
- $\mathcal{L}_3 = \{d, dc, dcc, dccc, dcccc, \dots\} = \{dc^r : r \in \mathbb{N}\}$.
 $\mathcal{L}_3$ est de cardinal dénombrable.

Exemples de langage (suite)

Soit Σ un alphabet

- $\mathcal{L}_4 = \emptyset$.
 $\mathcal{L}_4$ a pour cardinal 0.
- $\mathcal{L}_5 = \{\varepsilon\}$.
 $\mathcal{L}_5$ a pour cardinal 1.
- $\mathcal{L}_6 = \{x\}$ où $x \in \Sigma$.
 $\mathcal{L}_6$ a pour cardinal 1.
- $\mathcal{L}_7$ formé des mots de longueur 1 sur Σ .
 $\mathcal{L}_7$ a pour cardinal $|\Sigma|$.

Mathématiquement, $\mathcal{L}_7$ s'identifie à Σ . Mais, caveat, un **char** et un **string** de longueur 1 ne sont pas toujours échangeables en informatique.

Les opérations sur les langages : union et intersection

Définition

Si $\mathcal{L}_1$ et $\mathcal{L}_2$ sont des langages, l'**union** $\mathcal{L}_1 \cup \mathcal{L}_2$ et l'**intersection** $\mathcal{L}_1 \cap \mathcal{L}_2$ sont des langages.

Exemple

Soit $\Sigma = \{a, b\}$ et

$$\mathcal{L}_1 = \{w \in \Sigma^*; |w| = 2\} \quad \mathcal{L}_2 = \{a^n; n \in \mathbb{N}\}.$$

Alors

$$\mathcal{L}_1 \cup \mathcal{L}_2 = \{\varepsilon, a, aa, ab, ba, bb, aaa, aaaa, aaaaa, aaaaaa, \dots\}.$$

$$\mathcal{L}_1 \cap \mathcal{L}_2 = \{aa\}.$$

Les opérations sur les langages : complémentaire

Définition

Si $\mathcal{L}$ est un langage, le **complémentaire** $\Sigma^* \setminus \mathcal{L}$ est un langage.

Exemple

Soit $\Sigma = \{a, b, c\}$ et

$$\mathcal{L} = \{a^n; n \in \mathbb{N}\} = \{w \in \Sigma^*; |w|_b = |w|_c = 0\}.$$

Alors

$$\mathcal{L}^c = \{w \in \Sigma^*; |w|_b + |w|_c \geq 1\}.$$

Les opérations sur les langages : concaténation

Définition

Si $\mathcal{L}_1$ et $\mathcal{L}_2$ sont des langages, la **concatenation**

$\mathcal{L}_1\mathcal{L}_2 = \{w_1w_2; w_1 \in \mathcal{L}_1, w_2 \in \mathcal{L}_2\}$ est un langage.

Exemple

Soit $\Sigma = \{a, b\}$ et

$$\mathcal{L}_1 = \{w \in \Sigma^*; |w| = 2\} \quad \mathcal{L}_2 = \{a^n; n \in \mathbb{N}\}.$$

Alors

$$\begin{aligned} \mathcal{L} = \mathcal{L}_1\mathcal{L}_2 = & \{aa, ab, ba, bb, \\ & aaa, aba, baa, bba, \\ & aaaa, abaa, baaa, bbaa, \dots\} \end{aligned}$$

$\mathcal{L}$ contient des mots de longueur ≥ 2 , dont les deux premières lettres sont quelconques et les suivantes valent toutes **a**.

Les opérations sur les langages : étoile de Kleene

Définition

Si $\mathcal{L}$ est un langage, l'étoile de Kleene

$$\mathcal{L}^* = \{w_1 w_2 \cdots w_r; r \in \mathbb{N} \text{ et } w_1, w_2, \dots, w_r \in \mathcal{L}\}$$

est un langage.

Exemple

Soit $\Sigma = \{a, b\}$ et $\mathcal{L} = \{aa, ab\}$. Alors

$$\begin{aligned} \mathcal{L}^* = & \{\varepsilon, aa, ab, \\ & aaaa, aaab, abaa, abab, \\ & aaaaaa, aaaaab, aaabaa, aaabaa, \\ & abaaaa, abaaab, ababaa, ababaa, \\ & \dots\} \end{aligned}$$

$\mathcal{L}$ contient des mots de longueur paire, dont les lettres d'indice impair valent a.

Les langages rationnels

Définition inductive des langages rationnels

Soit Σ un alphabet. L'ensemble des **langages rationnels** $Rat(\Sigma)$ est le sous-ensemble de l'ensemble des langages défini inductivement par les axiomes :

$$\frac{}{\emptyset \in Rat(\Sigma)} \text{ (Vide)}$$

$$\frac{}{\{\varepsilon\} \in Rat(\Sigma)} \text{ (Mot vide)}$$

$$\frac{}{\{x\} \in Rat(\Sigma)} \text{ (Symbole)}$$

où x est un symbole de Σ et par les règles

$$\frac{\mathcal{L}_1 \in Rat(\Sigma) \quad \mathcal{L}_2 \in Rat(\Sigma)}{\mathcal{L}_1 \cup \mathcal{L}_2 \in Rat(\Sigma)} \text{ (union)}$$

$$\frac{\mathcal{L}_1 \in Rat(\Sigma) \quad \mathcal{L}_2 \in Rat(\Sigma)}{\mathcal{L}_1 \mathcal{L}_2 \in Rat(\Sigma)} \text{ (concat)}$$

$$\frac{\mathcal{L} \in Rat(\Sigma)}{\mathcal{L}^* \in Rat(\Sigma)} \text{ (étoile)}$$

Les langages rationnels : reformulation

L'ensemble des langages rationnels est la plus petite collection de langages contenant :

- le langage vide $\emptyset$,
- le langage formé du mot vide $\{\varepsilon\}$,
- les langages formés d'un mot unique lettre $\{x\}$ avec $x \in \Sigma$

et stable par union, concatenation et étoile de Kleene.

Exemple 1 de langage rationnel

Soit $\Sigma = \{a, b, c, d\}$ un alphabet et $\mathcal{L}$ fini

$$\mathcal{L} = \{bac, ad\}.$$

Le langage $\mathcal{L}$ est-il rationnel?

Exemple 1 de langage rationnel

Soit $\Sigma = \{a, b, c, d\}$ un alphabet et $\mathcal{L}$ fini

$$\mathcal{L} = \{bac, ad\}.$$

Le langage $\mathcal{L}$ est-il rationnel? On a la dérivation

$$\frac{\frac{\frac{}{\{b\} \in Rat} (ax.)}{\{ba\} \in Rat(\Sigma)} \quad \frac{\frac{}{\{a\} \in Rat} (ax.)}{\{bac\} \in Rat(\Sigma)} (concat)}{\{bac, ad\} \in Rat(\Sigma)} \quad \frac{\frac{\frac{}{\{c\} \in Rat} (ax.)}{\{ad\} \in Rat(\Sigma)} \quad \frac{\frac{}{\{a\} \in Rat} (ax.) \quad \frac{}{\{d\} \in Rat} (ax.)}{\{ad\} \in Rat(\Sigma)} (concat)}{\{bac, ad\} \in Rat(\Sigma)} (union)$$

Exemple 1 de langage rationnel

Soit $\Sigma = \{a, b, c, d\}$ un alphabet et $\mathcal{L}$ fini

$$\mathcal{L} = \{bac, ad\}.$$

Le langage $\mathcal{L}$ est-il rationnel? On a la dérivation

$$\frac{\frac{\frac{}{\{b\} \in Rat} (ax.)}{\{ba\} \in Rat(\Sigma)} \quad \frac{\frac{}{\{a\} \in Rat} (ax.)}{\{bac\} \in Rat(\Sigma)} (concat)}{\{bac, ad\} \in Rat(\Sigma)} \quad \frac{\frac{\frac{}{\{c\} \in Rat} (ax.)}{\{ad\} \in Rat(\Sigma)} \quad \frac{\frac{}{\{a\} \in Rat} (ax.) \quad \frac{}{\{d\} \in Rat} (ax.)}{\{ad\} \in Rat(\Sigma)} (concat)}{\{bac, ad\} \in Rat(\Sigma)} (union)$$

Le langage $\mathcal{L}$ est rationnel!

Exemple 2 de langage rationnel

Soit $\Sigma = \{a, b, c, d\}$ un alphabet et $\mathcal{L}$ fini

$$\mathcal{L} = \{dc^n; n \in \mathbb{N}\}.$$

Le langage $\mathcal{L}$ est-il rationnel?

Exemple 2 de langage rationnel

Soit $\Sigma = \{a, b, c, d\}$ un alphabet et $\mathcal{L}$ fini

$$\mathcal{L} = \{dc^n; n \in \mathbb{N}\}.$$

Le langage $\mathcal{L}$ est-il rationnel? On a la dérivation

$$\frac{\frac{}{\{d\} \in \text{Rat}(\Sigma)} \text{ (ax.)} \quad \frac{\frac{}{\{c\} \in \text{Rat}(\Sigma)} \text{ (ax.)} \quad \frac{}{\{c^n; n \in \mathbb{N}\} \in \text{Rat}(\Sigma)} \text{ (étoile)}}{\{dc^n; n \in \mathbb{N}\} \in \text{Rat}(\Sigma)} \text{ (concat)}$$

Exemple 2 de langage rationnel

Soit $\Sigma = \{a, b, c, d\}$ un alphabet et $\mathcal{L}$ fini

$$\mathcal{L} = \{dc^n; n \in \mathbb{N}\}.$$

Le langage $\mathcal{L}$ est-il rationnel? On a la dérivation

$$\frac{\frac{}{\{d\} \in \text{Rat}(\Sigma)} \text{ (ax.)} \quad \frac{\frac{}{\{c\} \in \text{Rat}(\Sigma)} \text{ (ax.)} \quad \frac{}{\{c^n; n \in \mathbb{N}\} \in \text{Rat}(\Sigma)} \text{ (étoile)}}{\{dc^n; n \in \mathbb{N}\} \in \text{Rat}(\Sigma)} \text{ (concat)}$$

Le langage $\mathcal{L}$ est rationnel!

Exemple 3 de langage rationnel

Soit $\Sigma = \{a, b\}$ un alphabet et $\mathcal{L}$ le langage des mots sur Σ de longueur impaire

$$\mathcal{L} = \{w \in \Sigma^*; |w| \text{ impaire}\}.$$

Le langage $\mathcal{L}$ est-il rationnel?

Exemple 3 de langage rationnel

Soit $\Sigma = \{a, b\}$ un alphabet et $\mathcal{L}$ le langage des mots sur Σ de longueur impaire

$$\mathcal{L} = \{w \in \Sigma^*; |w| \text{ impaire}\}.$$

Le langage $\mathcal{L}$ est-il rationnel? On a

$$\mathcal{L} = \{a, b\}\{aa, ab, ba, bb\}^*$$

et

Exemple 3 de langage rationnel

Soit $\Sigma = \{a, b\}$ un alphabet et $\mathcal{L}$ le langage des mots sur Σ de longueur impaire

$$\mathcal{L} = \{w \in \Sigma^*; |w| \text{ impaire}\}.$$

Le langage $\mathcal{L}$ est-il rationnel? On a

$$\mathcal{L} = \{a, b\}\{aa, ab, ba, bb\}^*$$

et la dérivation

$$\frac{\frac{\frac{}{\{a\}} \text{ (ax.)} \quad \frac{}{\{b\}} \text{ (ax.)}}{\{a, b\} \in \text{Rat}(\Sigma)} \text{ (union)} \quad \frac{\frac{\frac{}{\{a\}} \text{ (ax.)} \quad \frac{}{\{b\}} \text{ (ax.)}}{\{a, b\} \in \text{Rat}(\Sigma)} \text{ (union)} \quad \frac{}{\{aa, ab, ba, bb\} \in \text{Rat}(\Sigma)} \text{ (concat)}}{\frac{}{\{aa, ab, ba, bb\}^* \in \text{Rat}(\Sigma)} \text{ (étoile)}} \text{ (concat)} \\ \mathcal{L} = \{a, b\}\{aa, ab, ba, bb\}^* \in \text{Rat}(\Sigma)$$

Exemple 3 de langage rationnel

Soit $\Sigma = \{a, b\}$ un alphabet et $\mathcal{L}$ le langage des mots sur Σ de longueur impaire

$$\mathcal{L} = \{w \in \Sigma^*; |w| \text{ impaire}\}.$$

Le langage $\mathcal{L}$ est-il rationnel? On a

$$\mathcal{L} = \{a, b\}\{aa, ab, ba, bb\}^*$$

et la dérivation

$$\frac{\frac{\frac{}{\{a\}} \text{ (ax.)} \quad \frac{}{\{b\}} \text{ (ax.)}}{\{a, b\} \in \text{Rat}(\Sigma)} \text{ (union)} \quad \frac{\frac{\frac{\frac{}{\{a\}} \text{ (ax.)} \quad \frac{}{\{b\}} \text{ (ax.)}}{\{a, b\} \in \text{Rat}(\Sigma)} \text{ (union)} \quad \frac{}{\{aa, ab, ba, bb\} \in \text{Rat}(\Sigma)} \text{ (concat)}}{\{aa, ab, ba, bb\}^* \in \text{Rat}(\Sigma)} \text{ (étoile)}}{\mathcal{L} = \{a, b\}\{aa, ab, ba, bb\}^* \in \text{Rat}(\Sigma)} \text{ (concat)}$$

Le langage $\mathcal{L}$ est rationnel!

Définition inductive des expressions régulières

Soit Σ un alphabet. L'ensemble des **expressions régulières** $RegExp(\Sigma)$ est le langage sur $\Sigma \cup \{\perp, \underline{\epsilon}, (,), *, |\}$ défini inductivement par les axiomes :

$$\frac{}{\perp \in RegExp(\Sigma)} \text{ (Vide)} \qquad \frac{}{\underline{\epsilon} \in RegExp(\Sigma)} \text{ (Mot vide)} \qquad \frac{}{x \in RegExp(\Sigma)} \text{ (Symbole)}$$

où x est un symbole de Σ et par les règles

$$\frac{r_1 \in RegExp(\Sigma) \quad r_2 \in RegExp(\Sigma)}{(r_1 | r_2) \in RegExp(\Sigma)} \text{ (union)} \qquad \frac{r_1 \in RegExp(\Sigma) \quad r_2 \in RegExp(\Sigma)}{r_1 r_2 \in RegExp(\Sigma)} \text{ (concat)}$$
$$\frac{r \in RegExp(\Sigma)}{(r)^* \in RegExp(\Sigma)} \text{ (étoile)}$$

Dénotation d'un langage rationnel par un expression régulière

Soit r une expression régulière, on appelle **langage rationnel dénoté par r** le langage $\mathcal{L}(r)$ défini par

$$\frac{}{\mathcal{L}(\perp) = \emptyset} \text{ (Vide)}$$

$$\frac{}{\mathcal{L}(\varepsilon) = \{\varepsilon\}} \text{ (Mot vide)}$$

$$\frac{}{\mathcal{L}(x) = \{x\}} \text{ (Symbole)}$$

où x est un symbole de Σ et par

$$\frac{r_1 \in \text{RegExp}(\Sigma) \quad r_2 \in \text{RegExp}(\Sigma)}{\mathcal{L}((r_1 | r_2)) = \mathcal{L}(r_1) \cup \mathcal{L}(r_2)} \text{ (union)} \quad \frac{r_1 \in \text{RegExp}(\Sigma) \quad r_2 \in \text{RegExp}(\Sigma)}{\mathcal{L}(r_1 r_2) = \mathcal{L}(r_1) \mathcal{L}(r_2)} \text{ (concat)}$$

$$\frac{r \in \text{RegExp}(\Sigma)}{\mathcal{L}((r)^\star) = \mathcal{L}(r)^\star} \text{ (étoile)}$$

Exemples de langages dénotés par une regexp

$$r_1 = (a|b)$$

Exemples de langages dénotés par une regexp

$$r_1 = (a|b) \qquad \mathcal{L}(r_1) = \{a, b\}$$

Exemples de langages dénotés par une regexp

$$r_1 = (a|b) \qquad \mathcal{L}(r_1) = \{a, b\}$$

$$r_2 = (a|b)c$$

Exemples de langages dénotés par une regexp

$$\begin{array}{ll} r_1 = (a|b) & \mathcal{L}(r_1) = \{a, b\} \\ r_2 = (a|b)c & \mathcal{L}(r_2) = \{ac, bc\} \end{array}$$

Exemples de langages dénotés par une regexp

$$r_1 = (a|b)$$

$$\mathcal{L}(r_1) = \{a, b\}$$

$$r_2 = (a|b)c$$

$$\mathcal{L}(r_2) = \{ac, bc\}$$

$$r_3 = (bc)^*$$

Exemples de langages dénotés par une regexp

$$r_1 = (a|b)$$

$$\mathcal{L}(r_1) = \{a, b\}$$

$$r_2 = (a|b)c$$

$$\mathcal{L}(r_2) = \{ac, bc\}$$

$$r_3 = (bc)^*$$

$$\mathcal{L}(r_3) = \{\varepsilon, bc, bcbc, bcbcbc, \dots\}$$

Exemples de langages dénotés par une regexp

$$r_1 = (a|b) \qquad \mathcal{L}(r_1) = \{a, b\}$$

$$r_2 = (a|b)c \qquad \mathcal{L}(r_2) = \{ac, bc\}$$

$$r_3 = (bc)^* \qquad \mathcal{L}(r_3) = \{\varepsilon, bc, bcbc, bcbcbc, \dots\}$$

$$r_4 = (a|(bc)^*)$$

Exemples de langages dénotés par une regexp

$$r_1 = (a|b) \quad \mathcal{L}(r_1) = \{a, b\}$$

$$r_2 = (a|b)c \quad \mathcal{L}(r_2) = \{ac, bc\}$$

$$r_3 = (bc)^* \quad \mathcal{L}(r_3) = \{\varepsilon, bc, bc bc, bc bc bc, \dots\}$$

$$r_4 = (a|(bc)^*) \quad \mathcal{L}(r_4) = \{\varepsilon, a, bc, bc bc, bc bc bc, \dots\}$$

Exemples de langages dénotés par une regexp

$$r_1 = (a|b) \quad \mathcal{L}(r_1) = \{a, b\}$$

$$r_2 = (a|b)c \quad \mathcal{L}(r_2) = \{ac, bc\}$$

$$r_3 = (bc)^* \quad \mathcal{L}(r_3) = \{\varepsilon, bc, bc bc, bc bc bc, \dots\}$$

$$r_4 = (a|(bc)^*) \quad \mathcal{L}(r_4) = \{\varepsilon, a, bc, bc bc, bc bc bc, \dots\}$$

$$r_5 = (a|(bc)^*)d$$

Exemples de langages dénotés par une regexp

$$r_1 = (a|b) \quad \mathcal{L}(r_1) = \{a, b\}$$

$$r_2 = (a|b)c \quad \mathcal{L}(r_2) = \{ac, bc\}$$

$$r_3 = (bc)^* \quad \mathcal{L}(r_3) = \{\varepsilon, bc, bc bc, bc bc bc, \dots\}$$

$$r_4 = (a|(bc)^*) \quad \mathcal{L}(r_4) = \{\varepsilon, a, bc, bc bc, bc bc bc, \dots\}$$

$$r_5 = (a|(bc)^*)d \quad \mathcal{L}(r_5) = \{d, ad, bcd, bc bcd, bc bc bcd, \dots\}$$

Exemples de langages dénotés par une regexp

$$r_1 = (a|b) \quad \mathcal{L}(r_1) = \{a, b\}$$

$$r_2 = (a|b)c \quad \mathcal{L}(r_2) = \{ac, bc\}$$

$$r_3 = (bc)^* \quad \mathcal{L}(r_3) = \{\varepsilon, bc, bc bc, bc bc bc, \dots\}$$

$$r_4 = (a|(bc)^*) \quad \mathcal{L}(r_4) = \{\varepsilon, a, bc, bc bc, bc bc bc, \dots\}$$

$$r_5 = (a|(bc)^*)d \quad \mathcal{L}(r_5) = \{d, ad, bcd, bc bcd, bc bc bcd, \dots\}$$

$$r_6 = \perp d$$

Exemples de langages dénotés par une regexp

$$r_1 = (a|b) \quad \mathcal{L}(r_1) = \{a, b\}$$

$$r_2 = (a|b)c \quad \mathcal{L}(r_2) = \{ac, bc\}$$

$$r_3 = (bc)^* \quad \mathcal{L}(r_3) = \{\varepsilon, bc, bcbc, bcbcbc, \dots\}$$

$$r_4 = (a|(bc)^*) \quad \mathcal{L}(r_4) = \{\varepsilon, a, bc, bcbc, bcbcbc, \dots\}$$

$$r_5 = (a|(bc)^*)d \quad \mathcal{L}(r_5) = \{d, ad, bcd, bcbcd, bcbcbcd, \dots\}$$

$$r_6 = \perp d \quad \mathcal{L}(r_6) = \emptyset$$

Exemples de langages dénotés par une regexp

$$r_1 = (a|b) \quad \mathcal{L}(r_1) = \{a, b\}$$

$$r_2 = (a|b)c \quad \mathcal{L}(r_2) = \{ac, bc\}$$

$$r_3 = (bc)^* \quad \mathcal{L}(r_3) = \{\varepsilon, bc, bc bc, bc bc bc, \dots\}$$

$$r_4 = (a|(bc)^*) \quad \mathcal{L}(r_4) = \{\varepsilon, a, bc, bc bc, bc bc bc, \dots\}$$

$$r_5 = (a|(bc)^*)d \quad \mathcal{L}(r_5) = \{d, ad, bcd, bc bcd, bc bc bcd, \dots\}$$

$$r_6 = \perp \quad \mathcal{L}(r_6) = \emptyset$$

$$r_7 = \perp | d$$

Exemples de langages dénotés par une regexp

$$r_1 = (a|b) \quad \mathcal{L}(r_1) = \{a, b\}$$

$$r_2 = (a|b)c \quad \mathcal{L}(r_2) = \{ac, bc\}$$

$$r_3 = (bc)^* \quad \mathcal{L}(r_3) = \{\varepsilon, bc, bcbc, bcbcbc, \dots\}$$

$$r_4 = (a|(bc)^*) \quad \mathcal{L}(r_4) = \{\varepsilon, a, bc, bcbc, bcbcbc, \dots\}$$

$$r_5 = (a|(bc)^*)d \quad \mathcal{L}(r_5) = \{d, ad, bcd, bcbcd, bcbcbcd, \dots\}$$

$$r_6 = \perp \quad \mathcal{L}(r_6) = \emptyset$$

$$r_7 = \perp | d \quad \mathcal{L}(r_7) = \{d\}$$

Exemples de langages dénotés par une regexp

$$r_1 = (a|b) \quad \mathcal{L}(r_1) = \{a, b\}$$

$$r_2 = (a|b)c \quad \mathcal{L}(r_2) = \{ac, bc\}$$

$$r_3 = (bc)^* \quad \mathcal{L}(r_3) = \{\varepsilon, bc, bcbc, bc bcb c, \dots\}$$

$$r_4 = (a|(bc)^*) \quad \mathcal{L}(r_4) = \{\varepsilon, a, bc, bc b c, bc b c b c, \dots\}$$

$$r_5 = (a|(bc)^*)d \quad \mathcal{L}(r_5) = \{d, ad, bcd, bc b c d, bc b c b c d, \dots\}$$

$$r_6 = \perp d \quad \mathcal{L}(r_6) = \emptyset$$

$$r_7 = \perp | d \quad \mathcal{L}(r_7) = \{d\}$$

$$r_8 = \underline{\varepsilon} d$$

Exemples de langages dénotés par une regexp

$r_1 = (a b)$	$\mathcal{L}(r_1) = \{a, b\}$
$r_2 = (a b)c$	$\mathcal{L}(r_2) = \{ac, bc\}$
$r_3 = (bc)^*$	$\mathcal{L}(r_3) = \{\varepsilon, bc, bcbc, bcbcbc, \dots\}$
$r_4 = (a (bc)^*)$	$\mathcal{L}(r_4) = \{\varepsilon, a, bc, bcbc, bcbcbc, \dots\}$
$r_5 = (a (bc)^*)d$	$\mathcal{L}(r_5) = \{d, ad, bcd, bcbcd, bcbcbcd, \dots\}$
$r_6 = \perp d$	$\mathcal{L}(r_6) = \emptyset$
$r_7 = \perp d$	$\mathcal{L}(r_7) = \{d\}$
$r_8 = \underline{\varepsilon} d$	$\mathcal{L}(r_8) = \{d\}$

Exemples de langages dénotés par une regexp

$r_1 = (a b)$	$\mathcal{L}(r_1) = \{a, b\}$
$r_2 = (a b)c$	$\mathcal{L}(r_2) = \{ac, bc\}$
$r_3 = (bc)^*$	$\mathcal{L}(r_3) = \{\varepsilon, bc, bc bc, bc bc bc, \dots\}$
$r_4 = (a (bc)^*)$	$\mathcal{L}(r_4) = \{\varepsilon, a, bc, bc bc, bc bc bc, \dots\}$
$r_5 = (a (bc)^*)d$	$\mathcal{L}(r_5) = \{d, ad, bcd, bc bcd, bc bc bcd, \dots\}$
$r_6 = \perp d$	$\mathcal{L}(r_6) = \emptyset$
$r_7 = \perp d$	$\mathcal{L}(r_7) = \{d\}$
$r_8 = \underline{\varepsilon} d$	$\mathcal{L}(r_8) = \{d\}$
$r_9 = \underline{\varepsilon} d$	

Exemples de langages dénotés par une regexp

$r_1 = (a b)$	$\mathcal{L}(r_1) = \{a, b\}$
$r_2 = (a b)c$	$\mathcal{L}(r_2) = \{ac, bc\}$
$r_3 = (bc)^*$	$\mathcal{L}(r_3) = \{\varepsilon, bc, bcbc, bcbcbc, \dots\}$
$r_4 = (a (bc)^*)$	$\mathcal{L}(r_4) = \{\varepsilon, a, bc, bcbc, bcbcbc, \dots\}$
$r_5 = (a (bc)^*)d$	$\mathcal{L}(r_5) = \{d, ad, bcd, bcbcd, bcbcbcd, \dots\}$
$r_6 = \perp d$	$\mathcal{L}(r_6) = \emptyset$
$r_7 = \perp d$	$\mathcal{L}(r_7) = \{d\}$
$r_8 = \underline{\varepsilon} d$	$\mathcal{L}(r_8) = \{d\}$
$r_9 = \underline{\varepsilon} d$	$\mathcal{L}(r_9) = \{\varepsilon, d\}$

À retenir

Les **expressions rationnelles** (ou regexp) sont une convention syntaxique pour parler des langages rationnels.

Pour bien comprendre

À retenir

Les **expressions rationnelles** (ou regexp) sont une convention syntaxique pour parler des langages rationnels.

Dans la vraie vie

Les outils de manipulation de texte (par exemple **grep**) disposent de **moteurs d'expressions régulières**.

Ces outils permettent de retrouver une chaîne de caractère à partir d'un motif spécifié sous forme d'expression régulière.

Attention, en général, ces moteurs disposent de fonctionnalités enrichies qui permettent des recherches hors du champ des expressions régulières au sens strict.

Langages reconnaissables par un automate fini

Présentation générale des automates

Un **automate** est un modèle de calcul simple qui ne dispose que d'une mémoire finie (**état** dans lequel on se trouve).

Présentation générale des automates

Un **automate** est un modèle de calcul simple qui ne dispose que d'une mémoire finie (**état** dans lequel on se trouve).

Ce sont des machines abstraites qui savent reconnaître ou non l'**appartenance à un langage**.

Présentation générale des automates

Un **automate** est un modèle de calcul simple qui ne dispose que d'une mémoire finie (**état** dans lequel on se trouve).

Ce sont des machines abstraites qui savent reconnaître ou non l'**appartenance à un langage**.

On présente **4 variantes**, qui ont la même puissance de calcul et qui reprennent en généralisant les idées rencontrées dans les algorithmes de Knuth Morris Pratt et d'Aho Corasick.

Automates finis déterministes complets DFA

Définition

Un **automate fini déterministe (complet)** est un quintuplet

$(\Sigma, Q, q_0, \delta, F)$ où

Définition

Un **automate fini déterministe (complet)** est un quintuplet $(\Sigma, Q, q_0, \delta, F)$ où

- Σ est un **alphabet**

Définition

Un **automate fini déterministe (complet)** est un quintuplet $(\Sigma, Q, q_0, \delta, F)$ où

- Σ est un **alphabet**
- Q est un ensemble fini, appelé ensemble d'**états**

Définition

Un **automate fini déterministe (complet)** est un quintuplet $(\Sigma, Q, q_0, \delta, F)$ où

- Σ est un **alphabet**
- Q est un ensemble fini, appelé ensemble d'**états**
- $q_0 \in Q$ est un état particulier, appelé **état initial**

Définition

Un **automate fini déterministe (complet)** est un quintuplet $(\Sigma, Q, q_0, \delta, F)$ où

- Σ est un **alphabet**
- Q est un ensemble fini, appelé ensemble d'**états**
- $q_0 \in Q$ est un état particulier, appelé **état initial**
- $\delta : Q \times \Sigma \rightarrow Q$ est une fonction, dite **fonction de transition**.

Définition

Un **automate fini déterministe (complet)** est un quintuplet $(\Sigma, Q, q_0, \delta, F)$ où

- Σ est un **alphabet**
- Q est un ensemble fini, appelé ensemble d'**états**
- $q_0 \in Q$ est un état particulier, appelé **état initial**
- $\delta : Q \times \Sigma \rightarrow Q$ est une fonction, dite **fonction de transition**.
- $F \subseteq Q$ est un ensemble d'**états** dits **finaux** ou acceptants

Exemple de DFA

Exemple de DFA

On fixe $\Sigma = \{a, b\}$

Exemple de DFA

On fixe $\Sigma = \{a, b\}$, $Q = \{z, u, d\}$



Exemple de DFA

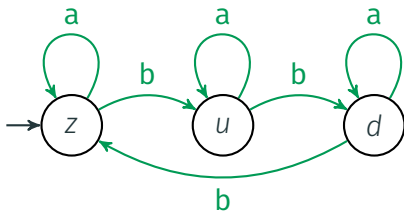
On fixe $\Sigma = \{a, b\}$, $Q = \{z, u, d\}$, $q_0 = z$.



Exemple de DFA

On fixe $\Sigma = \{a, b\}$, $Q = \{z, u, d\}$, $q_0 = z$. On définit $\delta : Q \times \Sigma \rightarrow Q$ par

$$\begin{array}{lll} \delta(z, a) = z & \delta(u, a) = u & \delta(d, a) = d \\ \delta(z, b) = u & \delta(u, b) = d & \delta(d, b) = z \end{array}$$

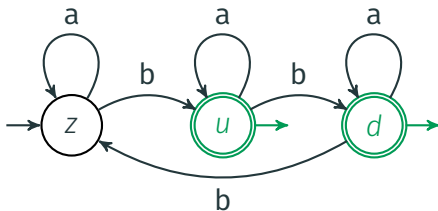


Exemple de DFA

On fixe $\Sigma = \{a, b\}$, $Q = \{z, u, d\}$, $q_0 = z$. On définit $\delta : Q \times \Sigma \rightarrow Q$ par

$$\begin{array}{lll} \delta(z, a) = z & \delta(u, a) = u & \delta(d, a) = d \\ \delta(z, b) = u & \delta(u, b) = d & \delta(d, b) = z \end{array}$$

et on pose $F = \{u, d\}$.

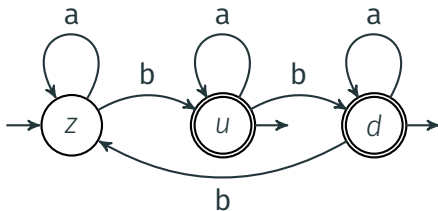


Exemple de DFA

On fixe $\Sigma = \{a, b\}$, $Q = \{z, u, d\}$, $q_0 = z$. On définit $\delta : Q \times \Sigma \rightarrow Q$ par

$$\begin{aligned}\delta(z, a) &= z & \delta(u, a) &= u & \delta(d, a) &= d \\ \delta(z, b) &= u & \delta(u, b) &= d & \delta(d, b) &= z\end{aligned}$$

et on pose $F = \{u, d\}$.



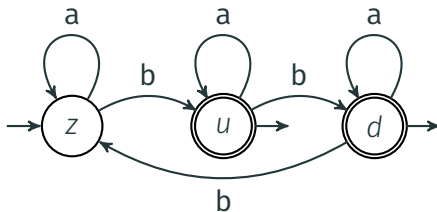
On étend δ en une fonction $\delta^* : Q \times \Sigma^* \rightarrow Q$ par

$$\delta^*(q, x_1 x_2 \cdots x_n) = \begin{cases} q & \text{si } n = 0 \\ \delta^*(\delta(q, x_1), x_2 \cdots x_n) & \text{sinon} \end{cases}$$

On se déplace entre états en consommant des lettres.

Exemple de calcul

Calculons $\delta^*(q_0, \text{abbabab})$ (où $q_0 = z$).



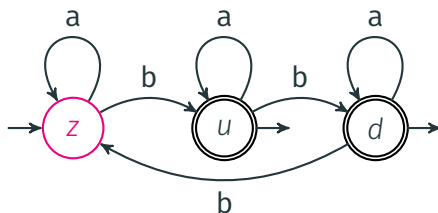
Exemple de calcul

Calculons $\delta^*(q_0, \text{abbabab})$ (où $q_0 = z$).

On a

$$\delta^*(q_0, \text{abbabab}) = \delta^*(z, \text{abbabab})$$

abbabab



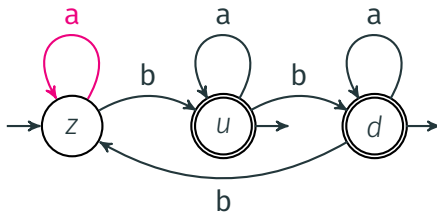
Exemple de calcul

Calculons $\delta^*(q_0, \text{abbabab})$ (où $q_0 = z$).

On a

$$\delta^*(q_0, \text{abbabab}) = \delta^*(\delta(z, a), \text{bbabab})$$

abbabab



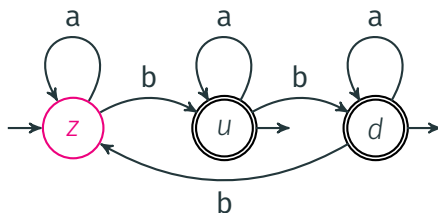
Exemple de calcul

Calculons $\delta^*(q_0, \text{abbabab})$ (où $q_0 = z$).

On a

$$\delta^*(q_0, \text{abbabab}) = \delta^*(z, \text{bbabab})$$

abbabab



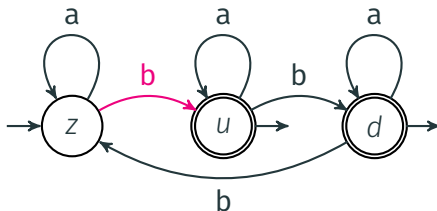
Exemple de calcul

Calculons $\delta^*(q_0, \text{abbabab})$ (où $q_0 = z$).

On a

$$\delta^*(q_0, \text{abbabab}) = \delta^*(\delta(z, b), \text{babab})$$

a**b**ababab



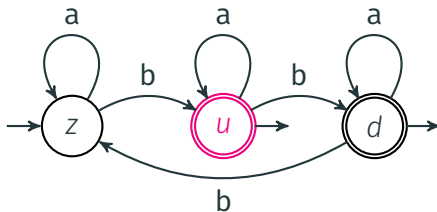
Exemple de calcul

Calculons $\delta^*(q_0, \text{abbabab})$ (où $q_0 = z$).

On a

$$\delta^*(q_0, \text{abbabab}) = \delta^*(u, \text{babab})$$

abbabab



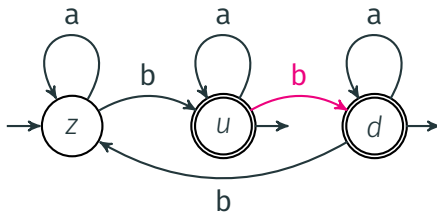
Exemple de calcul

Calculons $\delta^*(q_0, \text{abbabab})$ (où $q_0 = z$).

On a

$$\delta^*(q_0, \text{abbabab}) = \delta^*(\delta(u, b), \text{abab})$$

ab**b**abab



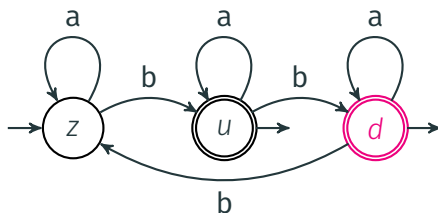
Exemple de calcul

Calculons $\delta^*(q_0, \text{abbabab})$ (où $q_0 = z$).

On a

$$\delta^*(q_0, \text{abbabab}) = \delta^*(d, \text{abab})$$

abbabab



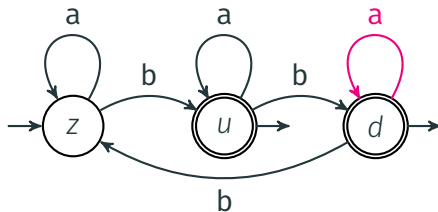
Exemple de calcul

Calculons $\delta^*(q_0, \text{abbabab})$ (où $q_0 = z$).

On a

$$\delta^*(q_0, \text{abbabab}) = \delta^*(\delta(d, a), \text{bab})$$

abb**a**bab



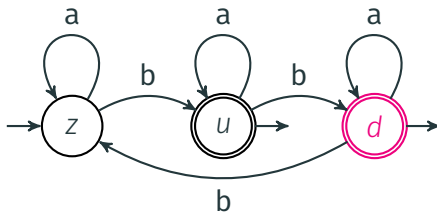
Exemple de calcul

Calculons $\delta^*(q_0, \text{abbabab})$ (où $q_0 = z$).

On a

$$\delta^*(q_0, \text{abbabab}) = \delta^*(d, \text{bab})$$

abbabab



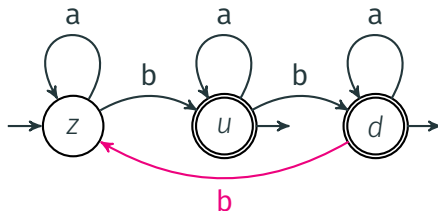
Exemple de calcul

Calculons $\delta^*(q_0, \text{abbabab})$ (où $q_0 = z$).

On a

$$\delta^*(q_0, \text{abbabab}) = \delta^*(\delta(d, b), ab)$$

abba**b**ab



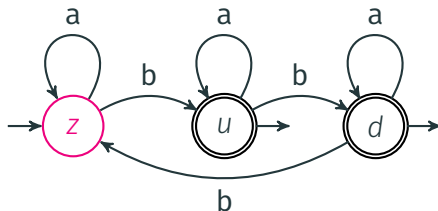
Exemple de calcul

Calculons $\delta^*(q_0, \text{abbabab})$ (où $q_0 = z$).

On a

$$\delta^*(q_0, \text{abbabab}) = \delta^*(z, \text{ab})$$

abbabab



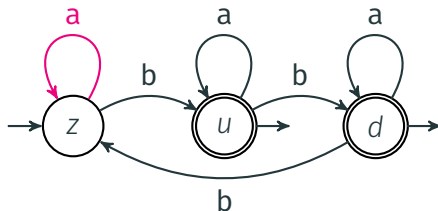
Exemple de calcul

Calculons $\delta^*(q_0, \text{abbabab})$ (où $q_0 = z$).

On a

$$\delta^*(q_0, \text{abbabab}) = \delta^*(\delta(z, a), b)$$

abbab**a**b



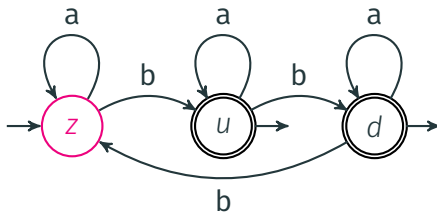
Exemple de calcul

Calculons $\delta^*(q_0, \text{abbabab})$ (où $q_0 = z$).

On a

$$\delta^*(q_0, \text{abbabab}) = \delta^*(z, \text{b})$$

abbabab



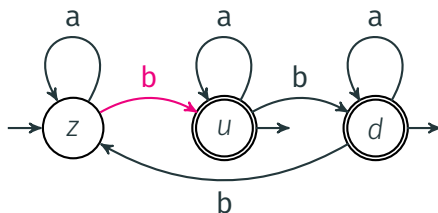
Exemple de calcul

Calculons $\delta^*(q_0, \text{abbabab})$ (où $q_0 = z$).

On a

$$\delta^*(q_0, \text{abbabab}) = \delta^*(\delta(z, b), \varepsilon)$$

abbabab



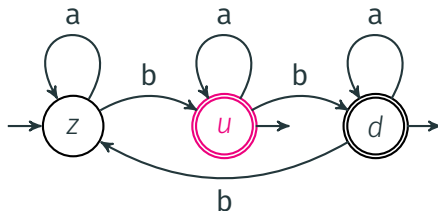
Exemple de calcul

Calculons $\delta^*(q_0, \text{abbabab})$ (où $q_0 = z$).

On a

$$\delta^*(q_0, \text{abbabab}) = \delta^*(u, \varepsilon)$$

abbabab



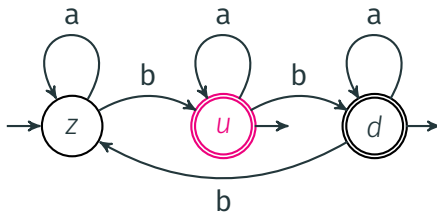
Exemple de calcul

Calculons $\delta^*(q_0, \text{abbabab})$ (où $q_0 = z$).

On a

$$\delta^*(q_0, \text{abbabab}) = u$$

abbabab



Mots acceptés (ou reconnus)

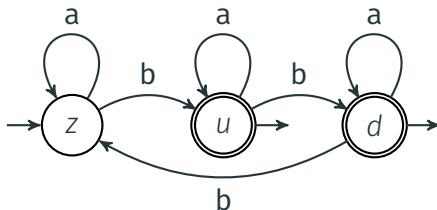
Définition

On dit qu'un mot $w \in \Sigma^*$ est **accepté** ou **reconnu** par l'automate si le calcul conduit de l'état initial à un état final, autrement dit si

$$\delta^*(q_0, w) \in F.$$

Exemple de mot accepté

Le mot $w = \text{abbabab}$ est accepté par l'automate



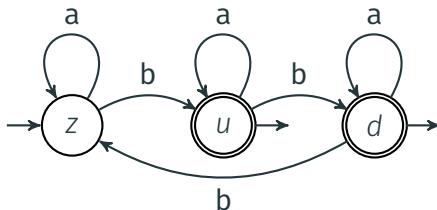
En effet,

$$\delta^*(z, w) = u$$

est un état final.

Exemple de mot accepté

Le mot $w = \text{baababaa}$ est / n'est pas (?) accepté par l'automate

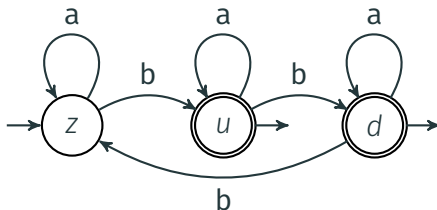


En effet,

$$\delta^*(z, w) = ???$$

Exemple de mot accepté

Le mot $w = \text{baababaa}$ n'est pas accepté par l'automate



En effet,

$$\delta^*(z, w) = z$$

n'est pas un état final.

Définition

L'ensemble $\mathcal{L}(\mathcal{A})$ des mots reconnus par un automate $\mathcal{A}$ s'appelle le langage accepté, ou reconnu, par l'automate $\mathcal{A}$.

Définition

L'ensemble $\mathcal{L}(\mathcal{A})$ des mots reconnus par un automate $\mathcal{A}$ s'appelle le **langage accepté**, ou **reconnu**, par l'automate $\mathcal{A}$.

Définition

Un langage $\mathcal{L}$ est dit **reconnaissable** s'il existe un automate $\mathcal{A}$ tel que $\mathcal{L} = \mathcal{L}(\mathcal{A})$.

Langage reconnu par un automate

Définition

L'ensemble $\mathcal{L}(\mathcal{A})$ des mots reconnus par un automate $\mathcal{A}$ s'appelle le **langage accepté**, ou **reconnu**, par l'automate $\mathcal{A}$.

Définition

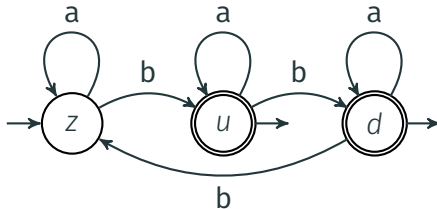
Un langage $\mathcal{L}$ est dit **reconnaissable** s'il existe un automate $\mathcal{A}$ tel que $\mathcal{L} = \mathcal{L}(\mathcal{A})$.

Définition

On dit que deux automates finis déterministes complets (DFA) sont **équivalents** s'ils reconnaissent le même langage.

Exemple de langage reconnu

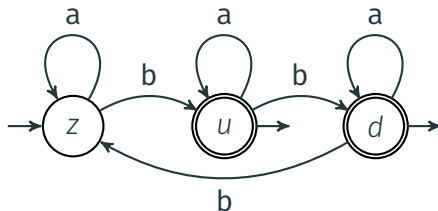
Le langage accepté ou reconnu par l'automate



est l'ensemble des mots ???.

Exemple de langage reconnu

Le langage accepté ou reconnu par l'automate

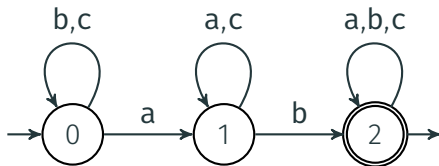


est l'ensemble des mots contenant un nombre d'occurrence du symbole **b** non divisible par 3.

$$\mathcal{L} = \{w \in \Sigma^*; |w|_b \not\equiv 0 \pmod{3}\}.$$

Exemple 2

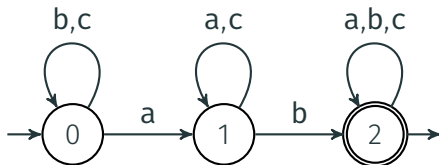
Soit l'alphabet $\Sigma = \{a, b, c\}$ et $\mathcal{A}$ l'automate



Alors $\mathcal{L}(\mathcal{A})$ est

Exemple 2

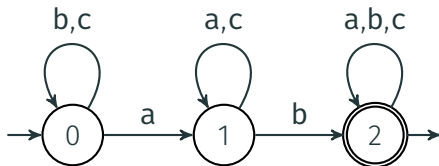
Soit l'alphabet $\Sigma = \{a, b, c\}$ et $\mathcal{A}$ l'automate



Alors $\mathcal{L}(\mathcal{A})$ est l'ensemble des mots dont **ab** est un sous-mot.

Exemple 2

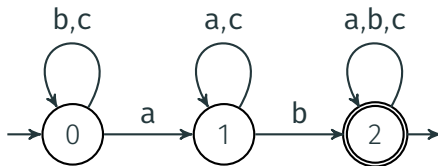
Soit l'alphabet $\Sigma = \{a, b, c\}$ et $\mathcal{A}$ l'automate



Alors $\mathcal{L}(\mathcal{A})$ est l'ensemble des mots contenant un **a** suivi, pas forcément immédiatement, d'un **b**.

Exemple 2

Soit l'alphabet $\Sigma = \{a, b, c\}$ et $\mathcal{A}$ l'automate



Alors $\mathcal{L}(\mathcal{A})$ est le langage dénoté par l'expression régulière

$$(b|c) * a(a|c) * b(a|b|c) * .$$

États accessibles

Définition

Un état q est dit **accessible** s'il existe un mot $w \in \Sigma^*$ tel que $q = \delta^*(q_0, w)$.

États accessibles

Définition

Un état q est dit **accessible** s'il existe un mot $w \in \Sigma^*$ tel que $q = \delta^*(q_0, w)$.

En supprimant les états inaccessibles, on conserve un automate équivalent.

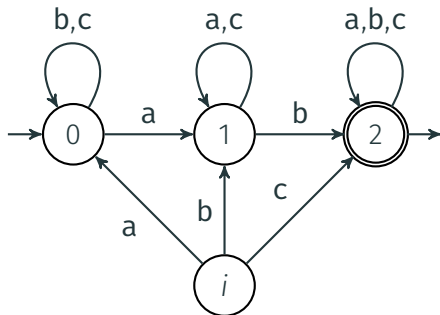
États accessibles

Définition

Un état q est dit **accessible** s'il existe un mot $w \in \Sigma^*$ tel que $q = \delta^*(q_0, w)$.

En supprimant les états inaccessibles, on conserve un automate équivalent.

Exemple



L'état i est inaccessible.

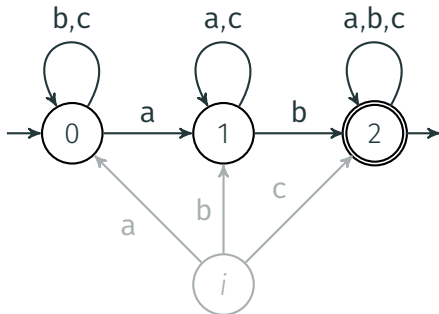
États accessibles

Définition

Un état q est dit **accessible** s'il existe un mot $w \in \Sigma^*$ tel que $q = \delta^*(q_0, w)$.

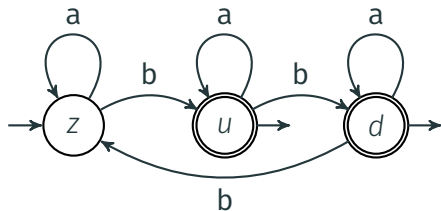
En supprimant les états inaccessibles, on conserve un automate équivalent.

Exemple



L'état i est inaccessible.

Est-ce compliqué à programmer en Python ?



```
initial = 'z'  
finals = {'u', 'd'}  
trans = { 'z':{'a':'z' , 'b':'u'},  
          'u':{'a':'u' , 'b':'d'},  
          'd':{'a':'d' , 'b':'z'} }
```

```
def accepte(q0,w):  
    if w=="":  
        return (q0 in finals)  
    else:  
        return accepte(trans[q0][w[0]], w[1:])
```

```
accepte(initial,"abbab")
```

Automates finis déterministes incomplets DFAi

Définition

Un automate fini déterministe à spécification incomplète est un quintuplet $(\Sigma, Q, q_0, \delta, F)$ où

Définition

Un automate fini déterministe à spécification incomplète est un quintuplet $(\Sigma, Q, q_0, \delta, F)$ où

- Σ est un alphabet

Définition

Un automate fini déterministe à spécification incomplète est un quintuplet $(\Sigma, Q, q_0, \delta, F)$ où

- Σ est un alphabet
- Q est un ensemble fini, appelé ensemble d'états

Définition

Un **automate fini déterministe à spécification incomplète** est un quintuplet $(\Sigma, Q, q_0, \delta, F)$ où

- Σ est un **alphabet**
- Q est un ensemble fini, appelé ensemble d'**états**
- $q_0 \in Q$ est un état particulier, appelé **état initial**

Définition

Un automate fini déterministe à spécification incomplète est un quintuplet $(\Sigma, Q, q_0, \delta, F)$ où

- Σ est un alphabet
- Q est un ensemble fini, appelé ensemble d'états
- $q_0 \in Q$ est un état particulier, appelé état initial
- $\delta : Q \times \Sigma \rightarrow Q$ est une fonction partielle, dite fonction de transition.

Définition

Un **automate fini déterministe à spécification incomplète** est un quintuplet $(\Sigma, Q, q_0, \delta, F)$ où

- Σ est un **alphabet**
- Q est un ensemble fini, appelé ensemble d'**états**
- $q_0 \in Q$ est un état particulier, appelé **état initial**
- $\delta : Q \times \Sigma \rightarrow Q$ est une fonction **partielle**, dite **fonction de transition**.
- $F \subseteq Q$ est un ensemble d'**états** dits **finaux** ou acceptants

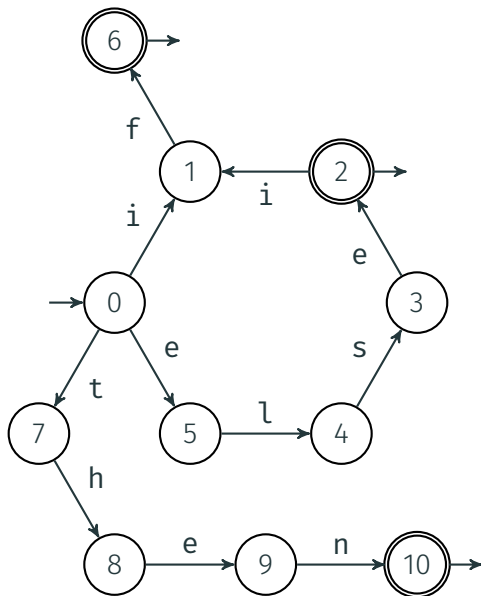
Définition

Un **automate fini déterministe à spécification incomplète** est un quintuplet $(\Sigma, Q, q_0, \delta, F)$ où

- Σ est un **alphabet**
- Q est un ensemble fini, appelé ensemble d'**états**
- $q_0 \in Q$ est un état particulier, appelé **état initial**
- $\delta : Q \times \Sigma \rightarrow Q$ est une fonction **partielle**, dite **fonction de transition**.
- $F \subseteq Q$ est un ensemble d'**états** dits **finaux** ou acceptants

On évite de stocker des transitions inutiles.

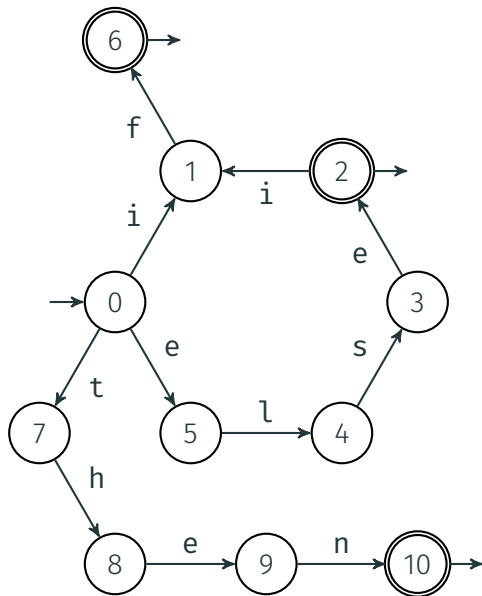
Exemple (analyse lexicale ou tokenisation)



On étend δ en une fonction partielle $\delta^* : Q \times \Sigma^* \rightarrow Q$ par

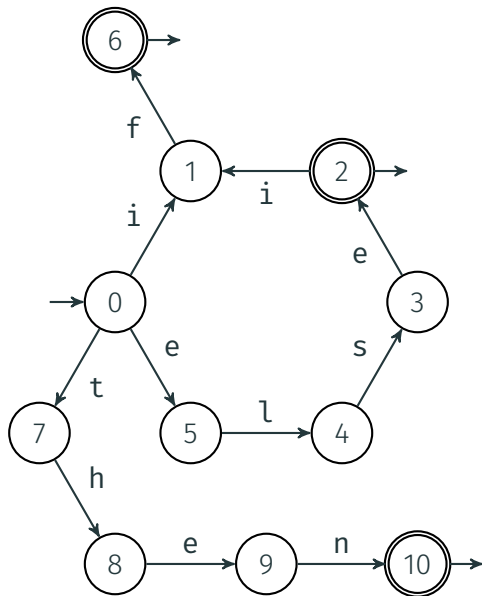
$$\delta^*(q, x_1 x_2 \cdots x_n) = \begin{cases} q & \text{si } n = 0 \\ \text{non défini} & \text{si } \delta(q, x_1) \text{ est non défini} \\ \delta^*(\delta(q, x_1), x_2 \cdots x_n) & \text{sinon} \end{cases}$$

Exemple (analyse lexicale ou tokenisation)



Le langage reconnu par cet automate est

Exemple (analyse lexicale ou tokenisation)



Le langage reconnu par cet automate est formé des 4 mots
{if, else, elseif, then}.

Définition

On dit qu'un mot $w \in \Sigma^*$ est **accepté** ou **reconnu** par l'automate si

$$\delta^*(q_0, w) \in F.$$

Définition

On dit qu'un mot $w \in \Sigma^*$ est **accepté** ou **reconnu** par l'automate si

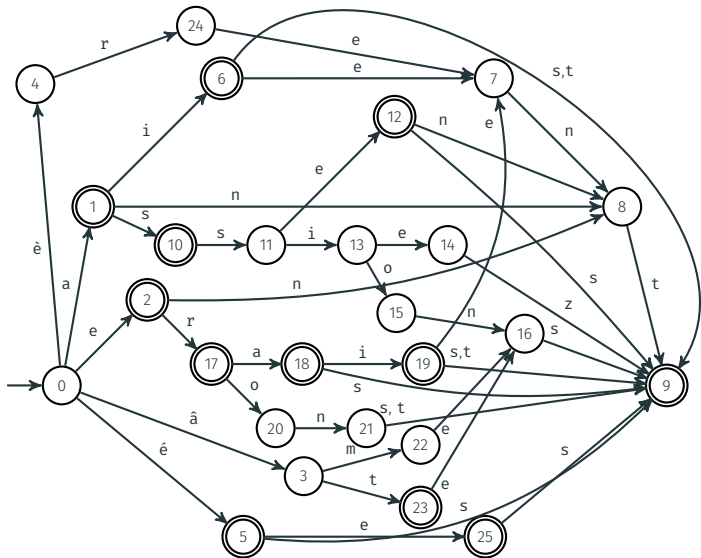
$$\delta^*(q_0, w) \in F.$$

Définition

L'ensemble $\mathcal{L}(\mathcal{A})$ des mots reconnus par un automate $\mathcal{A}$ s'appelle le **langage accepté**, ou **reconnu**, par l'automate $\mathcal{A}$.

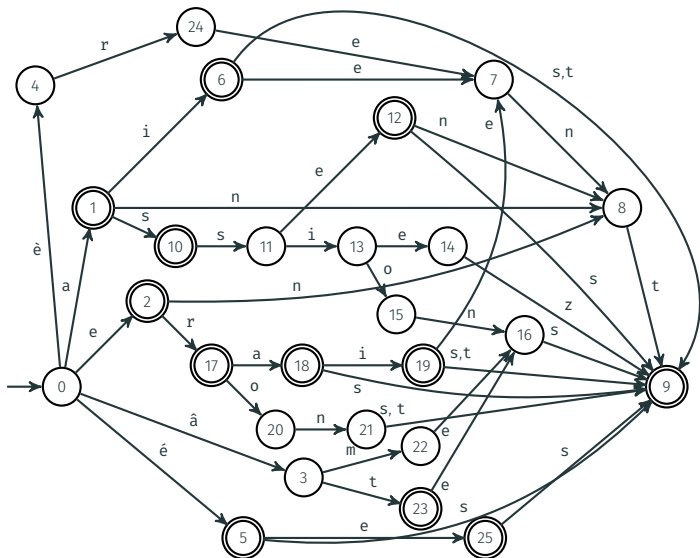
Exemple

Que reconnait l'automate suivant ?



Exemple

Automate des terminaisons des verbes du premier groupe.



Définition

On dit qu'un état $q_{\perp}$ est un **puits** si, pour tout symbole x ,

$$\delta^*(q_{\perp}, x) = q_{\perp}.$$

Définition

On dit qu'un état $q_{\perp}$ est un **puits** si, pour tout symbole x ,

$$\delta^*(q_{\perp}, x) = q_{\perp}.$$

Proposition

Pour tout DFAi à q état, il existe un DFA équivalent à $q + 1$ états.

Définition

On dit qu'un état $q_{\perp}$ est un **puits** si, pour tout symbole x ,
 $\delta^*(q_{\perp}, x) = q_{\perp}$.

Proposition

Pour tout DFAi à q état, il existe un DFA équivalent à $q + 1$ états.

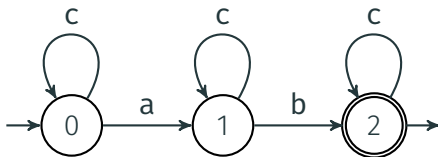
Démonstration.

Rajouter un état puits non-final. Compléter l'automate avec des transitions vers le puits. □

Preuve en image

Exemple

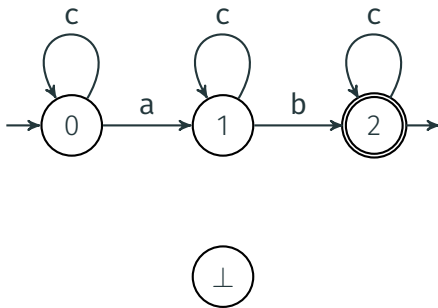
On considère, sur l'alphabet $\Sigma = \{a, b, c\}$, l'automate qui reconnaît le langage dénoté par l'expression régulière $c^*ac^*bc^*$



Preuve en image

Exemple

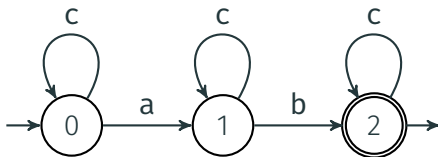
On considère, sur l'alphabet $\Sigma = \{a, b, c\}$, l'automate qui reconnaît le langage dénoté par l'expression régulière $c^*ac^*bc^*$



Preuve en image

Exemple

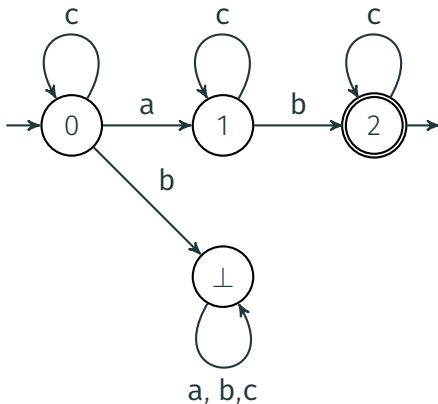
On considère, sur l'alphabet $\Sigma = \{a, b, c\}$, l'automate qui reconnaît le langage dénoté par l'expression régulière $c^*ac^*bc^*$



Preuve en image

Exemple

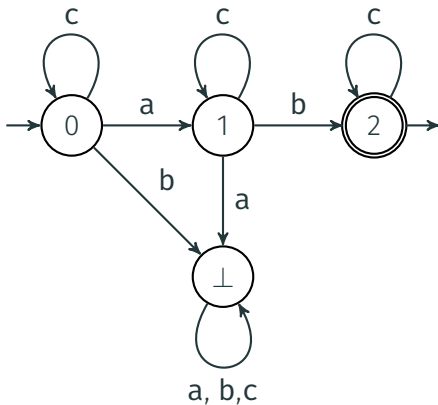
On considère, sur l'alphabet $\Sigma = \{a, b, c\}$, l'automate qui reconnaît le langage dénoté par l'expression régulière $c^*ac^*bc^*$



Preuve en image

Exemple

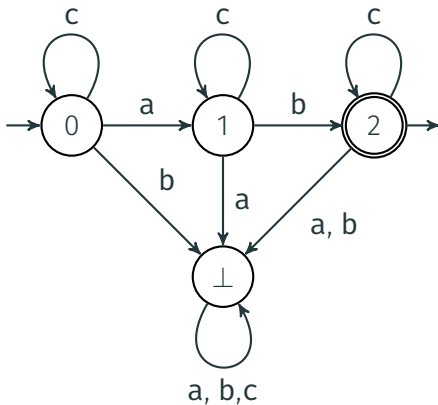
On considère, sur l'alphabet $\Sigma = \{a, b, c\}$, l'automate qui reconnaît le langage dénoté par l'expression régulière $c^*ac^*bc^*$



Preuve en image

Exemple

On considère, sur l'alphabet $\Sigma = \{a, b, c\}$, l'automate qui reconnaît le langage dénoté par l'expression régulière $c^*ac^*bc^*$



Définition

Un état $q \in Q$ est dit **co-accessible** s'il existe un mot $w \in \Sigma^*$ tel que $\delta^*(q, w) \in F$.

Définition

Un état $q \in Q$ est dit **co-accessible** s'il existe un mot $w \in \Sigma^*$ tel que $\delta^*(q, w) \in F$.

En supprimant les états non co-accessibles, on conserve un automate équivalent.

Définition

Un état $q \in Q$ est dit **co-accessible** s'il existe un mot $w \in \Sigma^*$ tel que $\delta^*(q, w) \in F$.

En supprimant les états non co-accessibles, on conserve un automate équivalent.

Définition

En supprimant les états non accessibles ou non co-accessibles d'un automate, on obtient l'automate **émondé**.

Automates finis non-déterministes (NFA)

Définition

Un **automate fini non-déterministe** est un quintuplet $(\Sigma, Q, l, \delta, F)$ où

Définition

Un **automate fini non-déterministe** est un quintuplet $(\Sigma, Q, l, \delta, F)$ où

- Σ est un **alphabet**

Définition

Un **automate fini non-déterministe** est un quintuplet $(\Sigma, Q, l, \delta, F)$ où

- Σ est un **alphabet**
- Q est un ensemble fini, appelé ensemble d'**états**

Définition

Un **automate fini non-déterministe** est un quintuplet $(\Sigma, Q, I, \delta, F)$ où

- Σ est un **alphabet**
- Q est un ensemble fini, appelé ensemble d'**états**
- $I \subseteq Q$ est un ensemble d'états, appelés **états initiaux**

Définition

Un **automate fini non-déterministe** est un quintuplet $(\Sigma, Q, I, \delta, F)$ où

- Σ est un **alphabet**
- Q est un ensemble fini, appelé ensemble d'**états**
- $I \subseteq Q$ est un ensemble d'états, appelés **états initiaux**
- $\delta \subseteq Q \times \Sigma \times Q$ est une **relation**, dite **relation de transition**.

Définition

Un **automate fini non-déterministe** est un quintuplet $(\Sigma, Q, I, \delta, F)$ où

- Σ est un **alphabet**
- Q est un ensemble fini, appelé ensemble d'**états**
- $I \subseteq Q$ est un ensemble d'états, appelés **états initiaux**
- $\delta \subseteq Q \times \Sigma \times Q$ est une **relation**, dite **relation de transition**.
- $F \subseteq Q$ est un ensemble d'**états** dits **finaux** ou acceptants

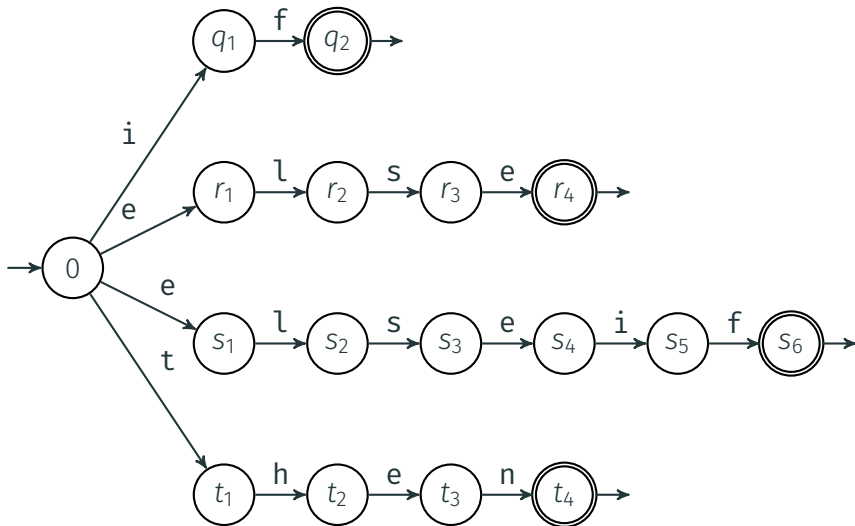
Définition

Un **automate fini non-déterministe** est un quintuplet $(\Sigma, Q, I, \delta, F)$ où

- Σ est un **alphabet**
- Q est un ensemble fini, appelé ensemble d'**états**
- $I \subseteq Q$ est un ensemble d'états, appelés **états initiaux**
- $\delta \subseteq Q \times \Sigma \times Q$ est une **relation**, dite **relation de transition**.
- $F \subseteq Q$ est un ensemble d'**états** dits **finaux** ou acceptants

On autorise des transitions multiples.

Exemple de NFA



Définition

Un mot $w = x_1x_2 \cdots x_n \in \Sigma^*$ est **accepté** par l'automate s'il existe une suite d'états $(q_0, q_1, \dots, q_n) \subseteq Q$ telle que

Définition

Un mot $w = x_1x_2 \cdots x_n \in \Sigma^*$ est **accepté** par l'automate s'il existe une suite d'états $(q_0, q_1, \dots, q_n) \subseteq Q$ telle que

1. l'état q_0 est initial,

Définition

Un mot $w = x_1x_2 \cdots x_n \in \Sigma^*$ est **accepté** par l'automate s'il existe une suite d'états $(q_0, q_1, \dots, q_n) \subseteq Q$ telle que

1. l'état q_0 est initial,
2. pour tout indice $1 \leq i \leq n$, le triplet $(q_{i-1}, x_i, q_i) \in Q \times \Sigma \times Q$ appartient à la relation de transition δ .

Définition

Un mot $w = x_1x_2 \cdots x_n \in \Sigma^*$ est **accepté** par l'automate s'il existe une suite d'états $(q_0, q_1, \dots, q_n) \subseteq Q$ telle que

1. l'état q_0 est initial,
2. pour tout indice $1 \leq i \leq n$, le triplet $(q_{i-1}, x_i, q_i) \in Q \times \Sigma \times Q$ appartient à la relation de transition δ .
3. l'état q_n est final.

Définition

Un mot $w = x_1x_2 \cdots x_n \in \Sigma^*$ est **accepté** par l'automate s'il existe une suite d'états $(q_0, q_1, \dots, q_n) \subseteq Q$ telle que

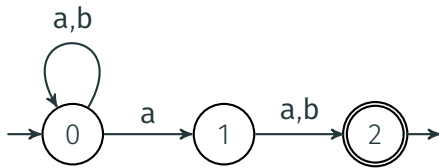
1. l'état q_0 est initial,
2. pour tout indice $1 \leq i \leq n$, le triplet $(q_{i-1}, x_i, q_i) \in Q \times \Sigma \times Q$ appartient à la relation de transition δ .
3. l'état q_n est final.

Définition

L'ensemble $\mathcal{L}(\mathcal{A})$ des mots reconnus par un automate NFA s'appelle le **langage accepté**, ou **reconnu**, par l'automate.

Exemple

Sur l'alphabet $\Sigma = \{a, b, c\}$, l'automate



reconnaît le langage dénoté par l'expression rationnelle $(a|b)^*a(a|b)$.

Proposition

Pour tout NFA à q état, il existe un DFA équivalent à moins de 2^q états.

Déterminisation

Proposition

Pour tout NFA à q état, il existe un DFA équivalent à moins de 2^q états.

Démonstration.

Soit $\mathcal{A} = (\Sigma, Q, I, \Delta, F)$ un NFA. On construit un DFA $(\Sigma, 2^Q, I, \delta, F')$ (où 2^Q est l'ensemble des parties de Q).

Déterminisation

Proposition

Pour tout NFA à q état, il existe un DFA équivalent à moins de 2^q états.

Démonstration.

Soit $\mathcal{A} = (\Sigma, Q, l, \Delta, F)$ un NFA. On construit un DFA $(\Sigma, 2^Q, l, \delta, F')$ (où 2^Q est l'ensemble des parties de Q).

On pose pour tout ensemble d'états $\mathbf{q} \subseteq Q$ et tout symbole $x \in \Sigma$,

$$\delta(\mathbf{q}, x) = \{q_1 \in Q; \exists q_0 \in \mathbf{q} \text{ t.q. } (q_0, x, q_1) \in \Delta\}.$$

Déterminisation

Proposition

Pour tout NFA à q état, il existe un DFA équivalent à moins de 2^q états.

Démonstration.

Soit $\mathcal{A} = (\Sigma, Q, I, \Delta, F)$ un NFA. On construit un DFA $(\Sigma, 2^Q, I, \delta, F')$ (où 2^Q est l'ensemble des parties de Q).

On pose pour tout ensemble d'états $\mathbf{q} \subseteq Q$ et tout symbole $x \in \Sigma$,

$$\delta(\mathbf{q}, x) = \{q_1 \in Q; \exists q_0 \in \mathbf{q} \text{ t.q. } (q_0, x, q_1) \in \Delta\}.$$

On note $F' = \{\mathbf{q} \subseteq Q; \mathbf{q} \cap F \neq \emptyset\}$.

Détermination

Proposition

Pour tout NFA à q état, il existe un DFA équivalent à moins de 2^q états.

Démonstration.

Soit $\mathcal{A} = (\Sigma, Q, I, \Delta, F)$ un NFA. On construit un DFA $(\Sigma, 2^Q, I, \delta, F')$ (où 2^Q est l'ensemble des parties de Q).

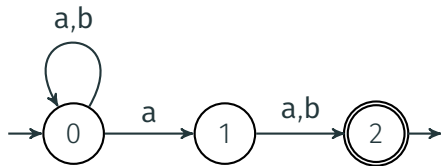
On pose pour tout ensemble d'états $\mathbf{q} \subseteq Q$ et tout symbole $x \in \Sigma$,

$$\delta(\mathbf{q}, x) = \{q_1 \in Q; \exists q_0 \in \mathbf{q} \text{ t.q. } (q_0, x, q_1) \in \Delta\}.$$

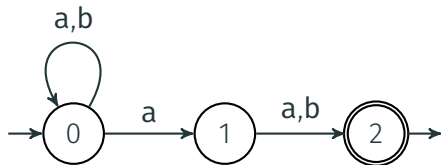
On note $F' = \{\mathbf{q} \subseteq Q; \mathbf{q} \cap F \neq \emptyset\}$.

On prouve par récurrence sur $|w|$ que $\delta^*(I, w)$ est l'ensemble des états auxquels on peut accéder par un chemin dans Δ démarré dans I et consommant les lettres de w . □

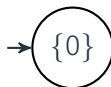
Exemple de détermination



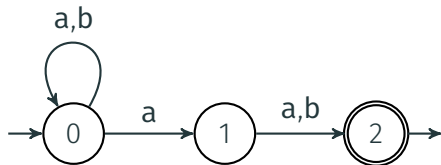
Exemple de détermination



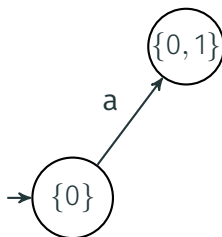
	a	b
$\{0\}$	$?$	$?$



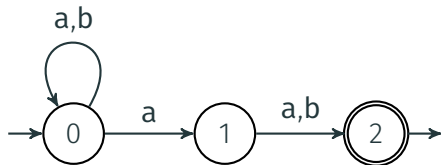
Exemple de détermination



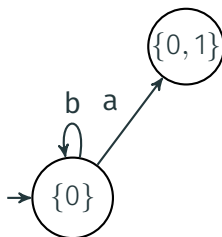
	a	b
$\{0\}$	$\{0, 1\}$	$?$
$\{0, 1\}$	$?$	$?$



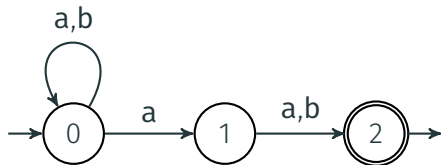
Exemple de détermination



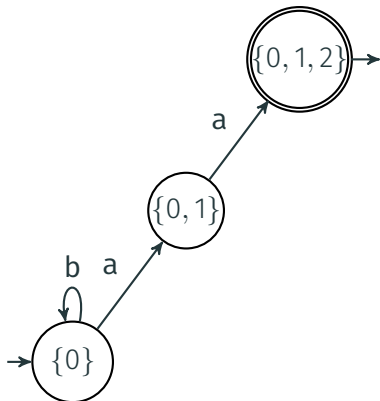
	a	b
$\{0\}$	$\{0, 1\}$	$\{0\}$
$\{0, 1\}$	$?$	$?$



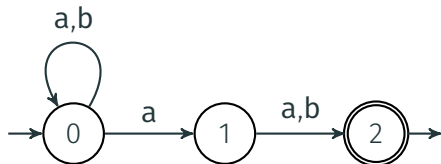
Exemple de détermination



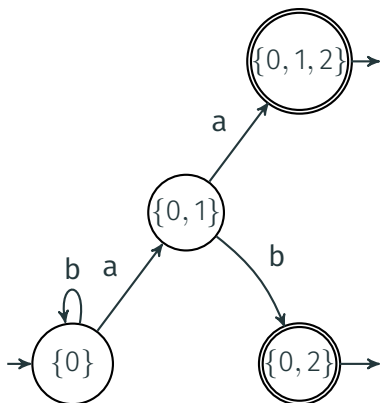
	a	b
$\{0\}$	$\{0, 1\}$	$\{0\}$
$\{0, 1\}$	$\{0, 1, 2\}$	$?$
$\{0, 1, 2\}$	$?$	$?$



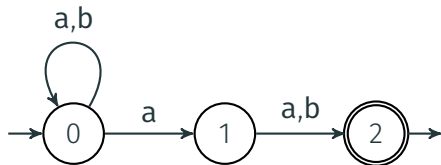
Exemple de détermination



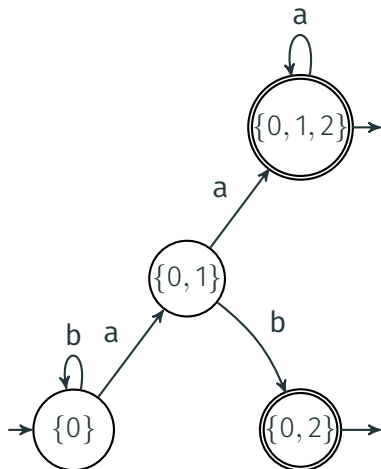
	a	b
$\{0\}$	$\{0, 1\}$	$\{0\}$
$\{0, 1\}$	$\{0, 1, 2\}$	$\{0, 2\}$
$\{0, 1, 2\}$	?	?
$\{0, 2\}$	?	?



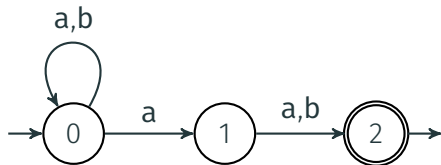
Exemple de détermination



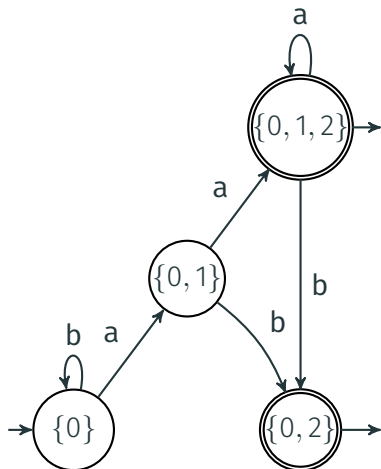
	a	b
$\{0\}$	$\{0, 1\}$	$\{0\}$
$\{0, 1\}$	$\{0, 1, 2\}$	$\{0, 2\}$
$\{0, 1, 2\}$	$\{0, 1, 2\}$	$?$
$\{0, 2\}$	$?$	$?$



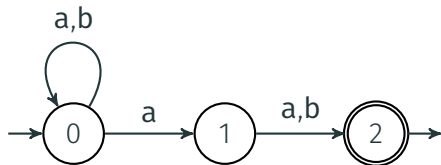
Exemple de détermination



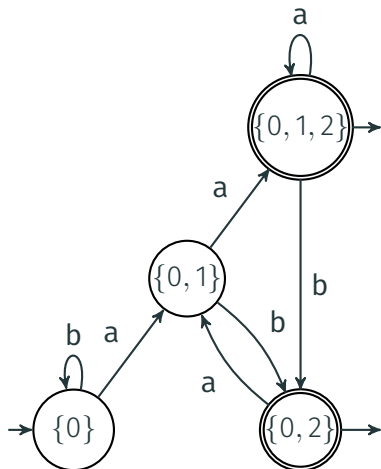
	a	b
$\{0\}$	$\{0, 1\}$	$\{0\}$
$\{0, 1\}$	$\{0, 1, 2\}$	$\{0, 2\}$
$\{0, 1, 2\}$	$\{0, 1, 2\}$	$\{0, 2\}$
$\{0, 2\}$	$?$	$?$



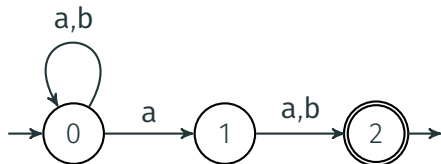
Exemple de détermination



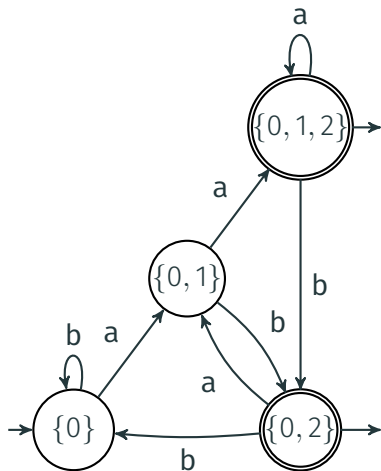
	a	b
$\{0\}$	$\{0, 1\}$	$\{0\}$
$\{0, 1\}$	$\{0, 1, 2\}$	$\{0, 2\}$
$\{0, 1, 2\}$	$\{0, 1, 2\}$	$\{0, 2\}$
$\{0, 2\}$	$\{0, 1\}$	$?$



Exemple de détermination



	<i>a</i>	<i>b</i>
$\{0\}$	$\{0, 1\}$	$\{0\}$
$\{0, 1\}$	$\{0, 1, 2\}$	$\{0, 2\}$
$\{0, 1, 2\}$	$\{0, 1, 2\}$	$\{0, 2\}$
$\{0, 2\}$	$\{0, 1\}$	$\{0\}$



Automates finis non-déterministes à transitions spontanées (ϵ NFA)

Définition

Un automate fini non-déterministe à transition spontanées est un quintuplet $(\Sigma, Q, I, \delta, F)$ où

Définition

Un automate fini non-déterministe à transition spontanées est un quintuplet $(\Sigma, Q, I, \delta, F)$ où

- Σ est un alphabet

Définition

Un automate fini non-déterministe à transition spontanées est un quintuplet $(\Sigma, Q, I, \delta, F)$ où

- Σ est un alphabet
- Q est un ensemble fini, appelé ensemble d'états

Définition

Un automate fini non-déterministe à transition spontanées est un quintuplet $(\Sigma, Q, I, \delta, F)$ où

- Σ est un alphabet
- Q est un ensemble fini, appelé ensemble d'états
- $I \subseteq Q$ est un ensemble d'états, appelés états initiaux

Définition

Un automate fini non-déterministe à transition spontanées est un quintuplet $(\Sigma, Q, I, \delta, F)$ où

- Σ est un alphabet
- Q est un ensemble fini, appelé ensemble d'états
- $I \subseteq Q$ est un ensemble d'états, appelés états initiaux
- $\delta \subseteq Q \times (\Sigma \cup \{\epsilon\}) \times Q$ est une relation, dite relation de transition.

Définition

Un automate fini non-déterministe à transition spontanées est un quintuplet $(\Sigma, Q, I, \delta, F)$ où

- Σ est un alphabet
- Q est un ensemble fini, appelé ensemble d'états
- $I \subseteq Q$ est un ensemble d'états, appelés états initiaux
- $\delta \subseteq Q \times (\Sigma \cup \{\epsilon\}) \times Q$ est une relation, dite relation de transition.
- $F \subseteq Q$ est un ensemble d'états dits finaux ou acceptants

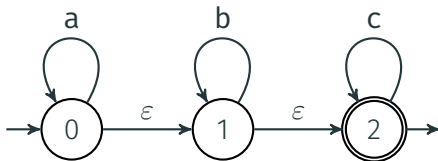
Définition

Un automate fini non-déterministe à transition spontanées est un quintuplet $(\Sigma, Q, I, \delta, F)$ où

- Σ est un alphabet
- Q est un ensemble fini, appelé ensemble d'états
- $I \subseteq Q$ est un ensemble d'états, appelés états initiaux
- $\delta \subseteq Q \times (\Sigma \cup \{\epsilon\}) \times Q$ est une relation, dite relation de transition.
- $F \subseteq Q$ est un ensemble d'états dits finaux ou acceptants

On autorise des transitions dites spontanées.

Exemple avec des transitions spontanées



Définition

Un mot $w \in \Sigma^*$ est **accepté** par l'automate s'il existe une suite d'états $(q_0, q_1, \dots, q_n) \in Q^{n+1}$ et une suite $(t_1, \dots, t_n) \in (\Sigma \cup \{\epsilon\})^n$ telle que

Définition

Un mot $w \in \Sigma^*$ est **accepté** par l'automate s'il existe une suite d'états $(q_0, q_1, \dots, q_n) \in Q^{n+1}$ et une suite $(t_1, \dots, t_n) \in (\Sigma \cup \{\epsilon\})^n$ telle que

1. l'état q_0 est initial,

Définition

Un mot $w \in \Sigma^*$ est **accepté** par l'automate s'il existe une suite d'états $(q_0, q_1, \dots, q_n) \in Q^{n+1}$ et une suite $(t_1, \dots, t_n) \in (\Sigma \cup \{\epsilon\})^n$ telle que

1. l'état q_0 est initial,
2. pour tout indice $1 \leq i \leq n$, le triplet $(q_{i-1}, t_i, q_i) \in Q \times \Sigma \times Q$ appartient à la relation de transition δ ,

Définition

Un mot $w \in \Sigma^*$ est **accepté** par l'automate s'il existe une suite d'états $(q_0, q_1, \dots, q_n) \in Q^{n+1}$ et une suite $(t_1, \dots, t_n) \in (\Sigma \cup \{\epsilon\})^n$ telle que

1. l'état q_0 est initial,
2. pour tout indice $1 \leq i \leq n$, le triplet $(q_{i-1}, t_i, q_i) \in Q \times \Sigma \times Q$ appartient à la relation de transition δ ,
3. l'état q_n est final,

Définition

Un mot $w \in \Sigma^*$ est **accepté** par l'automate s'il existe une suite d'états $(q_0, q_1, \dots, q_n) \in Q^{n+1}$ et une suite $(t_1, \dots, t_n) \in (\Sigma \cup \{\epsilon\})^n$ telle que

1. l'état q_0 est initial,
2. pour tout indice $1 \leq i \leq n$, le triplet $(q_{i-1}, t_i, q_i) \in Q \times \Sigma \times Q$ appartient à la relation de transition δ ,
3. l'état q_n est final,
4. la concatenation $t_1 t_2 \cdots t_n$ égale le mot w .

Définition

Un mot $w \in \Sigma^*$ est **accepté** par l'automate s'il existe une suite d'états $(q_0, q_1, \dots, q_n) \in Q^{n+1}$ et une suite $(t_1, \dots, t_n) \in (\Sigma \cup \{\epsilon\})^n$ telle que

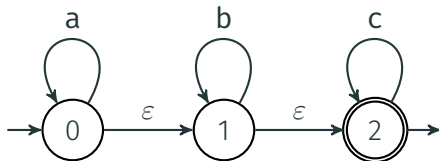
1. l'état q_0 est initial,
2. pour tout indice $1 \leq i \leq n$, le triplet $(q_{i-1}, t_i, q_i) \in Q \times \Sigma \times Q$ appartient à la relation de transition δ ,
3. l'état q_n est final,
4. la concatenation $t_1 t_2 \dots t_n$ égale le mot w .

Définition

L'ensemble $\mathcal{L}(\mathcal{A})$ des mots reconnus par un automate NFA s'appelle le **langage accepté**, ou **reconnu**, par l'automate.

Exemple

Sur l'alphabet $\Sigma = \{a, b, c\}$, l'automate



reconnaît le langage dénoté par l'expression rationnelle $a^*b^*c^*$.

Vers l'élimination des ε -transitions

Définition

On appelle ε -fermeture d'un état $q \in Q$ l'ensemble $C(q) \subseteq Q$ des états accessibles depuis q par ε -transition.

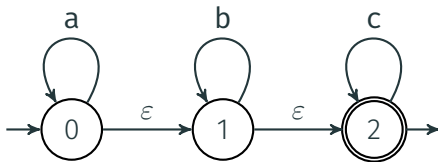
Vers l'élimination des ε -transitions

Définition

On appelle ε -fermeture d'un état $q \in Q$ l'ensemble $C(q) \subseteq Q$ des états accessibles depuis q par ε -transition.

Exemple

Dans l'automate



on a $C(0) =$

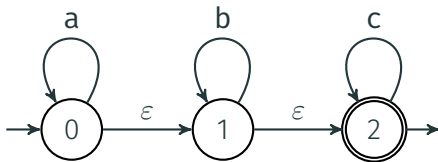
Vers l'élimination des ε -transitions

Définition

On appelle ε -fermeture d'un état $q \in Q$ l'ensemble $C(q) \subseteq Q$ des états accessibles depuis q par ε -transition.

Exemple

Dans l'automate



on a $C(0) = \{0, 1, 2\}$, $C(1) =$

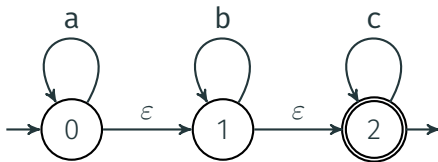
Vers l'élimination des ε -transitions

Définition

On appelle ε -fermeture d'un état $q \in Q$ l'ensemble $C(q) \subseteq Q$ des états accessibles depuis q par ε -transition.

Exemple

Dans l'automate



on a $C(0) = \{0, 1, 2\}$, $C(1) = \{1, 2\}$ et $C(2) =$

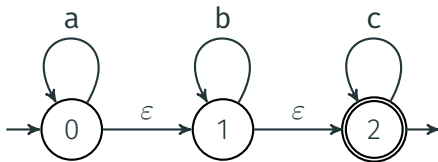
Vers l'élimination des ε -transitions

Définition

On appelle ε -fermeture d'un état $q \in Q$ l'ensemble $C(q) \subseteq Q$ des états accessibles depuis q par ε -transition.

Exemple

Dans l'automate



on a $C(0) = \{0, 1, 2\}$, $C(1) = \{1, 2\}$ et $C(2) = \{2\}$.

Élimination des ε -transitions

Proposition

Pour tout ε NFA, il existe un NFA équivalent avec le même ensemble d'états.

Élimination des ε -transitions

Proposition

Pour tout ε NFA, il existe un NFA équivalent avec le même ensemble d'états.

Démonstration.

Soit (Q, I, Δ, F) un ε NFA.

Élimination des ε -transitions

Proposition

Pour tout ε NFA, il existe un NFA équivalent avec le même ensemble d'états.

Démonstration.

Soit (Q, I, Δ, F) un ε NFA. On remplace Δ par

$$\Delta^{\S} = \{(q, x, q') ; \exists q'' \in C(q) \text{ et } (q'', x, q') \in \Delta\}.$$

Élimination des ε -transitions

Proposition

Pour tout ε NFA, il existe un NFA équivalent avec le même ensemble d'états.

Démonstration.

Soit (Q, I, Δ, F) un ε NFA. On remplace Δ par

$$\Delta^{\S} = \{(q, x, q') ; \exists q'' \in C(q) \text{ et } (q'', x, q') \in \Delta\}.$$

On remplace F par

$$F^{\S} = \{q \in Q ; C(q) \cap F \neq \emptyset\}.$$

Élimination des ε -transitions

Proposition

Pour tout ε NFA, il existe un NFA équivalent avec le même ensemble d'états.

Démonstration.

Soit (Q, I, Δ, F) un ε NFA. On remplace Δ par

$$\Delta^{\S} = \{(q, x, q') ; \exists q'' \in C(q) \text{ et } (q'', x, q') \in \Delta\}.$$

On remplace F par

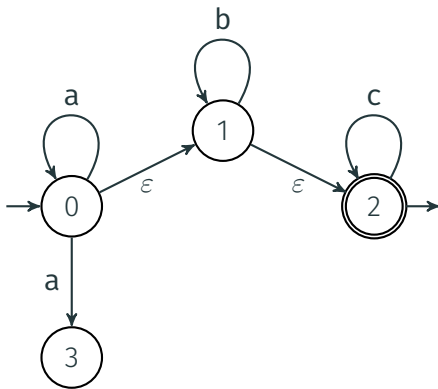
$$F^{\S} = \{q \in Q ; C(q) \cap F \neq \emptyset\}.$$

On prouve que $(Q, I, \Delta^{\S}, F^{\S})$ est équivalent.



Exemple d'élimination des transitions

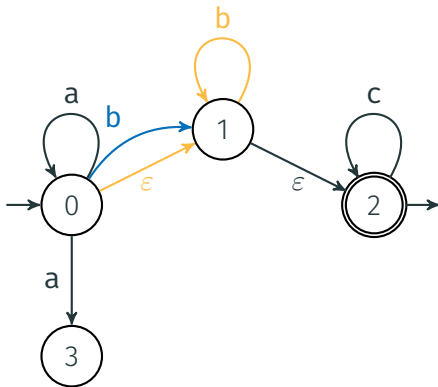
Considérons l'automate suivant qui reconnaît le langage dénoté par l'expression régulière $a^*b^*c^*$.



Exemple d'élimination des transitions

On complète les transition issues de $q = 0$.

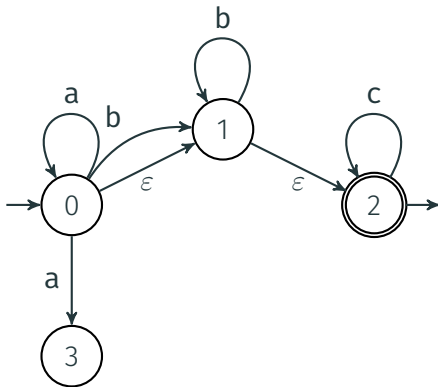
Notons que $C(0) = \{0, 1, 2\}$.



Exemple d'élimination des transitions

On complète les transition issues de $q = 0$.

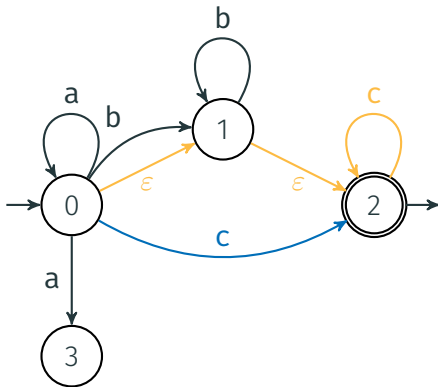
Notons que $C(0) = \{0, 1, 2\}$.



Exemple d'élimination des transitions

On complète les transition issues de $q = 0$.

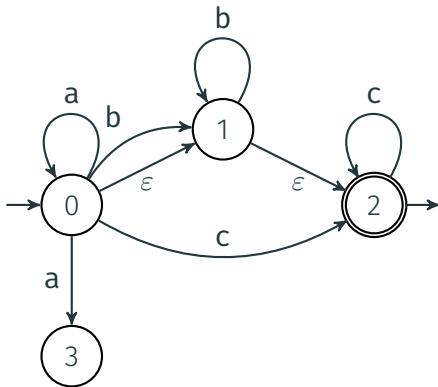
Notons que $C(0) = \{0, 1, 2\}$.



Exemple d'élimination des transitions

On complète les transition issues de $q = 0$.

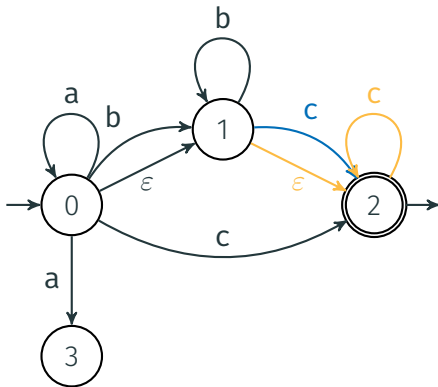
Notons que $C(0) = \{0, 1, 2\}$.



Exemple d'élimination des transitions

On complète les transition issues de $q = 1$.

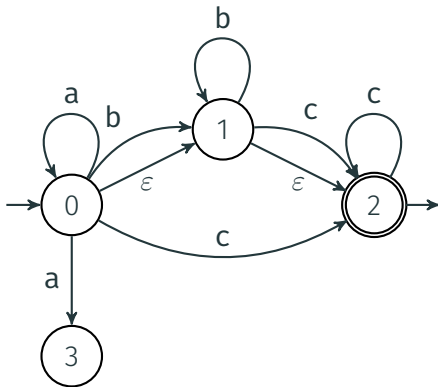
Notons que $C(1) = \{1, 2\}$.



Exemple d'élimination des transitions

On complète les transition issues de $q = 1$.

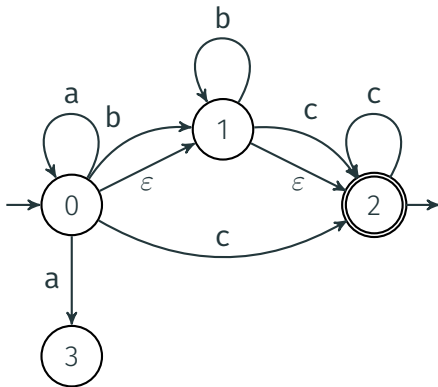
Notons que $C(1) = \{1, 2\}$.



Exemple d'élimination des transitions

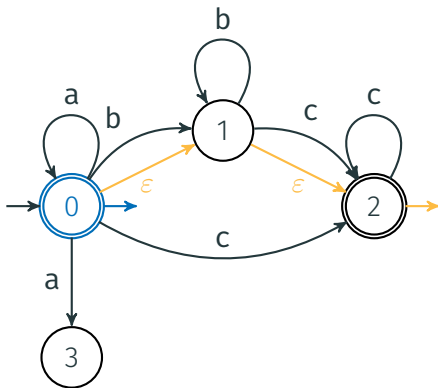
On complète les transition issues de $q = 2$.

Notons que $C(2) = \{2\}$: il n'y a pas d'ajouts à faire.



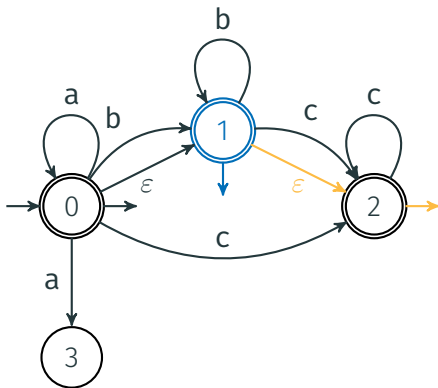
Exemple d'élimination des transitions

L'état $q = 0$ devient final car $C(0) = \{0, 1, 2\}$ contient un état final.

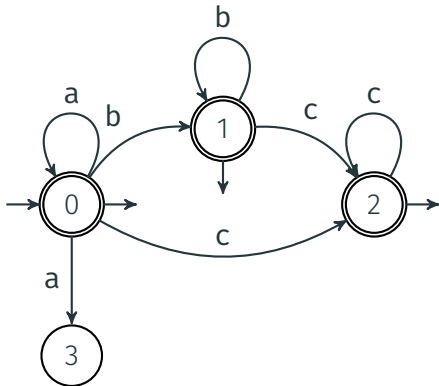


Exemple d'élimination des transitions

L'état $q = 1$ devient final car $C(1) = \{1, 2\}$ contient un état final.



Exemple d'élimination des transitions



Objectif

Rappel : les langages rationnels

Définition

Un langage $\mathcal{L}$ sur l'alphabet Σ est **rationnel** s'il s'agit de

Rappel : les langages rationnels

Définition

Un langage $\mathcal{L}$ sur l'alphabet Σ est **rationnel** s'il s'agit de

- $\mathcal{L} = \emptyset$ (le langage vide)

Rappel : les langages rationnels

Définition

Un langage $\mathcal{L}$ sur l'alphabet Σ est **rationnel** s'il s'agit de

- $\mathcal{L} = \emptyset$ (le langage vide)
- $\mathcal{L} = \{\epsilon\}$ (le langage contenant le mot vide)

Rappel : les langages rationnels

Définition

Un langage $\mathcal{L}$ sur l'alphabet Σ est **rationnel** s'il s'agit de

- $\mathcal{L} = \emptyset$ (le langage vide)
- $\mathcal{L} = \{\epsilon\}$ (le langage contenant le mot vide)
- $\mathcal{L} = \{x\}$, où $x \in \Sigma$ (le langage contenant le mot à une lettre)

Rappel : les langages rationnels

Définition

Un langage $\mathcal{L}$ sur l'alphabet Σ est **rationnel** s'il s'agit de

- $\mathcal{L} = \emptyset$ (le langage vide)
- $\mathcal{L} = \{\epsilon\}$ (le langage contenant le mot vide)
- $\mathcal{L} = \{x\}$, où $x \in \Sigma$ (le langage contenant le mot à une lettre)

ou si on peut le construire par

- union $\mathcal{L} = \mathcal{L}_1 \cup \mathcal{L}_2$

Rappel : les langages rationnels

Définition

Un langage $\mathcal{L}$ sur l'alphabet Σ est **rationnel** s'il s'agit de

- $\mathcal{L} = \emptyset$ (le langage vide)
- $\mathcal{L} = \{\epsilon\}$ (le langage contenant le mot vide)
- $\mathcal{L} = \{x\}$, où $x \in \Sigma$ (le langage contenant le mot à une lettre)

ou si on peut le construire par

- union $\mathcal{L} = \mathcal{L}_1 \cup \mathcal{L}_2$
- concatenation $\mathcal{L} = \mathcal{L}_1\mathcal{L}_2$

Rappel : les langages rationnels

Définition

Un langage $\mathcal{L}$ sur l'alphabet Σ est **rationnel** s'il s'agit de

- $\mathcal{L} = \emptyset$ (le langage vide)
- $\mathcal{L} = \{\epsilon\}$ (le langage contenant le mot vide)
- $\mathcal{L} = \{x\}$, où $x \in \Sigma$ (le langage contenant le mot à une lettre)

ou si on peut le construire par

- union $\mathcal{L} = \mathcal{L}_1 \cup \mathcal{L}_2$
- concatenation $\mathcal{L} = \mathcal{L}_1\mathcal{L}_2$
- ou étoile de Kleene $\mathcal{L} = \mathcal{L}_1^*$

à partir de langages rationnels $\mathcal{L}_1$ et $\mathcal{L}_2$.

Rappel : les langages reconnaissables

Définition

Un langage $\mathcal{L}$ sur l'alphabet Σ est **reconnaissable** s'il existe un automate fini tel que $\mathcal{L}$ est le langage reconnu par l'automate.

Rappel : les langages reconnaissables

Définition

Un langage $\mathcal{L}$ sur l'alphabet Σ est **reconnaissable** s'il existe un automate fini tel que $\mathcal{L}$ est le langage reconnu par l'automate.

Remarque

Le modèle d'automate (DFA, DFAi, NFA ou ε NFA) n'a pas d'importance.

1956



Stephen C. Kleene
(1909†1994)

Théorème (Kleene)

*L'ensemble des langages rationnels et celui des langages reconnaissables **coïncident**.*

D'une expression régulière à un
automate. Constructions de
Glouchkov

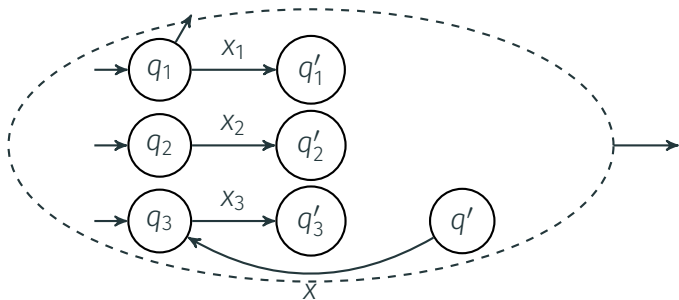
Lemme

Pour tout NFA $\mathcal{A}$ à q états, il existe un NFA équivalent $\mathcal{A}'$ à $q + 1$ états n'ayant qu'un **unique état initial** q_0 et tel qu'aucune transition n'aboutisse en q_0 .

Définition

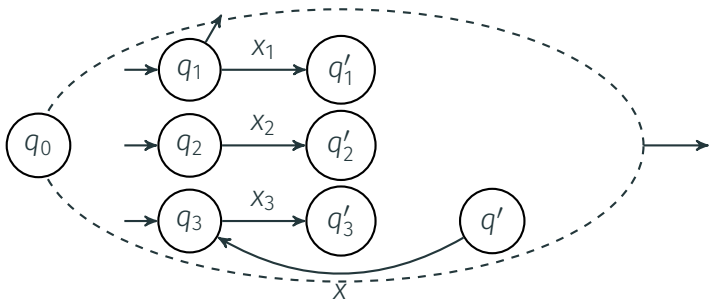
Un tel automate est dit **standard**.

Preuve : mise sous forme standard



Preuve : mise sous forme standard

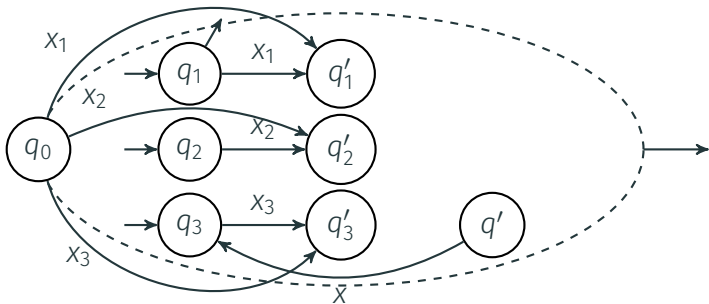
On ajoute un nouvel état q_0 .



Preuve : mise sous forme standard

On ajoute un nouvel état q_0 .

On ajoute des transitions (q_0, x, q') s'il existe $(q, x, q') \in \Delta$ tel que $q \in I$.

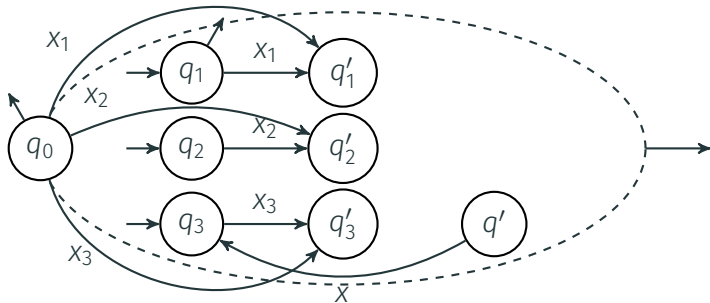


Preuve : mise sous forme standard

On ajoute un nouvel état q_0 .

On ajoute des transitions (q_0, x, q') s'il existe $(q, x, q') \in \Delta$ tel que $q \in I$.

On déclare q_0 final si $I \cap F \neq \emptyset$.



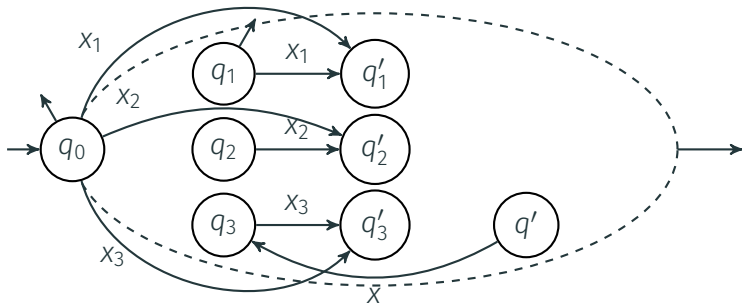
Preuve : mise sous forme standard

On ajoute un nouvel état q_0 .

On ajoute des transitions (q_0, x, q') s'il existe $(q, x, q') \in \Delta$ tel que $q \in I$.

On déclare q_0 final si $I \cap F \neq \emptyset$.

L'état q_0 est le seul initial.



Représentation sous forme standard

Nous numérotons les états de 0 à $n - 1$ et convenons que 0 est l'état initial.

En supposant que $\Sigma = \{a, b\}$, les transitions par **a** et par **b** sont représentées par deux matrices d'incidence T_a et T_b : s'il existe une transition de u vers v d'étiquette **a**, alors $T_a[u][v] = \text{Vrai}$.

L'ensemble des états finals est représenté par un vecteur.

Implémentation python d'automates standards

(Nous supposons que $\Sigma = \{a, b\}$).

```
class StandardAutomaton():
    def __init__(self, n):
        self.n = n
        self.transitionA = [[False for _ in range(n)] for _ in range(n)]
        self.transitionB = [[False for _ in range(n)] for _ in range(n)]
        self.finals = set()
    def add_final_state(self, s):
        self.finals.add(s)
    def add_transition(self, s, t, x):
        if x == "a":
            self.transitionA[s][t] = True
        if x == "b":
            self.transitionB[s][t] = True
    def copy_non_initial_transition_from(self, other, start):
        for u in range(1, other.n):
            for v in range(1, other.n):
                self.transitionA[u-1+start][v-1+start] = other.transitionA[u][v]
                self.transitionB[u-1+start][v-1+start] = other.transitionB[u][v]
```

Constructions de base : le langage vide

Le langage $\mathcal{L}(\perp) = \emptyset$ est reconnu par l'automate



Matrices d'incidences :



```
class Rat():
    pass

class Empty(Rat):
    def __init__(self):
        pass
    def to_automaton(self):
        return StandardAutomaton(1)
```

Constructions de base : le langage du mot vide

Le langage $\mathcal{L}(\underline{\varepsilon}) = \{\varepsilon\}$ est reconnu par l'automate



Matrices d'incidences :



```
class Letter(Rat):  
    def __init__(self,x):  
        self.op = x  
    def to_automaton(self):  
        A = StandardAutomaton(2)  
        A.add_final_state(1)  
        A.add_transition(0,1,self.op)  
        return A
```

Constructions de base : le langage du mot à une lettre

Le langage $\mathcal{L}(x) = \{x\}$ est reconnu par l'automate



Matrices d'incidences,
avec $x = a$:

Transitions a

F	V
F	F

Transitions b

F	F
F	F

Finals

F
V

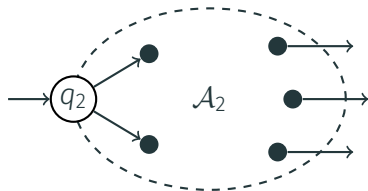
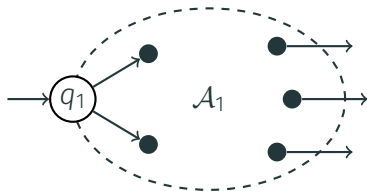
```
class Letter(Rat):  
    def __init__(self,x):  
        self.op = x  
    def to_automaton(self):  
        A = StandardAutomaton(2)  
        A.add_final_state(1)  
        A.add_transition(0,1,self.op)  
        return A
```

Proposition

Si $\mathcal{L}_1$ et $\mathcal{L}_2$ sont des langages reconnaissables, alors le langage $\mathcal{L}_1 \cup \mathcal{L}_2$ est reconnaissable.

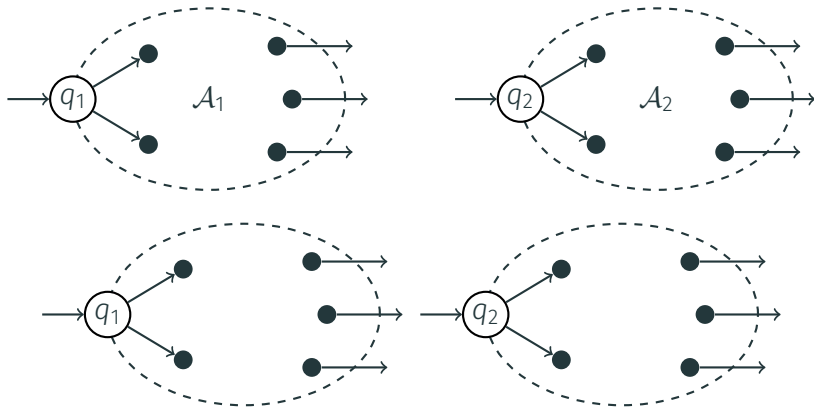
Preuve : union

Soient deux automates standards $\mathcal{A}_1$ et $\mathcal{A}_2$ qui reconnaissent $\mathcal{L}_1$ et $\mathcal{L}_2$.



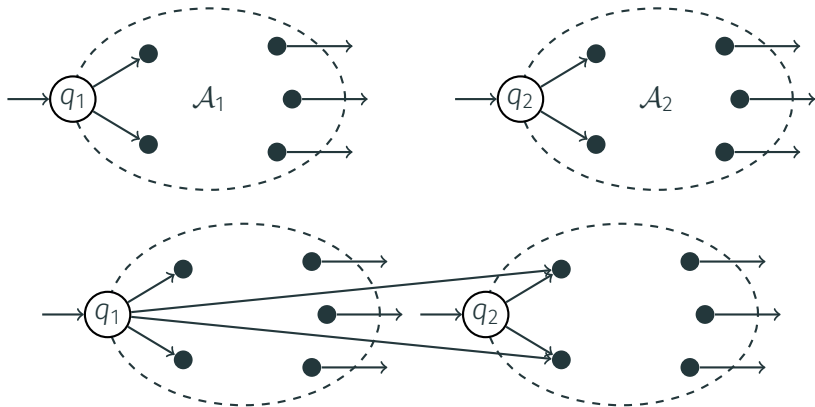
Preuve : union

Soient deux automates standards $\mathcal{A}_1$ et $\mathcal{A}_2$ qui reconnaissent $\mathcal{L}_1$ et $\mathcal{L}_2$.



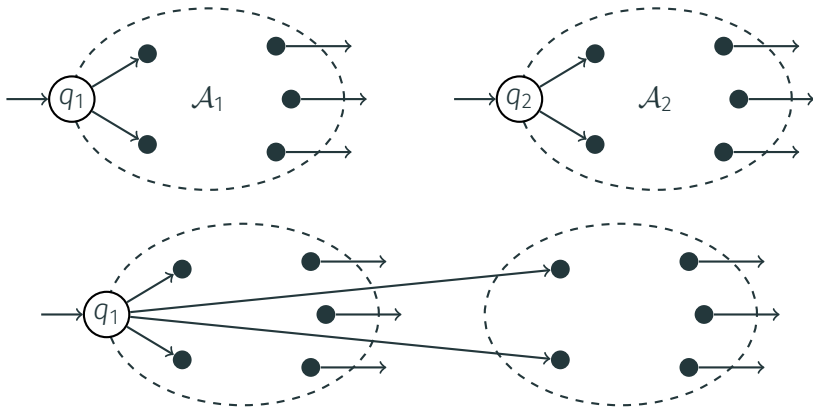
Preuve : union

Soient deux automates standards $\mathcal{A}_1$ et $\mathcal{A}_2$ qui reconnaissent $\mathcal{L}_1$ et $\mathcal{L}_2$.



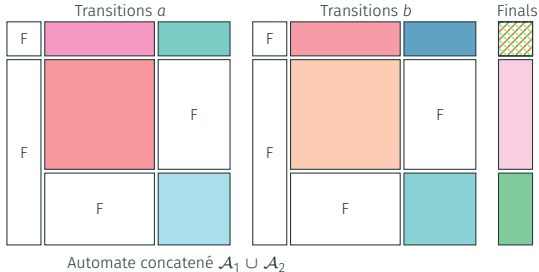
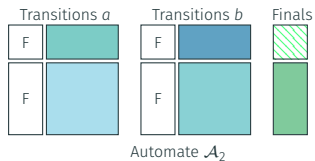
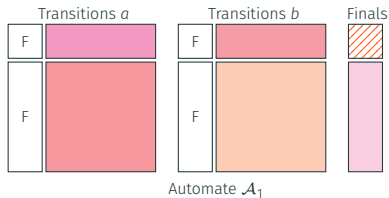
Preuve : union

Soient deux automates standards $\mathcal{A}_1$ et $\mathcal{A}_2$ qui reconnaissent $\mathcal{L}_1$ et $\mathcal{L}_2$.



NB : L'état q_2 peut être supprimé car il n'y a pas de transition entrante en q_2 .

Schéma d'implémentation : l'union



Implémentation python : l'union

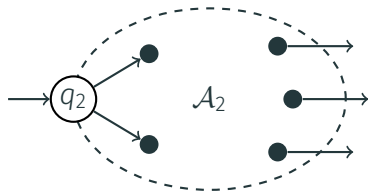
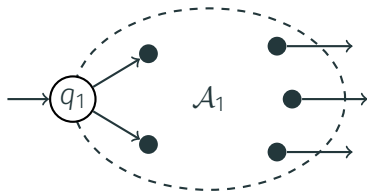
```
class Union(Rat):
    def __init__(self, L1, L2):
        self.op1 = L1
        self.op2 = L2
    def to_automaton(self):
        A1 = self.op1.to_automaton()
        A2 = self.op2.to_automaton()
        n1 = A1.n
        n2 = A2.n
        A = StandardAutomaton(n1+n2-1)
        A.copy_non_initial_transition_from(A1,1)
        A.copy_non_initial_transition_from(A2,n1)
        for t in range(1,n1):
            A.transitionA[0][t] = A1.transitionA[0][t]
            A.transitionB[0][t] = A1.transitionB[0][t]
        for t in range(1,n2):
            A.transitionA[0][n1+t-1] = A2.transitionA[0][t]
            A.transitionB[0][n1+t-1] = A2.transitionB[0][t]
        for f in A1 finals:
            A.add_final_state(f)
        for f in A2 finals:
            A.add_final_state(n1+f-1)
        return A
```

Proposition

Si $\mathcal{L}_1$ et $\mathcal{L}_2$ sont des langages reconnaissables, alors le langage $\mathcal{L}_1\mathcal{L}_2$ est reconnaissable.

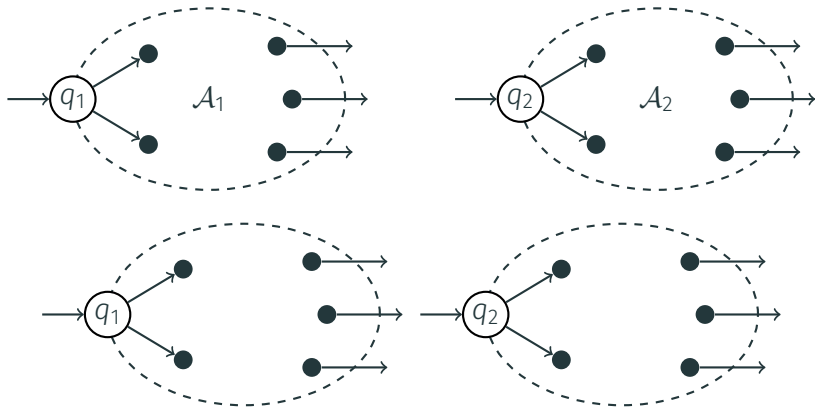
Preuve : concaténation

Soient deux automates standards $\mathcal{A}_1$ et $\mathcal{A}_2$ qui reconnaissent $\mathcal{L}_1$ et $\mathcal{L}_2$.



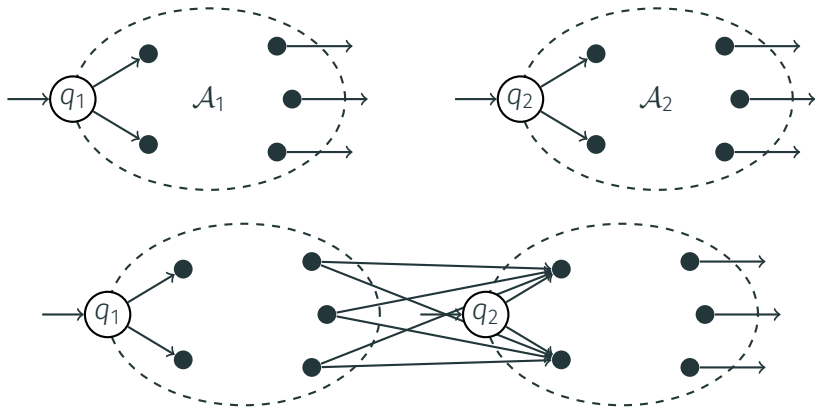
Preuve : concaténation

Soient deux automates standards $\mathcal{A}_1$ et $\mathcal{A}_2$ qui reconnaissent $\mathcal{L}_1$ et $\mathcal{L}_2$.



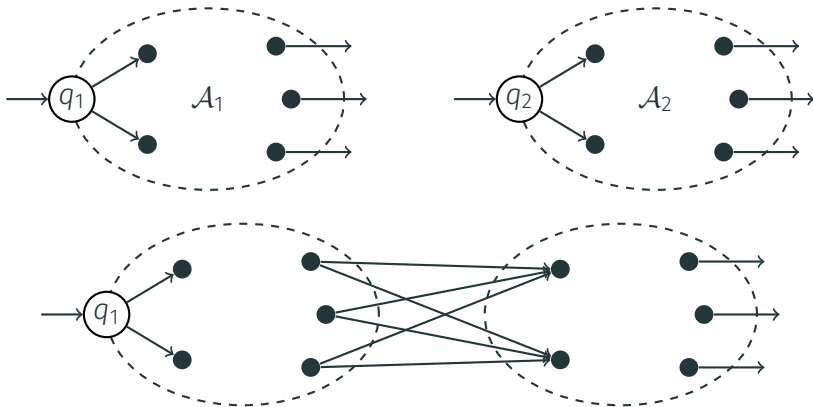
Preuve : concaténation

Soient deux automates standards $\mathcal{A}_1$ et $\mathcal{A}_2$ qui reconnaissent $\mathcal{L}_1$ et $\mathcal{L}_2$.



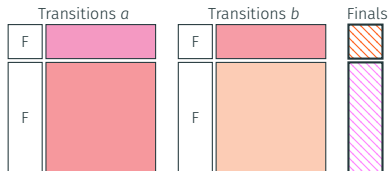
Preuve : concaténation

Soient deux automates standards $\mathcal{A}_1$ et $\mathcal{A}_2$ qui reconnaissent $\mathcal{L}_1$ et $\mathcal{L}_2$.

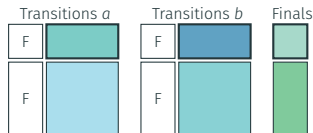


NB : L'état q_2 peut être supprimé car il n'y a pas de transition entrante en q_2 .

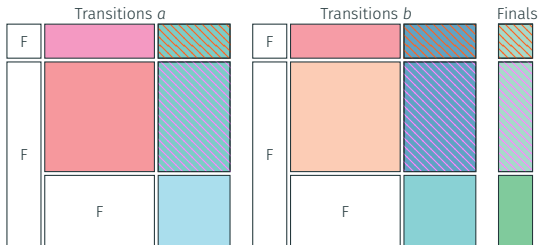
Schéma d'implémentation : la concaténation



Automate $\mathcal{A}_1$



Automate $\mathcal{A}_2$



Automate concaténé $\mathcal{A}_1\mathcal{A}_2$

On peut voir la construction comme un produit matriciel entre vecteur des états finaux de $\mathcal{A}_1$ et vecteur des transitions initiales de $\mathcal{A}_2$.

Implémentation python : la concaténation

```
class Concat(Rat):
    def __init__(self, L1, L2):
        self.op1 = L1
        self.op2 = L2
    def to_automaton(self):
        A1 = self.op1.to_automaton()
        A2 = self.op2.to_automaton()
        n1 = A1.n
        n2 = A2.n
        A = StandardAutomaton(n1+n2-1)
        A.copy_non_initial_transition_from(A1,1)
        A.copy_non_initial_transition_from(A2,n1)
        for t in range(1,n1):
            A.transitionA[0][t] = A1.transitionA[0][t]
            A.transitionB[0][t] = A1.transitionB[0][t]
        for f in A1.finals:
            for t in range(1,n2):
                A.transitionA[f][n1+t-1] = A2.transitionA[0][t]
                A.transitionB[f][n1+t-1] = A2.transitionB[0][t]
        for f in A2.finals:
            A.add_final_state(n1+f-1)
        return A
```

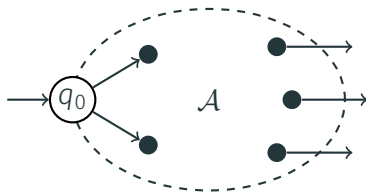
Étoile de Kleene d'un langage

Proposition

Si $\mathcal{L}$ est un langage reconnaissable, alors le langage $\mathcal{L}^*$ est reconnaissable.

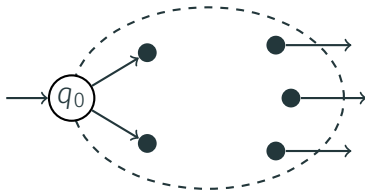
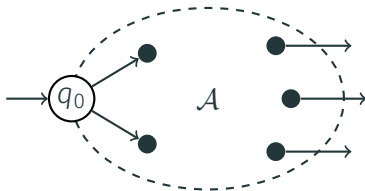
Preuve : étoile de Kleene

Soient un automate standard $\mathcal{A}$ qui reconnaît $\mathcal{L}$



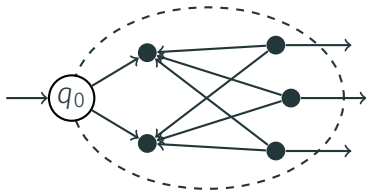
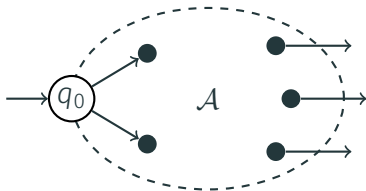
Preuve : étoile de Kleene

Soient un automate standard $\mathcal{A}$ qui reconnaît $\mathcal{L}$



Preuve : étoile de Kleene

Soient un automate standard $\mathcal{A}$ qui reconnaît $\mathcal{L}$



Preuve : étoile de Kleene

Soient un automate standard $\mathcal{A}$ qui reconnaît $\mathcal{L}$

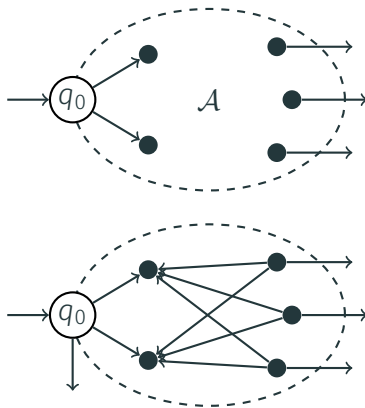
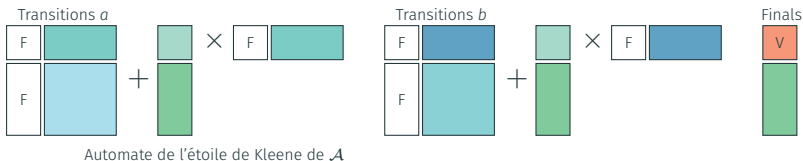
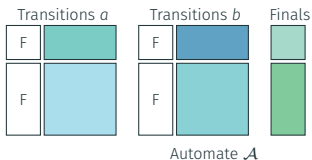


Schéma d'implémentation : la concaténation



On peut voir la construction comme un ajout du produit matriciel entre vecteur des états finaux et vecteur des transitions initiales de $\mathcal{A}_2$.

Implémentation python : l'étoile de Kleene

```
class Star(Rat):
    def __init__(self, L):
        self.op = L
    def to_automaton(self):
        A0 = self.op.to_automaton()
        n = A0.n
        A = StandardAutomaton(n)
        A.copy_non_initial_transition_from(A0,1)
        for t in range(1,n):
            A.transitionA[0][t] = A0.transitionA[0][t]
            A.transitionB[0][t] = A0.transitionB[0][t]
        A.finals = A0.finals.copy()
        A.add_final_state(0)
        for f in A0.finals:
            for t in range(1,n):
                if A0.transitionA[0][t]:
                    A.transitionA[f][t] = True
                if A0.transitionB[0][t]:
                    A.transitionB[f][t] = True
        return A
```

Théorème

Tout langage rationnel est reconnaissable par un automate.

Démonstration.

Par induction structurelle en appliquant les résultats précédents. □

1961



Viktor M. Glouckov
(1923†1982)

Théorème

Tout langage rationnel est reconnaissable par un automate.

Démonstration.

Par induction structurelle en appliquant les résultats précédents. □

La construction conduit à l'automate dit de **Glouckov**.

1986



Gérard Berry
(1948)

- Il existe un algorithme, dit de **Berry-Sethi**, qui permet d'obtenir efficacement l'automate de Glouchkov.



Ravi Sethi
(1947)

1968



Kenneth L. Thompson
(1943)

- Il existe un algorithme, dit de **Berry-Sethi**, qui permet d'obtenir efficacement l'automate de Glouchkov.
- Une construction alternative permet d'obtenir l'automate, dit de **Thomson**.

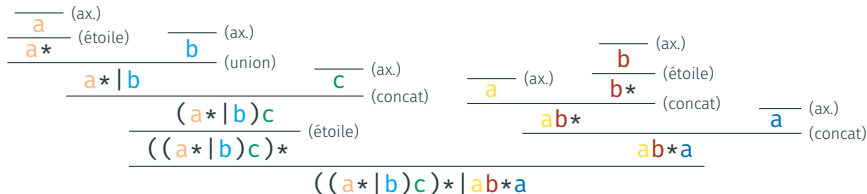
Exemple de compilation d'une regexp en automate

Construisons l'automate de Glouckov qui reconnaît le langage $\mathcal{L}(r)$ où r est l'expression rationnelle $r = ((a^* | b)c)^* | ab^*a$.

Exemple de compilation d'une regexp en automate

Construisons l'automate de Glouchkov qui reconnaît le langage $\mathcal{L}(r)$ où r est l'expression rationnelle $r = ((a^* | b)c)^* | ab^*a$.

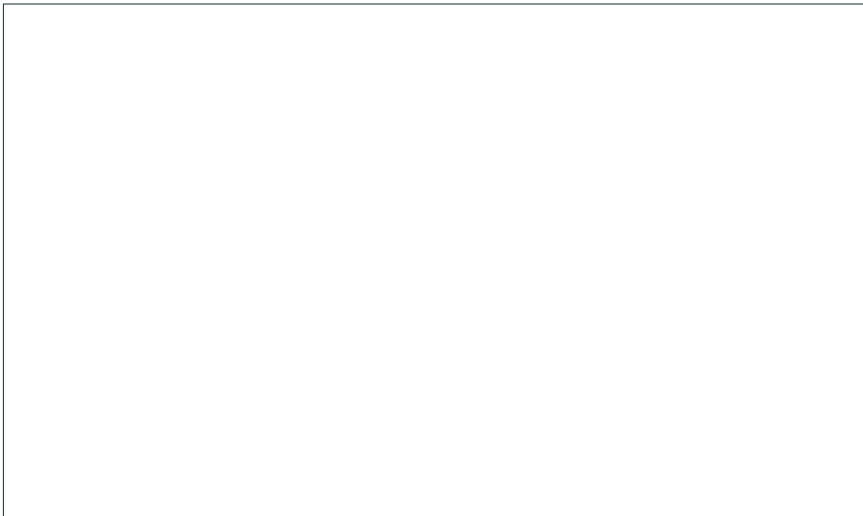
On commence par l'arbre de dérivation de l'expression régulière.



et on applique les règles de constructions dans le même ordre.

Exemple de compilation d'une regexp en automate

Construisons l'automate de Glouckov qui reconnaît le langage $\mathcal{L}(r)$ où r est l'expression rationnelle $r = ((a^* | b)c)^* | ab^*a$.



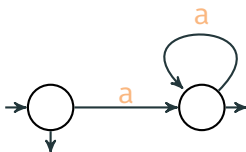
Exemple de compilation d'une regexp en automate

Construisons l'automate de Glouckov qui reconnaît le langage $\mathcal{L}(r)$ où r est l'expression rationnelle $r = ((a^* | b)c)^* | ab^*a$.



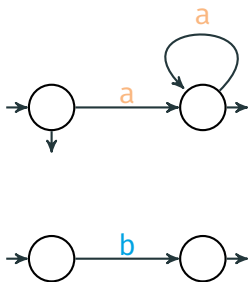
Exemple de compilation d'une regexp en automate

Construisons l'automate de Glouckov qui reconnaît le langage $\mathcal{L}(r)$ où r est l'expression rationnelle $r = ((a^* | b)c)^* | ab^*a$.



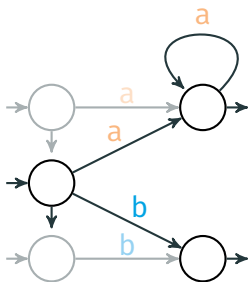
Exemple de compilation d'une regexp en automate

Construisons l'automate de Glouckov qui reconnaît le langage $\mathcal{L}(r)$ où r est l'expression rationnelle $r = ((a^* | b)c)^* | ab^*a$.



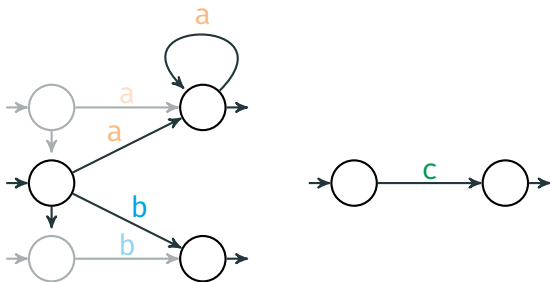
Exemple de compilation d'une regexp en automate

Construisons l'automate de Glouchkov qui reconnaît le langage $\mathcal{L}(r)$ où r est l'expression rationnelle $r = ((a^* | b)c)^* | ab^*a$.



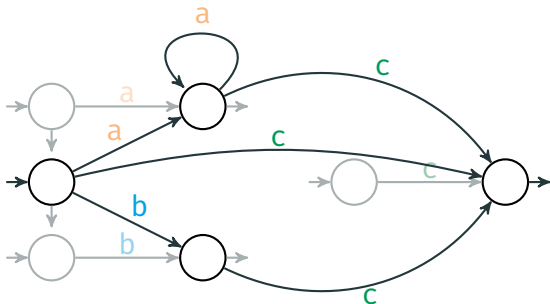
Exemple de compilation d'une regexp en automate

Construisons l'automate de Glouckov qui reconnaît le langage $\mathcal{L}(r)$ où r est l'expression rationnelle $r = ((a^* | b)c)^* | ab^*a$.



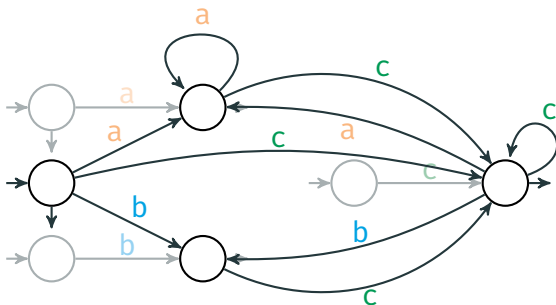
Exemple de compilation d'une regexp en automate

Construisons l'**automate de Glouckov** qui reconnaît le langage $\mathcal{L}(r)$ où r est l'expression rationnelle $r = ((a^* | b)c)^* | ab^*a$.



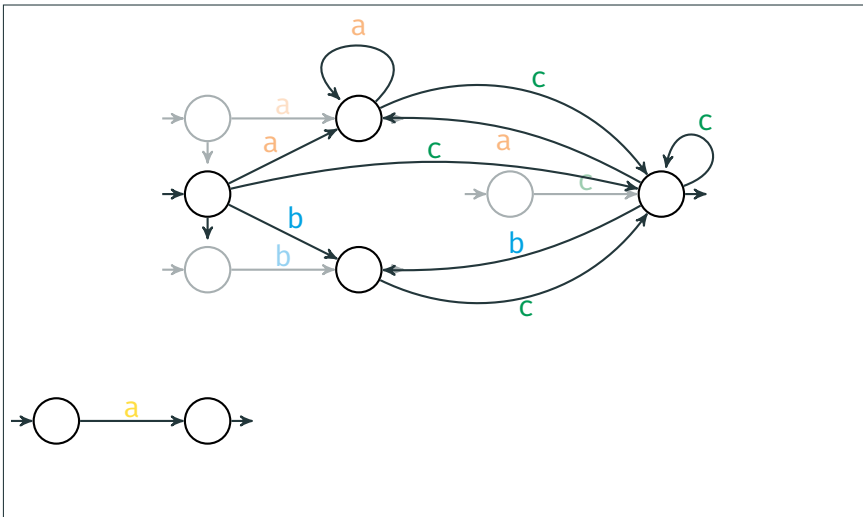
Exemple de compilation d'une regexp en automate

Construisons l'**automate de Glouchkov** qui reconnaît le langage $\mathcal{L}(r)$ où r est l'expression rationnelle $r = ((a^* | b)c)^* | ab^*a$.



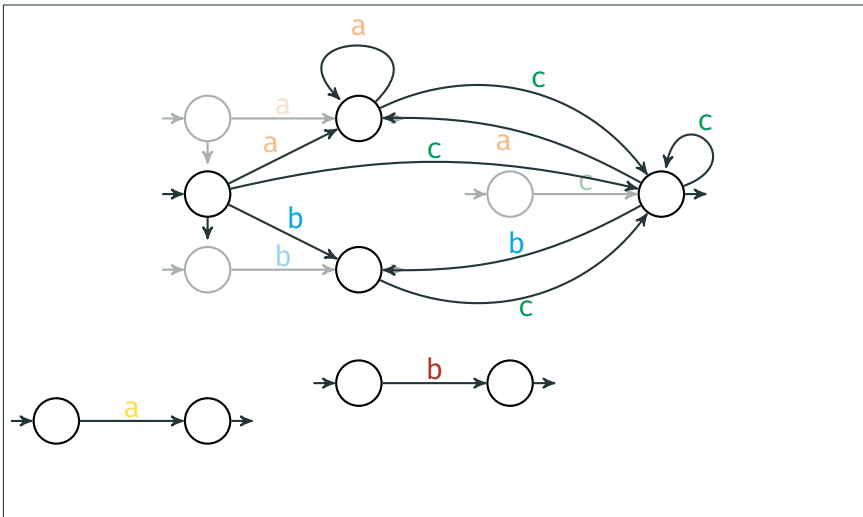
Exemple de compilation d'une regexp en automate

Construisons l'automate de Glouckov qui reconnaît le langage $\mathcal{L}(r)$ où r est l'expression rationnelle $r = ((a^* | b)c)^* | ab^*a$.



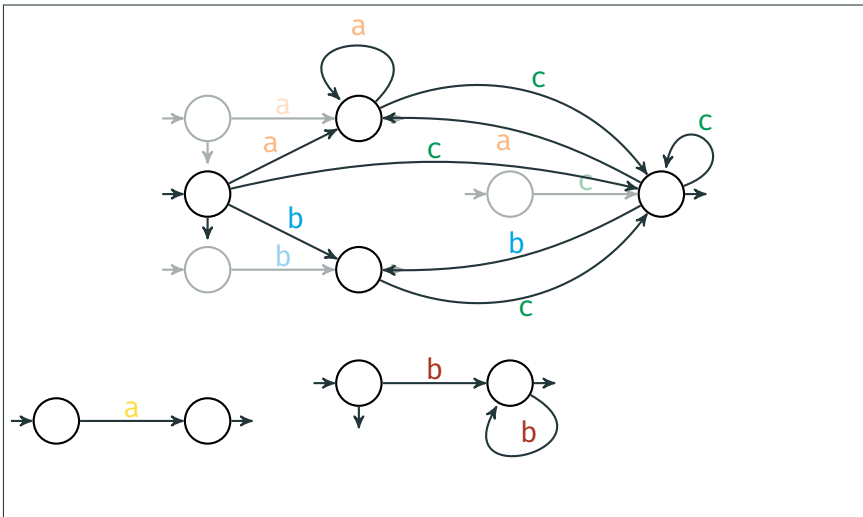
Exemple de compilation d'une regexp en automate

Construisons l'automate de Glouchkov qui reconnaît le langage $\mathcal{L}(r)$ où r est l'expression rationnelle $r = ((a^* | b)c)^* | ab^*a$.



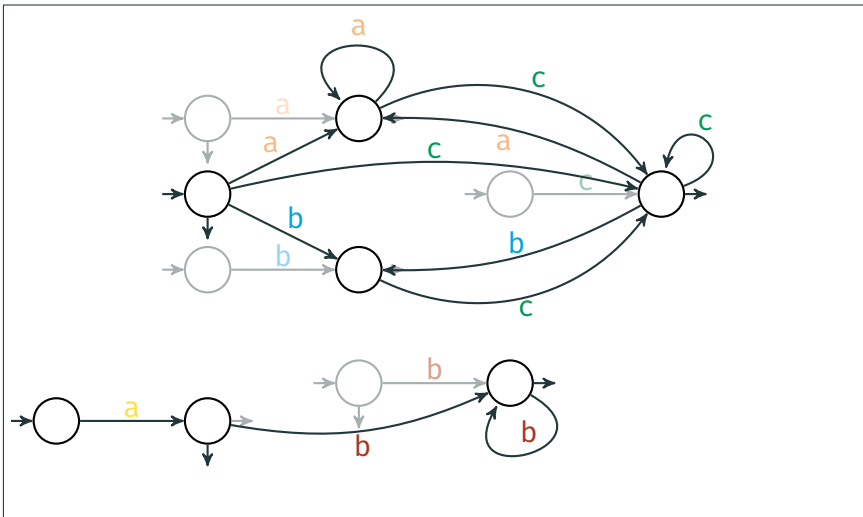
Exemple de compilation d'une regexp en automate

Construisons l'automate de Glouckov qui reconnaît le langage $\mathcal{L}(r)$ où r est l'expression rationnelle $r = ((a^* | b)c)^* | ab^*a$.



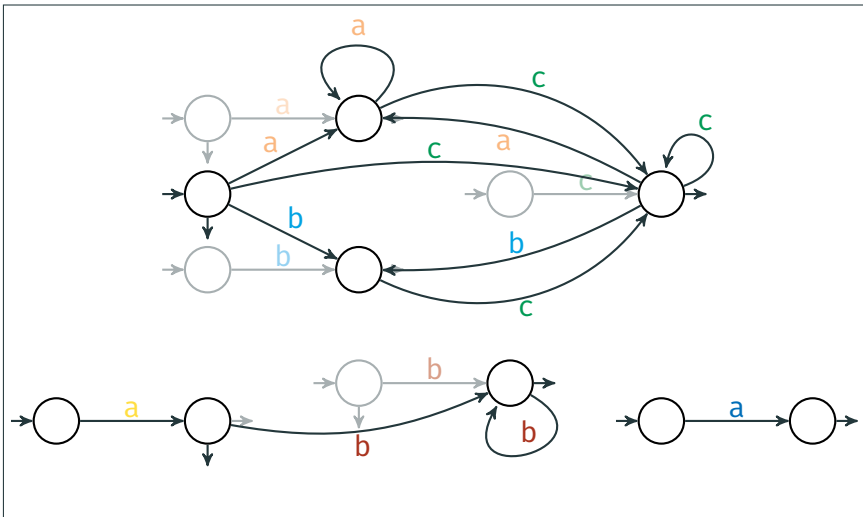
Exemple de compilation d'une regexp en automate

Construisons l'automate de Glouckov qui reconnaît le langage $\mathcal{L}(r)$ où r est l'expression rationnelle $r = ((a^* | b)c)^* | ab^*a$.



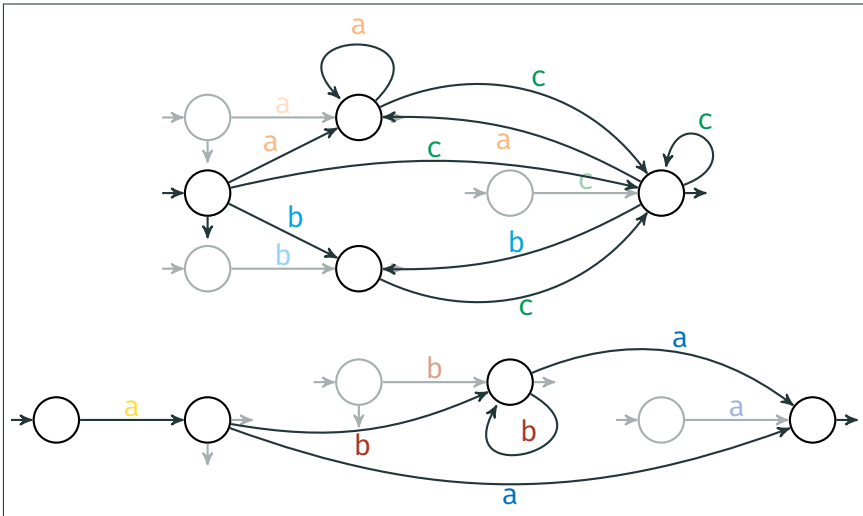
Exemple de compilation d'une regexp en automate

Construisons l'automate de Glouckov qui reconnaît le langage $\mathcal{L}(r)$ où r est l'expression rationnelle $r = ((a^* | b)c)^* | ab^*a$.



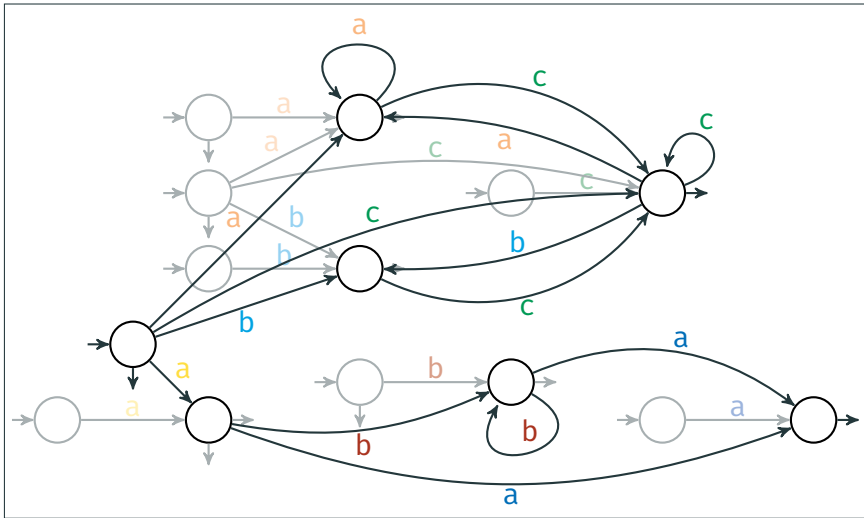
Exemple de compilation d'une regexp en automate

Construisons l'**automate de Glouchkov** qui reconnaît le langage $\mathcal{L}(r)$ où r est l'expression rationnelle $r = ((a^* | b)c)^* | ab^*a$.



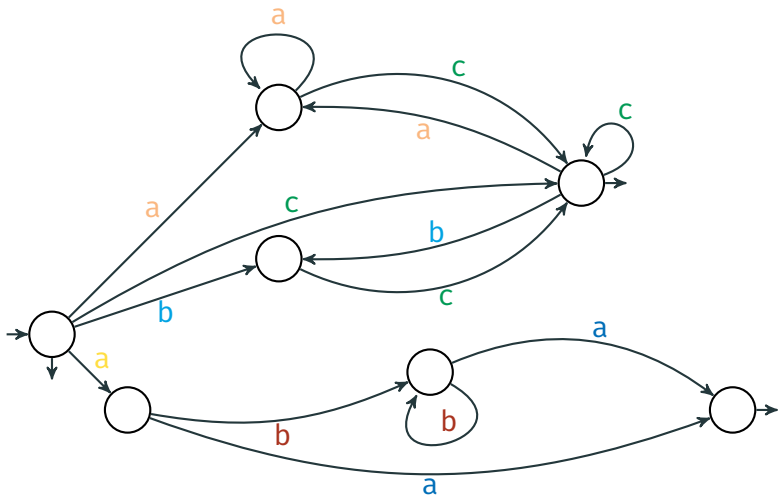
Exemple de compilation d'une regexp en automate

Construisons l'automate de Glouckov qui reconnaît le langage $\mathcal{L}(r)$ où r est l'expression rationnelle $r = ((a^* | b)c)^* | ab^*a$.



Exemple de compilation d'une regexp en automate

Construisons l'**automate de Glouchkov** qui reconnaît le langage $\mathcal{L}(r)$ où r est l'expression rationnelle $r = ((a^* | b)c)^* | ab^*a$.



- Dans l'automate de Glouchkov, il y a un état de plus que de lettres de Σ dans l'expression régulière.
- Chaque état non initial correspond à une lettre qui apparaît dans l'expression régulière.
- L'automate ne contient aucune transition spontanée.

Autres constructions

Proposition

Si $\mathcal{L}$ est un langage reconnaissable sur un alphabet Σ , alors son complémentaire $\Sigma^* \setminus \mathcal{L}$ l'est aussi.

Soit $\mathcal{A} = (Q, q_0, F, \delta)$ un DFA qui reconnaît $\mathcal{L}$.

Soit $\mathcal{A} = (Q, q_0, F, \delta)$ un DFA qui reconnaît $\mathcal{L}$. Alors $(Q, q_0, Q \setminus F, \delta)$ reconnaît $\Sigma^* \setminus \mathcal{L}$.

Proposition

Si $\mathcal{L}_1$ et $\mathcal{L}_2$ sont des langages reconnaissables, alors le langage $\mathcal{L}_1 \cap \mathcal{L}_2$ est reconnaissable.

Preuve : intersection

Soit $\mathcal{A}_1 = (Q_1, q_1, F_1, \delta_1)$ et $\mathcal{A}_2 = (Q_2, q_2, F_2, \delta_2)$ deux DFA qui reconnaissent $\mathcal{L}_1$ et $\mathcal{L}_2$.

Soit $\mathcal{A}_1 = (Q_1, q_1, F_1, \delta_1)$ et $\mathcal{A}_2 = (Q_2, q_2, F_2, \delta_2)$ deux DFA qui reconnaissent $\mathcal{L}_1$ et $\mathcal{L}_2$.

On construit un DFA sur $Q_1 \times Q_2$ en posant

$$\delta((q, q'), x) = (\delta_1(q, x), \delta_2(q', x)).$$

Preuve : intersection

Soit $\mathcal{A}_1 = (Q_1, q_1, F_1, \delta_1)$ et $\mathcal{A}_2 = (Q_2, q_2, F_2, \delta_2)$ deux DFA qui reconnaissent $\mathcal{L}_1$ et $\mathcal{L}_2$.

On construit un DFA sur $Q_1 \times Q_2$ en posant

$$\delta((q, q'), x) = (\delta_1(q, x), \delta_2(q', x)).$$

Avec $F = F_1 \times F_2$ et (q_1, q_2) comme état initial.

Variations autour de la même construction

Proposition

Si $\mathcal{L}_1$ et $\mathcal{L}_2$ sont des langages reconnaissables, alors les langages $\mathcal{L}_1 \cup \mathcal{L}_2$ et $\mathcal{L}_1 \Delta \mathcal{L}_2$ (différence symétrique) sont reconnaissables.

Démonstration.

Même construction de l'automate produit en prenant

$F = F_1 \times Q_2 \cup Q_1 \times F_2$ et $F = F_1 \times F_2^c \cup F_1^c \times F_2$ respectivement.

□

D'un automate à une expression
régulière. Élimination des états

Définition

Un automate fini à transitions étiquetées par des expressions rationnelles est un quintuplet $(\Sigma, Q, I, \delta, F)$ où

Définition

Un automate fini à transitions étiquetées par des expressions rationnelles est un quintuplet $(\Sigma, Q, I, \delta, F)$ où

- Σ est un alphabet

Définition

Un automate fini à transitions étiquetées par des expressions rationnelles est un quintuplet $(\Sigma, Q, I, \delta, F)$ où

- Σ est un alphabet
- Q est un ensemble fini, appelé ensemble d'états

Définition

Un automate fini à transitions étiquetées par des expressions rationnelles est un quintuplet $(\Sigma, Q, I, \delta, F)$ où

- Σ est un alphabet
- Q est un ensemble fini, appelé ensemble d'états
- $I \subseteq Q$ est un ensemble d'états particuliers, appelés états initiaux

Définition

Un automate fini à transitions étiquetées par des expressions rationnelles est un quintuplet $(\Sigma, Q, I, \delta, F)$ où

- Σ est un alphabet
- Q est un ensemble fini, appelé ensemble d'états
- $I \subseteq Q$ est un ensemble d'états particuliers, appelés états initiaux
- $\delta : Q \times \text{Regexp}(\Sigma) \times Q$, où $\text{Regexp}(\Sigma)$ désigne l'ensemble des expressions rationnelles sur Σ , est une relation de transition finie.

Définition

Un **automate fini** à transitions étiquetées par des expressions **rationnelles** est un quintuplet $(\Sigma, Q, I, \delta, F)$ où

- Σ est un **alphabet**
- Q est un ensemble fini, appelé ensemble d'**états**
- $I \subseteq Q$ est un ensemble d'états particuliers, appelés **états initiaux**
- $\delta : Q \times \text{Regexp}(\Sigma) \times Q$, où $\text{Regexp}(\Sigma)$ désigne l'ensemble des expressions rationnelles sur Σ , est une **relation de transition** finie.
- $F \subseteq Q$ est un ensemble d'**états** dits **finaux** ou acceptants

Définition

Un **automate fini à transitions étiquetées par des expressions rationnelles** est un quintuplet $(\Sigma, Q, I, \delta, F)$ où

- Σ est un **alphabet**
- Q est un ensemble fini, appelé ensemble d'**états**
- $I \subseteq Q$ est un ensemble d'états particuliers, appelés **états initiaux**
- $\delta : Q \times \text{Regexp}(\Sigma) \times Q$, où $\text{Regexp}(\Sigma)$ désigne l'ensemble des expressions rationnelles sur Σ , est une **relation de transition** finie.
- $F \subseteq Q$ est un ensemble d'**états** dits **finaux** ou acceptants

Fonctionnement : d'un état à un autre, l'automate peut consommer un groupe de lettres accepté par l'expression régulière.

Equivalence avec les autres classes d'automates

Proposition

Pour tout RNFA, il existe un ϵ NFA équivalent.

Démonstration.

Remplacer toute transition étiquetée par une expression rationnelle r par un automate reconnaissant r . □

Proposition

Pour tout RNFA $\mathcal{A}$, il existe une expression régulière r telle que

$$\mathcal{L}(\mathcal{A}) = \mathcal{L}(r).$$

Proposition

Pour tout RNFA $\mathcal{A}$, il existe une expression régulière r telle que

$$\mathcal{L}(\mathcal{A}) = \mathcal{L}(r).$$

Démonstration.

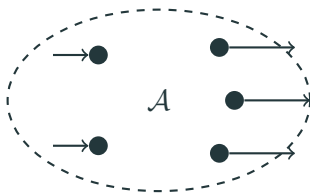
Idee générale : se ramener à un RNFA de la forme



avec $r \in \text{Regexp}(\Sigma)$.

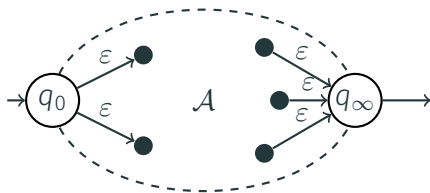


On commence par emballer l'automate RNFA entre deux états.



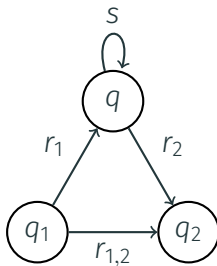
Détails de la preuve

On commence par emballer l'automate RNFA entre deux états.



Détails de la preuve

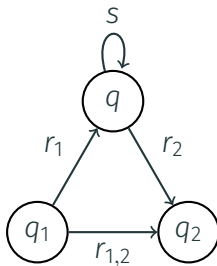
Afin de supprimer l'état q , pour tous états q_1 prédécesseurs de q et tous les états q_2 successeurs de q ,



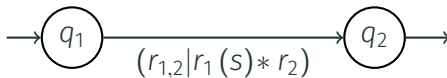
devient

Détails de la preuve

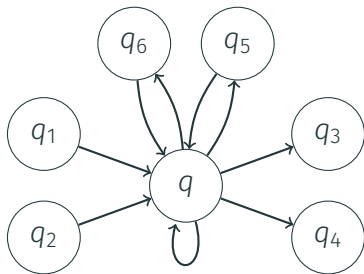
Afin de supprimer l'état q , pour tous états q_1 prédécesseurs de q et tous les états q_2 successeurs de q ,



devient

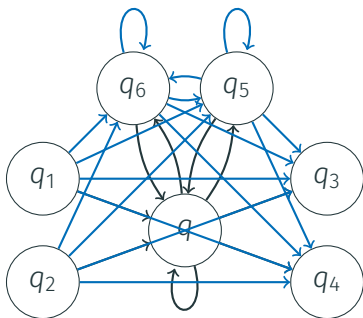


Attention à considérer tous les prédécesseurs et tous les successeurs

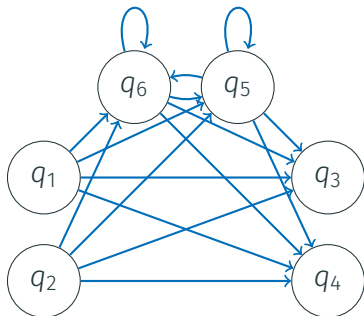


Détails de la preuve

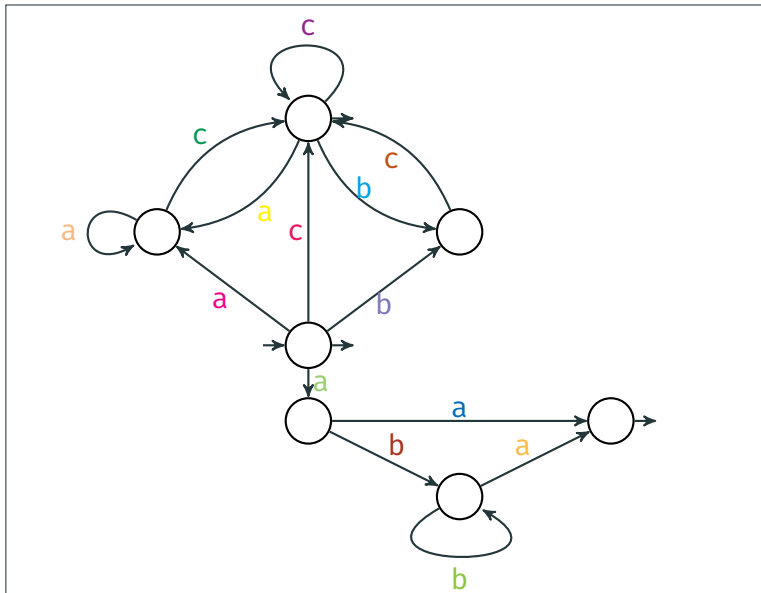
Attention à considérer tous les prédécesseurs et tous les successeurs



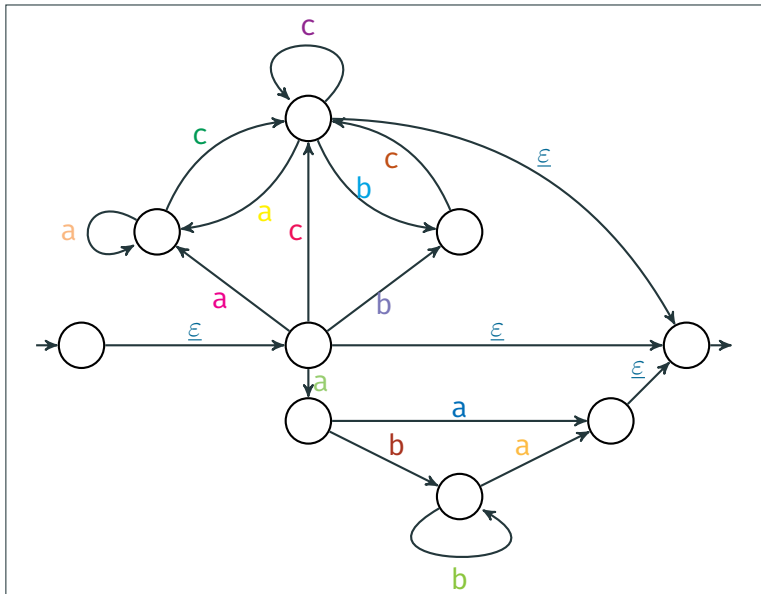
Attention à considérer tous les prédécesseurs et tous les successeurs



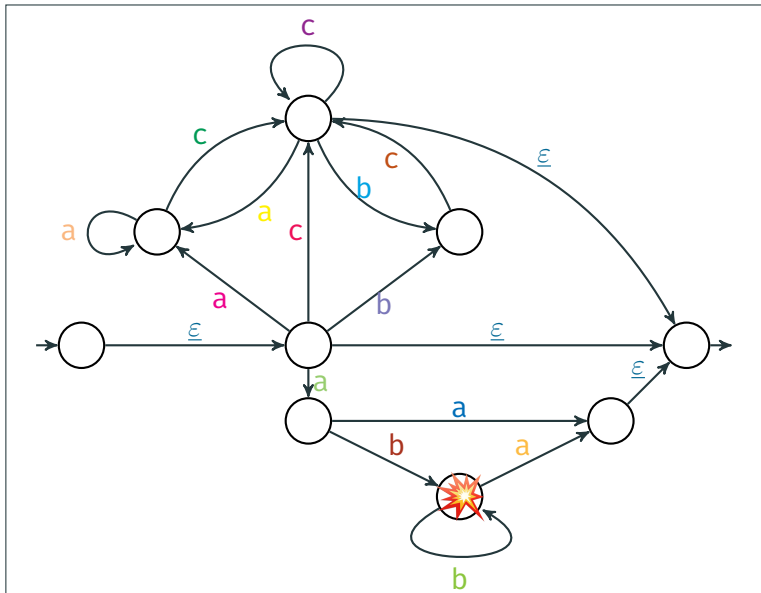
Exemple



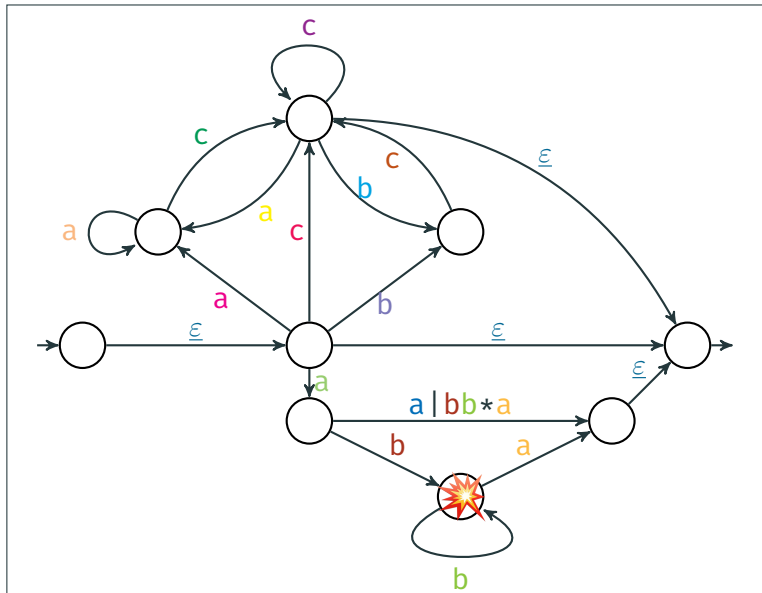
Example



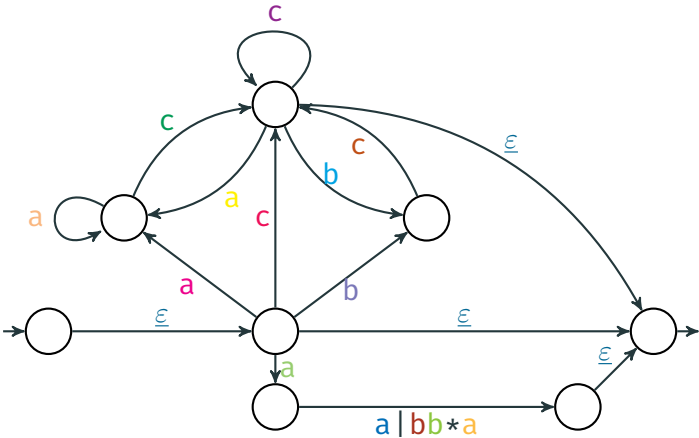
Example



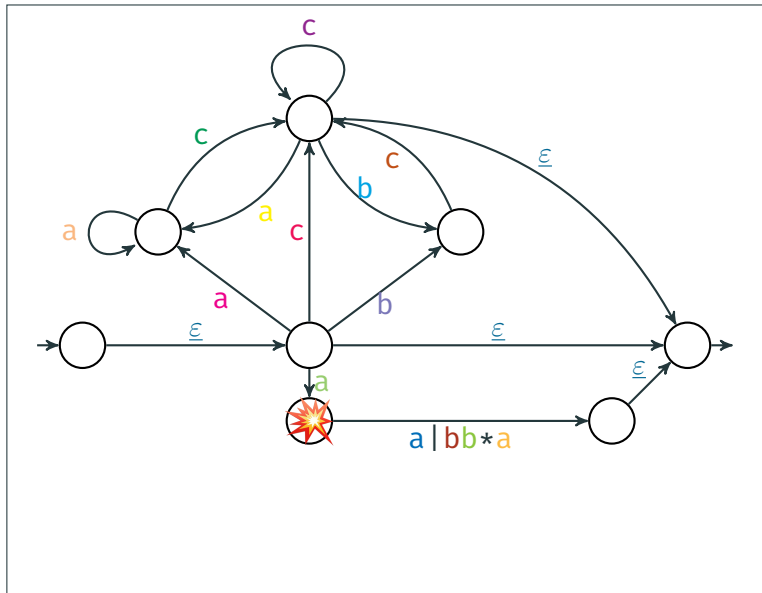
Example



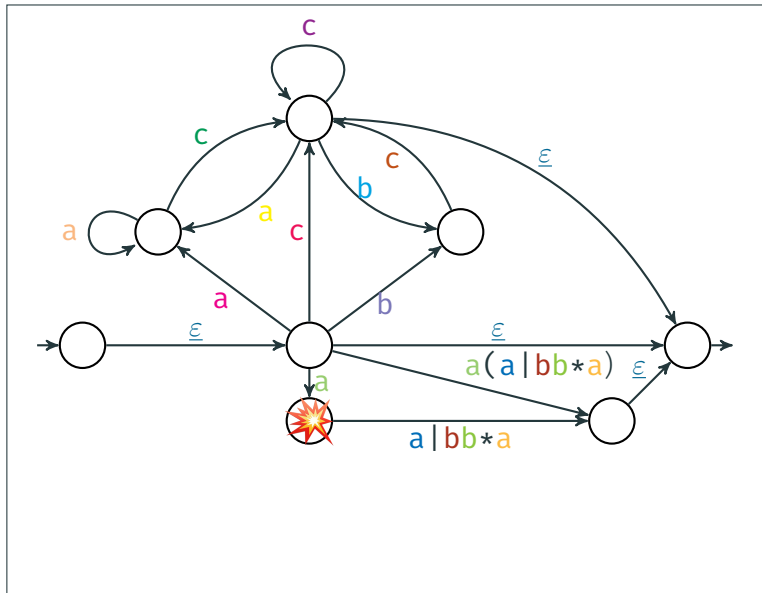
Example



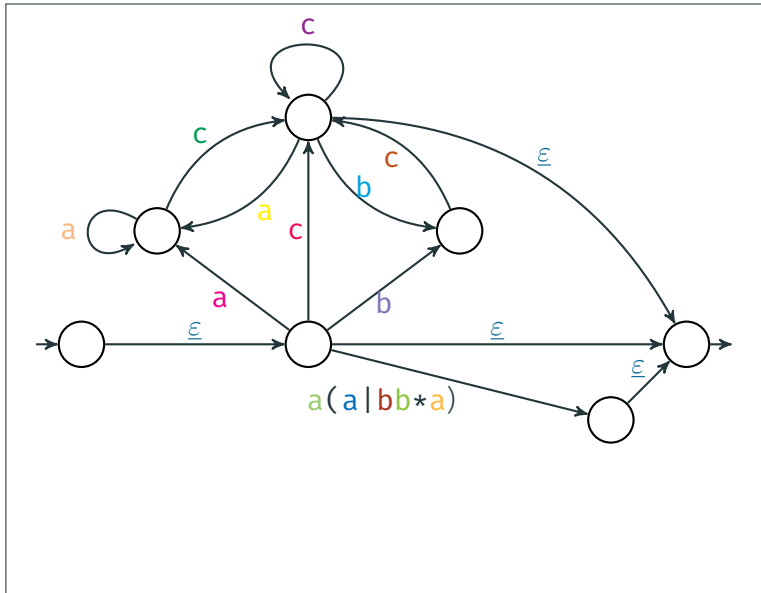
Example



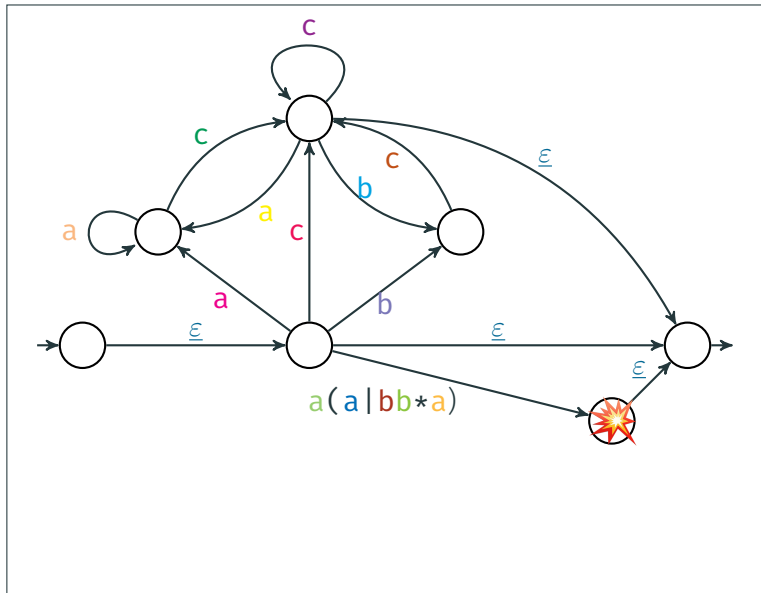
Example



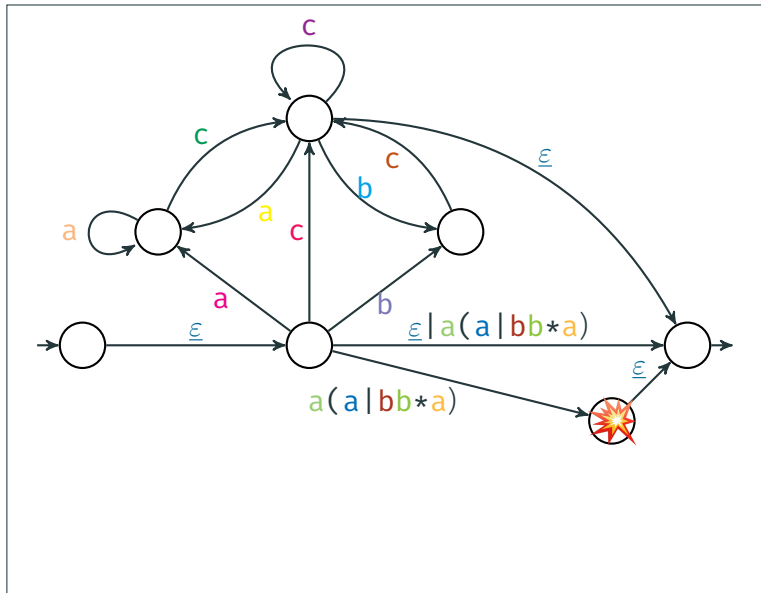
Example



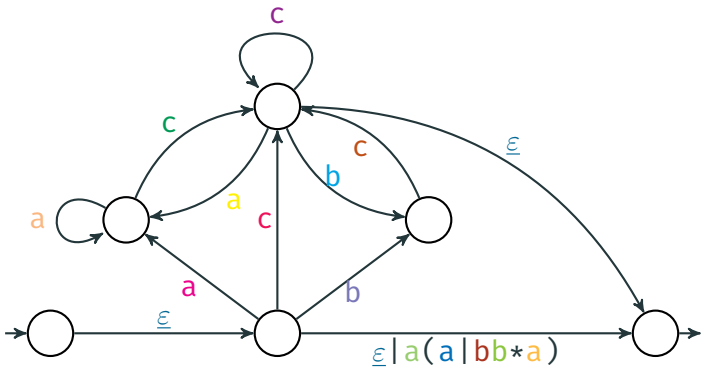
Example



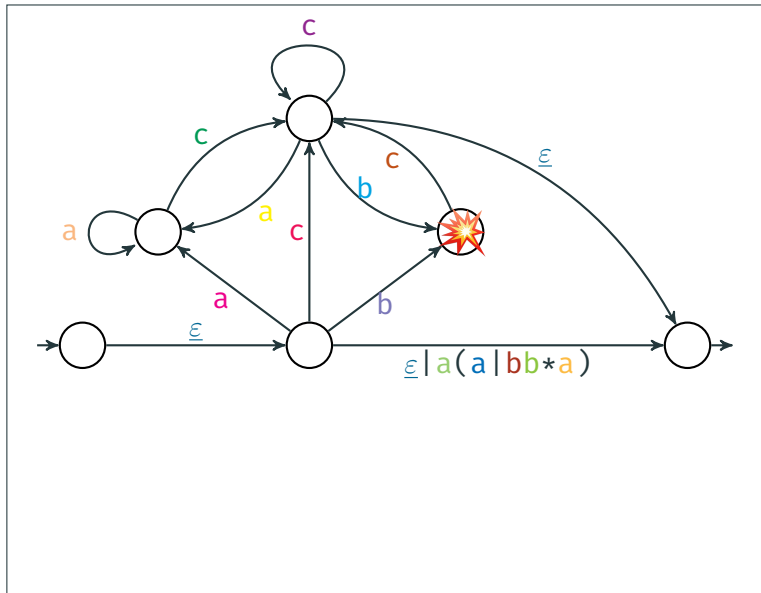
Example



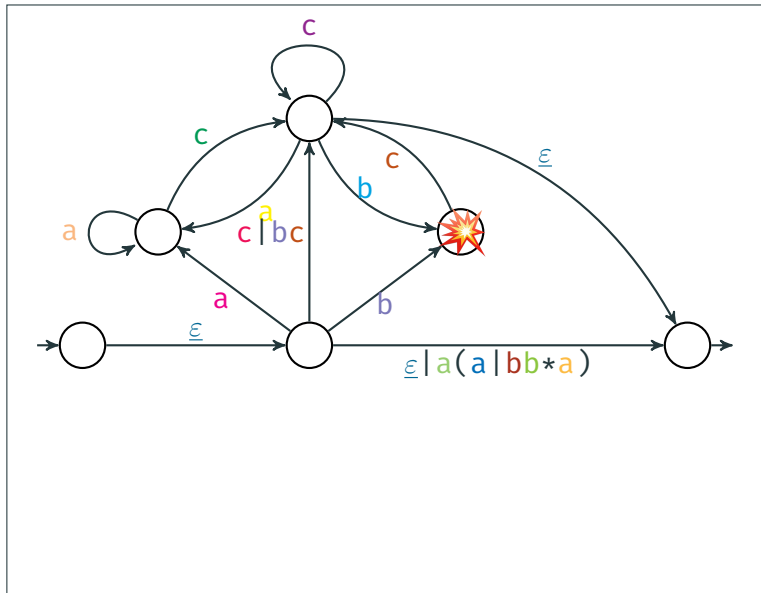
Example



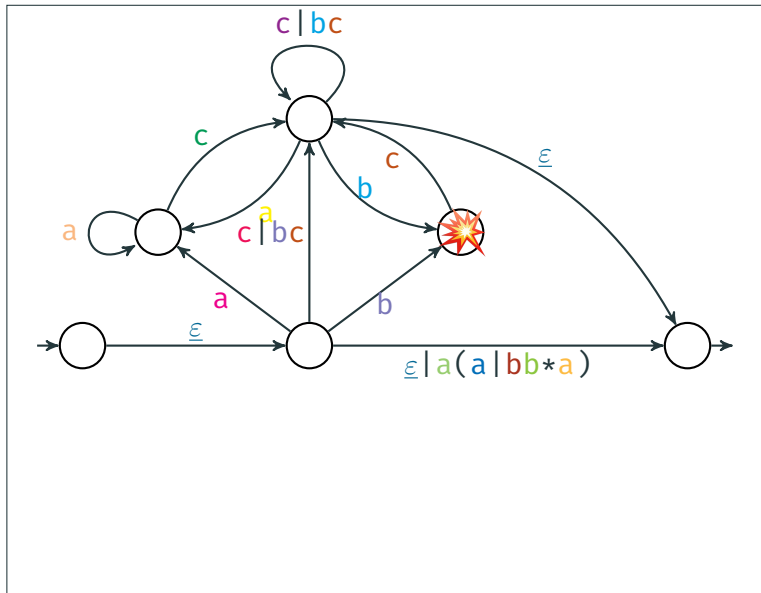
Example



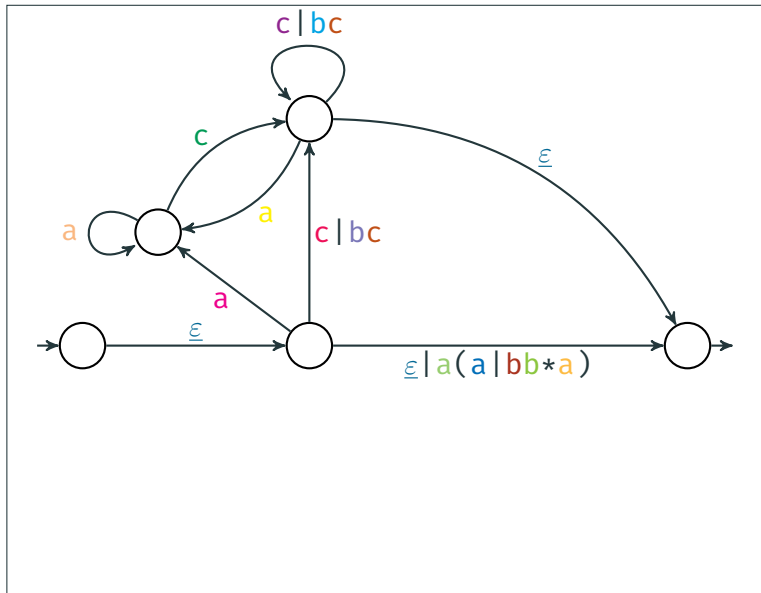
Example



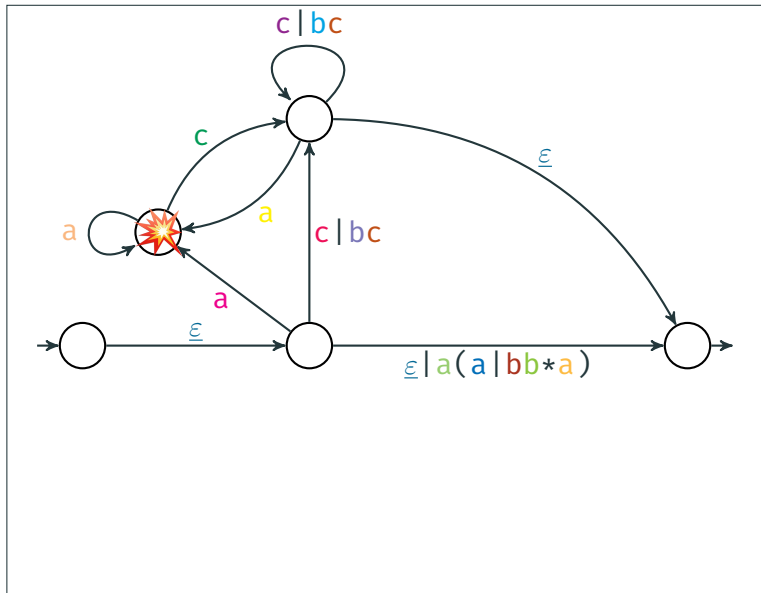
Example



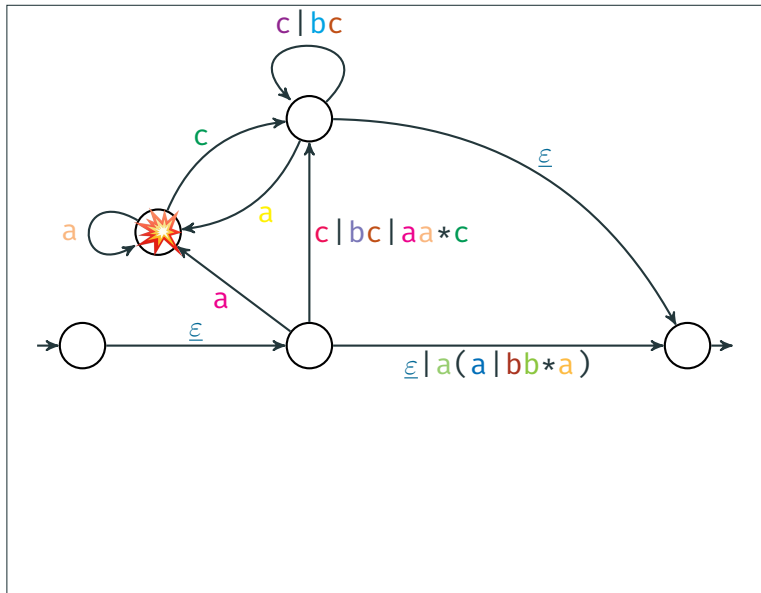
Example



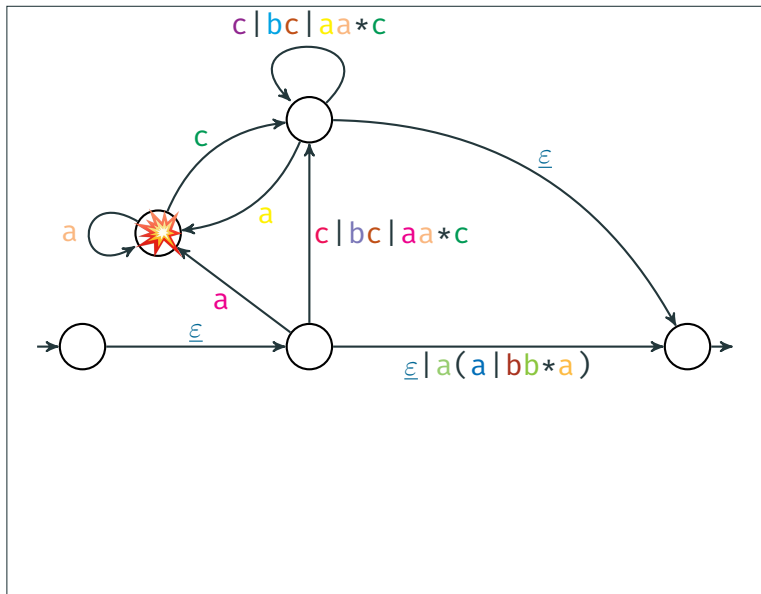
Exemple



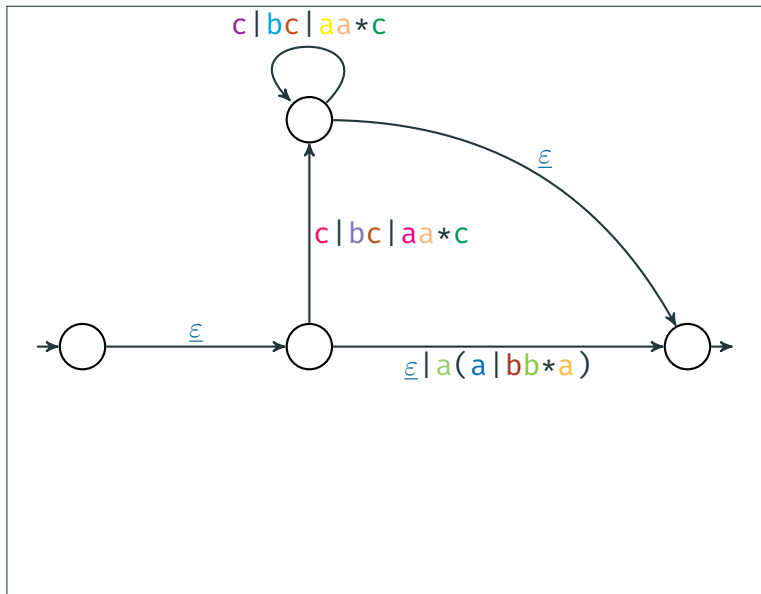
Example



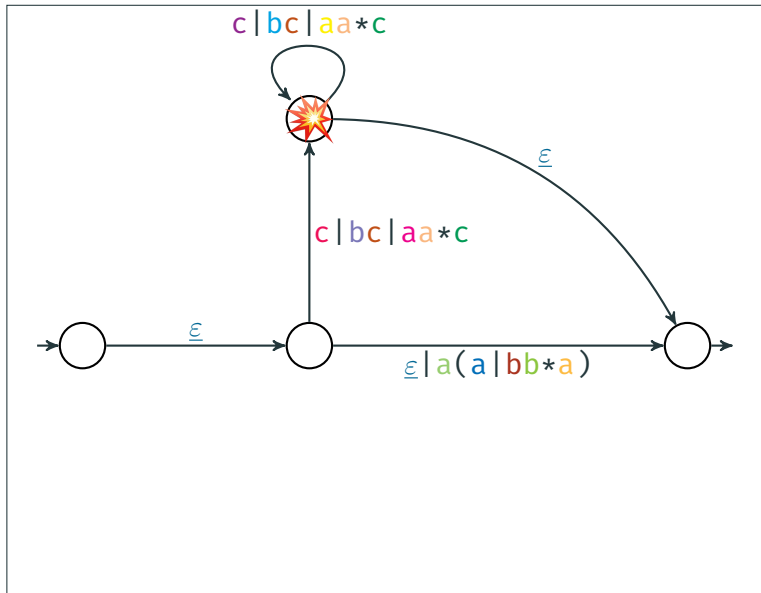
Exemple



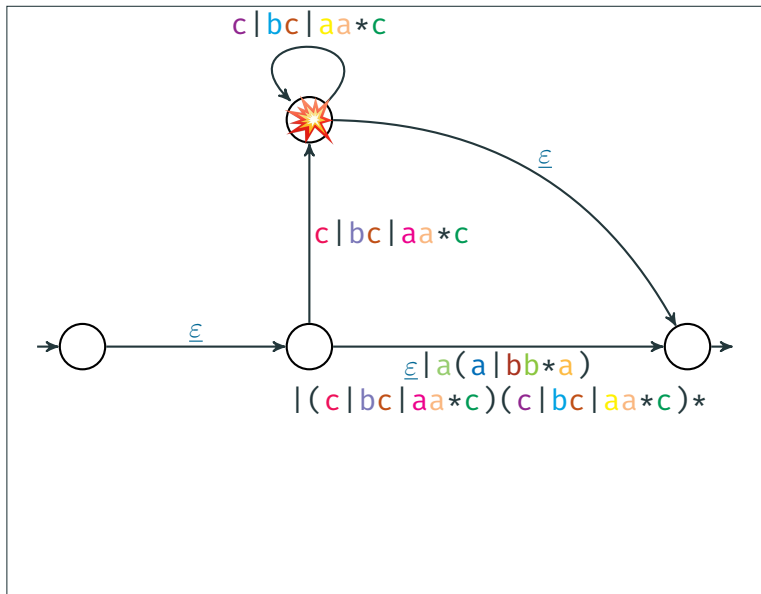
Exemple



Exemple



Exemple



Example



Exemple



1963



Janusz A. Brzozowski
(1935†2019)

- L'algorithme présenté dans le cours s'appelle parfois la **méthode de Brzozowski et McCluskey**.



Edward J. McCluskey
(1929†2016)

1960



Robert F. McNaughton
(1924†2014)

- L'algorithme présenté dans le cours s'appelle parfois la **méthode de Brzozowski et McCluskey**.
- Il existe une méthode alternative, dite de **McNaughton et Yamada**.



Hisao Yamada
(1930†2008)

1956



Stephen C. Kleene
(1909†1994)

Théorème (Kleene)

*L'ensemble des langages rationnels et celui des langages reconnaissables **coïncident**.*

Détection du caractère non-rationnel d'un langage

Proposition

Soit $\mathcal{L}$ un langage rationnel. Il existe alors un entier k tel que tout mot $t \in \mathcal{L}$ de longueur $|t| \geq k$ admette une factorisation $t = uvw$ en trois facteurs $u, v, w \in \Sigma^*$ où

Proposition

Soit $\mathcal{L}$ un langage rationnel. Il existe alors un entier k tel que tout mot $t \in \mathcal{L}$ de longueur $|t| \geq k$ admette une factorisation $t = uvw$ en trois facteurs $u, v, w \in \Sigma^*$ où

1. $|v| \geq 1$

Proposition

Soit $\mathcal{L}$ un langage rationnel. Il existe alors un entier k tel que tout mot $t \in \mathcal{L}$ de longueur $|t| \geq k$ admette une factorisation $t = uvw$ en trois facteurs $u, v, w \in \Sigma^*$ où

1. $|v| \geq 1$
2. $|uv| \leq k$

Proposition

Soit $\mathcal{L}$ un langage rationnel. Il existe alors un entier k tel que tout mot $t \in \mathcal{L}$ de longueur $|t| \geq k$ admette une factorisation $t = uvw$ en trois facteurs $u, v, w \in \Sigma^*$ où

1. $|v| \geq 1$
2. $|uv| \leq k$
3. pour tout entier i , on a $uv^i w \in \mathcal{L}$.

Démonstration

Soit $\mathcal{A}$ un DFA reconnaissant $\mathcal{L}$ et k son nombre d'état.

Démonstration

Soit $\mathcal{A}$ un DFA reconnaissant $\mathcal{L}$ et k son nombre d'état.

Soit $t = x_1x_2 \dots x_n \in \mathcal{L}$ avec $n \geq k$.

Démonstration

Soit $\mathcal{A}$ un DFA reconnaissant $\mathcal{L}$ et k son nombre d'état.

Soit $t = x_1x_2 \dots x_n \in \mathcal{L}$ avec $n \geq k$.

Notons $q_0, \dots, q_n$ les états traversés.

Démonstration

Soit $\mathcal{A}$ un DFA reconnaissant $\mathcal{L}$ et k son nombre d'état.

Soit $t = x_1x_2 \dots x_n \in \mathcal{L}$ avec $n \geq k$.

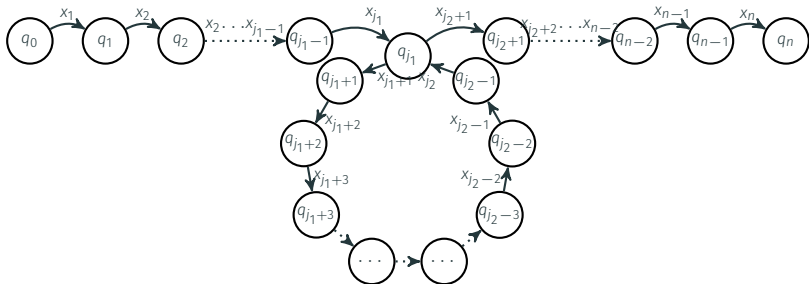
Notons $q_0, \dots, q_n$ les états traversés. Par le lemme des tiroirs, il existe $j_1 < j_2 \leq k$ tels que $q_{j_1} = q_{j_2}$.

Démonstration

Soit $\mathcal{A}$ un DFA reconnaissant $\mathcal{L}$ et k son nombre d'état.

Soit $t = x_1x_2 \dots x_n \in \mathcal{L}$ avec $n \geq k$.

Notons $q_0, \dots, q_n$ les états traversés. Par le lemme des tiroirs, il existe $j_1 < j_2 \leq k$ tels que $q_{j_1} = q_{j_2}$.

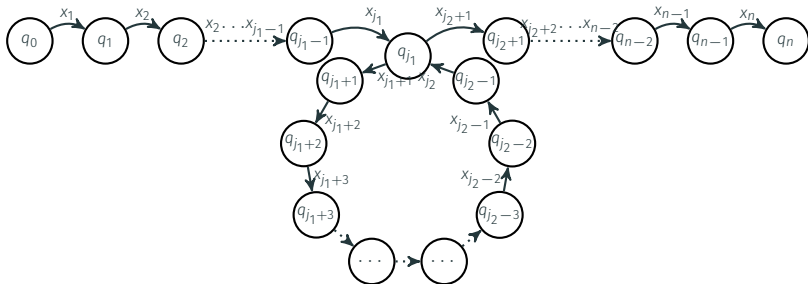


Démonstration

Soit $\mathcal{A}$ un DFA reconnaissant $\mathcal{L}$ et k son nombre d'état.

Soit $t = x_1x_2 \dots x_n \in \mathcal{L}$ avec $n \geq k$.

Notons $q_0, \dots, q_n$ les états traversés. Par le lemme des tiroirs, il existe $j_1 < j_2 \leq k$ tels que $q_{j_1} = q_{j_2}$.



La décomposition $u = x_1 \dots x_{j_1}$, $v = x_{j_1+1} \dots x_{j_2}$ et $w = x_{j_2+1} \dots x_n$ convient.

Usage du lemme de pompage

Le lemme de pompage permet de montrer qu'un langage n'est pas rationnel.

Usage du lemme de pompage

Le lemme de pompage permet de montrer qu'un langage n'est pas rationnel.

Attention à bien nier les conclusions du lemme!

Exemple d'usage du lemme de pompage

Exemple

Montrer que n'est pas rationnel le langage sur $\Sigma = \{a, b\}$

$$\mathcal{L} = \{a^n b^n; n \in \mathbb{N}\}.$$

Exemple d'usage du lemme de pompage

Exemple

Montrer que **n'est pas rationnel** le langage sur $\Sigma = \{a, b\}$

$$\mathcal{L} = \{a^n b^n; n \in \mathbb{N}\}.$$

Conséquence pratique : on ne peut pas gérer l'imbrication de boucles **if / endif** par un automate fini.

Exemple d'usage du lemme de pompage

Exemple

Montrer que **n'est pas rationnel** le langage sur $\Sigma = \{a, b\}$

$$\mathcal{L} = \{a^n b^n; n \in \mathbb{N}\}.$$

Conséquence pratique : on ne peut pas gérer l'imbrication de boucles **if / endif** par un automate fini.

Démonstration.

Supposons que $\mathcal{L}$ est rationnel. Soit k l'entier donné par le lemme de pompage. Appliquons le lemme au mot $t = a^k b^k$ et tirons la factorisation $t = uvw$ avec $v \neq \epsilon$.

Exemple d'usage du lemme de pompage

Exemple

Montrer que **n'est pas rationnel** le langage sur $\Sigma = \{a, b\}$

$$\mathcal{L} = \{a^n b^n; n \in \mathbb{N}\}.$$

Conséquence pratique : on ne peut pas gérer l'imbrication de boucles **if / endif** par un automate fini.

Démonstration.

Supposons que $\mathcal{L}$ est rationnel. Soit k l'entier donné par le lemme de pompage. Appliquons le lemme au mot $t = a^k b^k$ et tirons la factorisation $t = uvw$ avec $v \neq \epsilon$.

Comme $|uv| \leq k$, v ne contient que des **a**. Mais alors, d'après le lemme $uv^0w = a^{k-|v|}b^k$ appartient à $\mathcal{L}$ et $|v| = 0$, ce qui n'est pas le cas.

Contradiction !



Minimisation

Relation de séparabilité

Définition

Soit $\mathcal{L}$ un langage. On dit que deux mots w et $w' \in \Sigma^*$ sont **inséparables**, et on note $w \equiv w'$, si pour tout suffixe $z \in \Sigma^*$, les mots wz et $w'z$ simultanément appartiennent à $\mathcal{L}$ ou bien simultanément n'appartiennent pas à $\mathcal{L}$.

Relation de séparabilité

Définition

Soit $\mathcal{L}$ un langage. On dit que deux mots w et $w' \in \Sigma^*$ sont **inséparables**, et on note $w \equiv w'$, si pour tout suffixe $z \in \Sigma^*$, les mots wz et $w'z$ simultanément appartiennent à $\mathcal{L}$ ou bien simultanément n'appartiennent pas à $\mathcal{L}$.

Deux mots inséparables ont le même comportement vis-à-vis de $\mathcal{L}$ quand on les complète identiquement.

Remarque

En appelant **résiduel de $\mathcal{L}$ par w** l'ensemble

$$w^{-1}\mathcal{L} = \{t \in \Sigma^*; wt \in \mathcal{L}\},$$

deux mots $w \equiv w'$ sont inséparables ssi leurs résiduels sont égaux

$$w^{-1}\mathcal{L} = w'^{-1}\mathcal{L}.$$

Théorème de Myhill et Nerode

1958



John R. Myhill
(1923+1987)

Théorème

Un langage $\mathcal{L}$ est rationnel si et seulement si la relation d'inséparabilité ne possède qu'un nombre fini k de classes d'équivalences.



Anil Nerode
(1932)

Théorème de Myhill et Nerode

1958



John R. Myhill
(1923-1987)

Théorème

Un langage $\mathcal{L}$ est rationnel si et seulement si la relation d'inséparabilité ne possède qu'un *nombre fini k de classes d'équivalences*.

Dans ce cas, il existe un unique DFA à k états qui reconnaît $\mathcal{L}$.



Anil Nerode
(1932)

Théorème de Myhill et Nerode

1958



John R. Myhill
(1923+1987)

Théorème

Un langage $\mathcal{L}$ est rationnel si et seulement si la relation d'inséparabilité ne possède qu'un nombre fini k de classes d'équivalences.

Dans ce cas, il existe un unique DFA à k états qui reconnaît $\mathcal{L}$.

On note $Q = \Sigma^* / \equiv$ les classes d'équivalences.



Anil Nerode
(1932)

Démonstration (Sens $\Leftarrow$)

Observation clé

Si les mots w et $w' \in \Sigma^*$ sont inséparables, alors, pour toute lettre $x \in \Sigma$, les mots wx et $w'x$ sont aussi inséparables.

Démonstration (Sens $\Leftarrow$)

Observation clé

Si les mots w et $w' \in \Sigma^*$ sont inséparables, alors, pour toute lettre $x \in \Sigma$, les mots wx et $w'x$ sont aussi inséparables.

Ainsi, il y a un sens à définir une fonction $(\Sigma^* / \equiv) \times \Sigma \rightarrow (\Sigma^* / \equiv)$ par

$$\delta([w], x) = [wx].$$

Démonstration (Sens $\Leftarrow$)

Observation clé

Si les mots w et $w' \in \Sigma^*$ sont inséparables, alors, pour toute lettre $x \in \Sigma$, les mots wx et $w'x$ sont aussi inséparables.

Ainsi, il y a un sens à définir une fonction $(\Sigma^* / \equiv) \times \Sigma \rightarrow (\Sigma^* / \equiv)$ par

$$\delta([w], x) = [wx].$$

Notons $q_0 = [\varepsilon]$ et $F = \{[u], u \in L\}$.

Démonstration (Sens $\Leftarrow$)

Observation clé

Si les mots w et $w' \in \Sigma^*$ sont inséparables, alors, pour toute lettre $x \in \Sigma$, les mots wx et $w'x$ sont aussi inséparables.

Ainsi, il y a un sens à définir une fonction $(\Sigma^* / \equiv) \times \Sigma \rightarrow (\Sigma^* / \equiv)$ par

$$\delta([w], x) = [wx].$$

Notons $q_0 = [\varepsilon]$ et $F = \{[u], u \in L\}$.

Quand $Q = (\Sigma^* / \equiv)$ est fini, on a trouvé un automate (Q, q_0, δ, F) à k états qui reconnaît $\mathcal{L}$. En effet, pour tout mot,

$$\delta(q_0, w) = [w].$$

Démonstration (Sens $\Rightarrow$)

Soit $\mathcal{B} = (Q_{\mathcal{B}}, q_{0,\mathcal{B}}, F_{\mathcal{B}}, \delta_{\mathcal{B}})$ un automate reconnaissant $\mathcal{L}$.

Démonstration (Sens $\Rightarrow$)

Soit $\mathcal{B} = (Q_{\mathcal{B}}, q_{0,\mathcal{B}}, F_{\mathcal{B}}, \delta_{\mathcal{B}})$ un automate reconnaissant $\mathcal{L}$.

Si $\delta_{\mathcal{B}}^*(q_{0,\mathcal{B}}, w) = \delta_{\mathcal{B}}^*(q_{0,\mathcal{B}}, w')$, alors w et w' sont inséparables.

Démonstration (Sens $\Rightarrow$)

Soit $\mathcal{B} = (Q_{\mathcal{B}}, q_{0,\mathcal{B}}, F_{\mathcal{B}}, \delta_{\mathcal{B}})$ un automate reconnaissant $\mathcal{L}$.

Si $\delta_{\mathcal{B}}^*(q_{0,\mathcal{B}}, w) = \delta_{\mathcal{B}}^*(q_{0,\mathcal{B}}, w')$, alors w et w' sont inséparables.

Reformulons : en notant $\equiv_{\mathcal{B}}$ la relation d'équivalence sur Σ^* définie par $w \equiv_{\mathcal{B}} w'$ si $\delta_{\mathcal{B}}^*(q_{0,\mathcal{B}}, w) = \delta_{\mathcal{B}}^*(q_{0,\mathcal{B}}, w')$, la relation $\equiv_{\mathcal{B}}$ est plus fine que $\equiv$ et possède au plus $|Q_{\mathcal{B}}|$ classes.

Démonstration (Sens $\Rightarrow$)

Soit $\mathcal{B} = (Q_{\mathcal{B}}, q_{0,\mathcal{B}}, F_{\mathcal{B}}, \delta_{\mathcal{B}})$ un automate reconnaissant $\mathcal{L}$.

Si $\delta_{\mathcal{B}}^*(q_{0,\mathcal{B}}, w) = \delta_{\mathcal{B}}^*(q_{0,\mathcal{B}}, w')$, alors w et w' sont inséparables.

Reformulons : en notant $\equiv_{\mathcal{B}}$ la relation d'équivalence sur Σ^* définie par $w \equiv_{\mathcal{B}} w'$ si $\delta_{\mathcal{B}}^*(q_{0,\mathcal{B}}, w) = \delta_{\mathcal{B}}^*(q_{0,\mathcal{B}}, w')$, la relation $\equiv_{\mathcal{B}}$ est plus fine que $\equiv$ et possède au plus $|Q_{\mathcal{B}}|$ classes.

Par conséquent $|Q| = |\Sigma^* / \equiv|$ satisfait $|Q| \leq |Q_{\mathcal{B}}|$ et est fini.

Démonstration (Sens $\Rightarrow$)

Soit $\mathcal{B} = (Q_{\mathcal{B}}, q_{0,\mathcal{B}}, F_{\mathcal{B}}, \delta_{\mathcal{B}})$ un automate reconnaissant $\mathcal{L}$.

Si $\delta_{\mathcal{B}}^*(q_{0,\mathcal{B}}, w) = \delta_{\mathcal{B}}^*(q_{0,\mathcal{B}}, w')$, alors w et w' sont inséparables.

Reformulons : en notant $\equiv_{\mathcal{B}}$ la relation d'équivalence sur Σ^* définie par $w \equiv_{\mathcal{B}} w'$ si $\delta_{\mathcal{B}}^*(q_{0,\mathcal{B}}, w) = \delta_{\mathcal{B}}^*(q_{0,\mathcal{B}}, w')$, la relation $\equiv_{\mathcal{B}}$ est plus fine que $\equiv$ et possède au plus $|Q_{\mathcal{B}}|$ classes.

Par conséquent $|Q| = |\Sigma^* / \equiv|$ satisfait $|Q| \leq |Q_{\mathcal{B}}|$ et est fini.

Il existe par conséquent un unique automate minimal ou **automate canonique** qui reconnaît un langage rationnel.

1956

Idée générale

Fusionner les états. On commence par une « sur-fusion » trop grossière qu'on relâche petit à petit jusqu'à ce que cela marche.



Edward F. Moore
(1925+2003)

1956

Idée générale

Fusionner les états. On commence par une « sur-fusion » trop grossière qu'on relâche petit à petit jusqu'à ce que cela marche.

Algorithme

1. Démarrer avec un automate **déterministe complet** sans états inaccessibles



Edward F. Moore
(1925+2003)

1956

Idée générale

Fusionner les états. On commence par une « sur-fusion » trop grossière qu'on relâche petit à petit jusqu'à ce que cela marche.

Algorithme

1. Démarrer avec un automate **déterministe complet** sans états inaccessibles
2. Initialiser $\Pi = (F, Q \setminus F)$.



Edward F. Moore
(1925+2003)

1956



Edward F. Moore
(1925+2003)

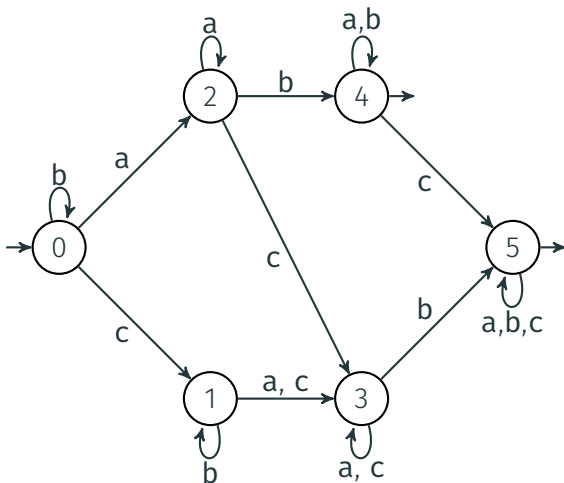
Idée générale

Fusionner les états. On commence par une « sur-fusion » trop grossière qu'on relâche petit à petit jusqu'à ce que cela marche.

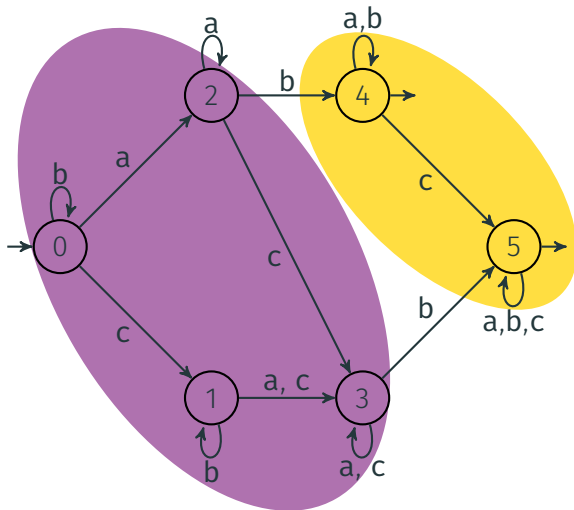
Algorithme

1. Démarrer avec un automate **déterministe complet** sans états inaccessibles
2. Initialiser $\Pi = (F, Q \setminus F)$.
3. Tant qu'il existe une part $C \in \Pi$ et une lettre $x \in \Sigma$ telle que pour $q \neq q'$, $\delta(q, x)$ et $\delta(q', x)$ tombent dans des parts différentes, **diviser** C en fonction de $\delta(\bullet, x)$.

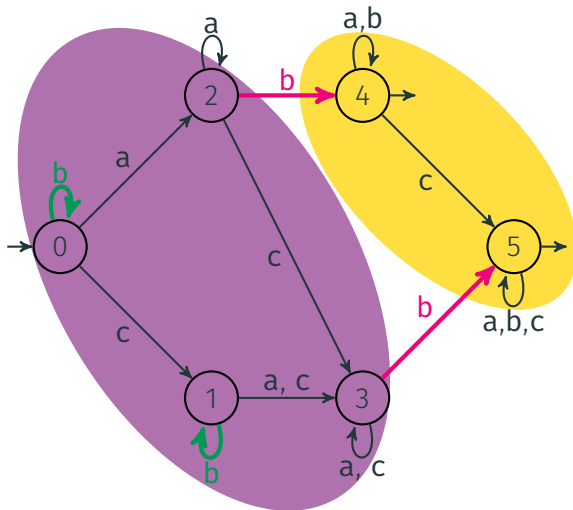
Exemple de minimisation



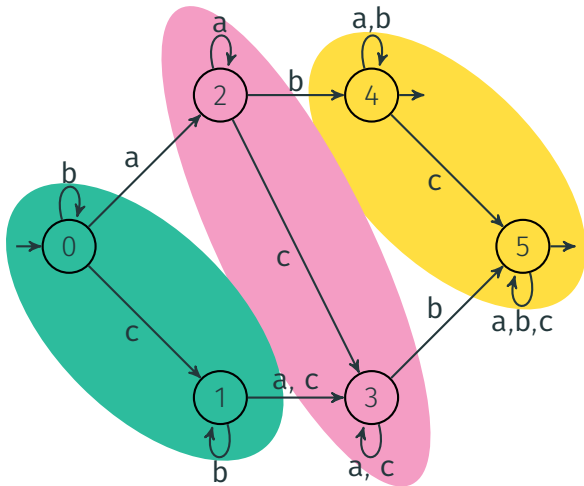
Exemple de minimisation



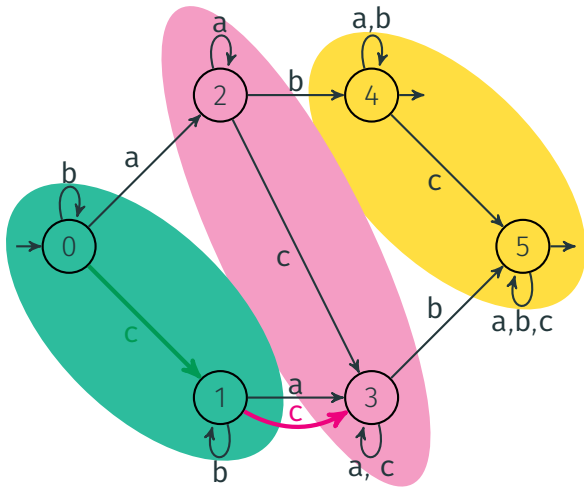
Exemple de minimisation



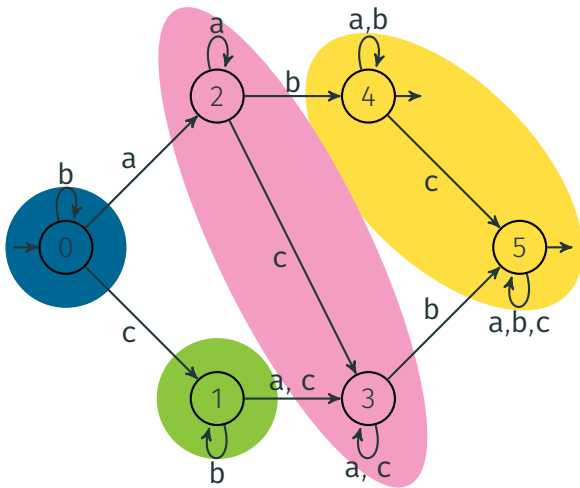
Exemple de minimisation



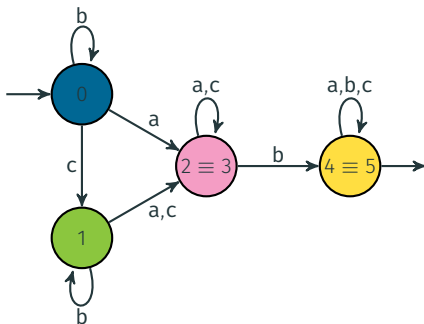
Exemple de minimisation



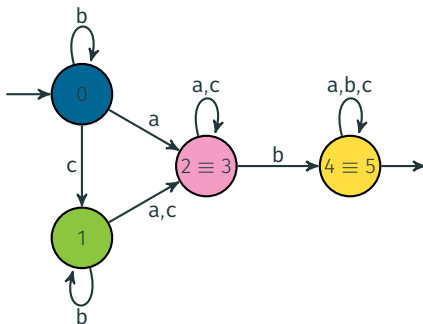
Exemple de minimisation



Exemple de minimisation

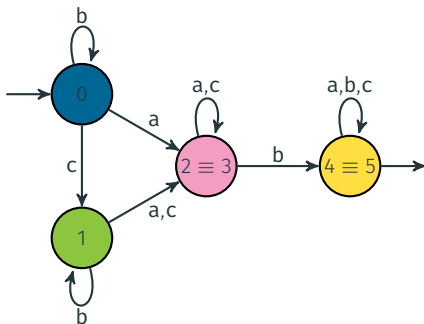


Exemple de minimisation



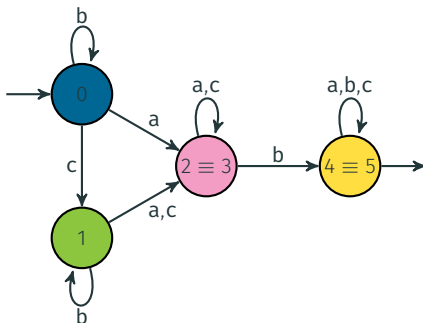
Etat	Mots de la classe	Résiduel de ces mots
Q	$\Sigma^* / \equiv$	$w^{-1}L$
0		

Exemple de minimisation



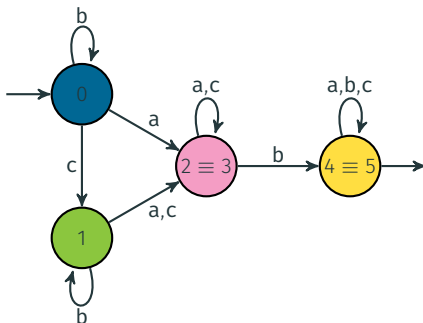
Etat	Mots de la classe	Résiduel de ces mots
Q	$\Sigma^* / \equiv$	$w^{-1}L$
0	b^*	

Exemple de minimisation



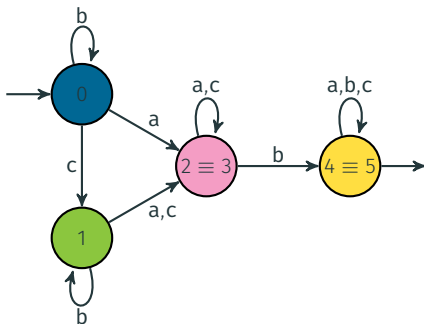
Etat Q	Mots de la classe $\Sigma^* / \equiv$	Résiduel de ces mots $w^{-1}L$
0	b^*	$(a cb^*(a c))(a c)^*b\Sigma^*$

Exemple de minimisation



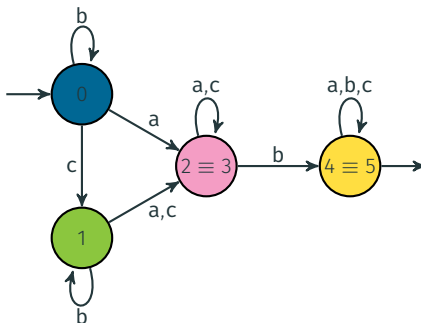
Etat Q	Mots de la classe $\Sigma^* / \equiv$	Résiduel de ces mots $w^{-1}L$
0	b^*	$(a cb^*(a c))(a c)^*b\Sigma^*$
1	b^*cb^*	$(a c)(a c)^*b\Sigma^*$

Exemple de minimisation



Etat Q	Mots de la classe $\Sigma^* / \equiv$	Résiduel de ces mots $w^{-1}L$
0	b^*	$(a cb^*(a c))(a c)^*b\Sigma^*$
1	b^*cb^*	$(a c)(a c)^*b\Sigma^*$
2, 3	$b^*(a cb^*(a c))(a c)^*$	$b\Sigma^*$

Exemple de minimisation



Etat Q	Mots de la classe $\Sigma^* / \equiv$	Résiduel de ces mots $w^{-1}L$
0	b^*	$(a cb^*(a c))(a c)^*b\Sigma^*$
1	b^*cb^*	$(a c)(a c)^*b\Sigma^*$
2, 3	$b^*(a cb^*(a c))(a c)^*$	$b\Sigma^*$
4, 5	$b^*(a cb^*(a c))(a c)^*b(a b c)^*$	Σ^*

Proposition

Étant données deux expressions rationnelles r et r' , on peut décider par un algorithme si r et r' **dénotent le même langage**.

Égalité de langages réguliers

Proposition

Étant données deux expressions rationnelles r et r' , on peut décider par un algorithme si r et r' dénotent le même langage.

Démonstration.

On construit deux automates minimaux $\mathcal{A}$ et $\mathcal{A}'$ reconnaissant les langages dénotés par r et r' .

Égalité de langages réguliers

Proposition

Étant données deux expressions rationnelles r et r' , on peut décider par un algorithme si r et r' **dénotent le même langage**.

Démonstration.

On construit deux **automates minimaux** $\mathcal{A}$ et $\mathcal{A}'$ reconnaissant les langages dénotés par r et r' .

On vérifie que $\mathcal{A}$ et $\mathcal{A}'$ sont **isomorphes** : par un parcours de graphe, on fait correspondre les états des deux automates par une bijection qui respecte les transitions. □